Mục lục

Mục lục **1**

Danh sách hình vẽ **4**

Danh sách bảng **7**

1 TỔNG QUAN **8**

1.1 Giới thiệu đề tài 8

1.1.1 Bối cảnh 8

1.1.2 Mục tiêu 9

1.2 Cơ sở lý thuyết 9

1.2.1 Kiến trúc Microservices 9

1.2.2 API Gateway Pattern 10

1.2.3 CQRS và Mediator Pattern 10

1.2.4 Kiến trúc Hướng sự kiện (Event-Driven Architecture) 11

1.2.5 Transactional Outbox Pattern 11

1.2.6 Change Data Capture (CDC) với Debezium 11

1.2.7 Tìm kiếm và Chỉ mục với Elasticsearch 12

1.2.8 .NET Aspire — Service Discovery và Orchestration 12

1.2.9 Hệ quản trị Cơ sở dữ liệu PostgreSQL và ORM 12

1.2.10 Hệ thống Thời gian thực với SignalR và Redis Backplane 13

1.2.11 Bảo mật, Thiết kế API và Ánh xạ dữ liệu 13

1.2.12 Tích hợp Cổng thanh toán Ngoại vi (Payment Gateway) 14

2 THIẾT KẾ HỆ THỐNG **15**

2.1 Phân tích yêu cầu hệ thống 15

2.1.1 Yêu cầu chức năng (Functional Requirements) 15

2.1.2 Yêu cầu phi chức năng (Non-Functional Requirements) 38

2.2 Thiết kế lược đồ dữ liệu tích hợp 40

2.2.1 Thiết kế Dữ liệu Property Service 40

2.2.2 Thiết kế Dữ liệu Booking Service 41

2.2.3 Thiết kế Dữ liệu Chat Service 42

2.2.4 Thiết kế Dữ liệu User Service 43

2.2.5 Thiết kế Dữ liệu Search Service (Elasticsearch) 44
2.2.6 Thiết kế Dữ liệu Payment Service 46
2.2.7 Cơ sở Dữ liệu Phân tán & Cấu trúc Outbox / Inbox Pattern 48
2.3 Sơ đồ Tuần tự Nghiệp vụ (Sequence Diagrams) 48
2.4 Kiến trúc tổng thể 93
2.4.1 Sơ đồ Kiến trúc Tổng thể 93
2.4.2 Cơ chế giao tiếp giữa các dịch vụ (Inter-service Communication) . . 95
2.4.3 Áp dụng Mẫu thiết kế (Design Patterns / Architectural Patterns) . . 97
2.5 Thiết kế cấu hình các node và mạng 105
2.5.1 Cấu trúc Phân vùng Mạng (Network Segmentation) 105
2.5.2 Cấu hình Điều phối bằng .NET Aspire 106

3 TRIỂN KHAI HỆ THỐNG 107
3.1 Môi trường và công nghệ triển khai 107
3.1.1 Ngôn ngữ và Framework 107
3.1.2 Cơ sở hạ tầng và Middleware 107
3.1.3 Môi trường phát triển và triển khai 108
3.1.4 Cấu hình điều phối .NET Aspire 108
3.2 Kết quả triển khai các chức năng chính 109
3.2.1 Xác thực và Quản lý tài khoản (UC-01 – UC-05) 109
3.2.2 Quản lý bất động sản (UC-06 – UC-15) 110
3.2.3 Tìm kiếm và Đặt phòng (UC-16 – UC-25) 111
3.2.4 Thanh toán và Hoàn tiền (UC-26 – UC-30) 113
3.2.5 Tin nhắn theo ngữ cảnh (UC-36 – UC-40) 114
3.2.6 Quản trị hệ thống (UC-41 – UC-45) 115
3.3 Đánh giá kết quả thực nghiệm 115
3.3.1 Mức độ hoàn thành Use Case 115
3.3.2 Đánh giá hiệu năng 116
3.3.3 Đánh giá tính nhất quán dữ liệu 117
3.3.4 Đánh giá bảo mật cơ bản 118

4 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 119
4.1 Những kết quả đã đạt được 119
4.2 Hạn chế và hướng phát triển 120
4.2.1 Hạn chế hiện tại 120
4.2.2 Hướng phát triển ngắn hạn (3–6 tháng) 121
4.2.3 Hướng phát triển dài hạn (>6 tháng) 121

TÀI LIỆU THAM KHẢO 122

A BẢNG PHÂN CÔNG CÔNG VIỆC **123**

Danh sách hình vẽ

2.1 Sơ đồ Use Case Tổng quát của hệ thống AirVnV 15
2.2 Sơ đồ Thực thể Liên kết (ERD) của Property Service 41
2.3 Sơ đồ Thực thể Liên kết (ERD) của Booking Service 42
2.4 Sơ đồ Thực thể Liên kết (ERD) của Chat Service 43
2.5 Sơ đồ Thực thể Liên kết (ERD) của User Service 44
2.6 Sơ đồ Lược đồ Tài liệu (Document Schema) của Search Service 45
2.7 Sơ đồ Thực thể Liên kết (ERD) của Payment Service 47
2.8 Sơ đồ Tuần tự: UC-01 Đăng ký tài khoản 49
2.9 Sơ đồ Tuần tự: UC-02 Xác thực Email 50
2.10 Sơ đồ Tuần tự: UC-03 Đăng nhập hệ thống 51
2.11 Sơ đồ Tuần tự: UC-04 Cập nhật hồ sơ cá nhân 52
2.12 Sơ đồ Tuần tự: UC-05 Đổi mật khẩu 53
2.13 Sơ đồ Tuần tự: UC-06 Quên mật khẩu 54
2.14 Sơ đồ Tuần tự: UC-07 Làm mới Token 55
2.15 Sơ đồ Tuần tự: UC-08 Nhận Thông báo 56
2.16 Sơ đồ Tuần tự: UC-09 Đăng tin Bất động sản mới 57
2.17 Sơ đồ Tuần tự: UC-10 Quản lý hình ảnh & tiện ích 58
2.18 Sơ đồ Tuần tự: UC-11 Thiết lập Giá theo ngày 59
2.19 Sơ đồ Tuần tự: UC-12 Khóa lịch rảnh 60
2.20 Sơ đồ Tuần tự: UC-13 Xóa Bất động sản 61
2.21 Sơ đồ Tuần tự: UC-14 Gửi duyệt Bất động sản 62
2.22 Sơ đồ Tuần tự: UC-15 Cập nhật trạng thái phòng 63
2.23 Sơ đồ Tuần tự: UC-16 Xem Dashboard Chủ nhà 64
2.24 Sơ đồ Tuần tự: UC-17 Phản hồi đánh giá 65
2.25 Sơ đồ Tuần tự: UC-18 Tìm kiếm Bất động sản 66
2.26 Sơ đồ Tuần tự: UC-19 Tìm kiếm gợi ý 67
2.27 Sơ đồ Tuần tự: UC-20 Xem chi tiết Bất động sản 68
2.28 Sơ đồ Tuần tự: UC-21 Đặt phòng 69
2.29 Sơ đồ Tuần tự: UC-22 Chủ nhà Duyệt/Từ chối đặt phòng 70
2.30 Sơ đồ Tuần tự: UC-23 Khách Hủy đặt phòng 71

2.31 Sơ đồ Tuần tự: UC-24 Chủ nhà Hủy đặt phòng 72
2.32 Sơ đồ Tuần tự: UC-25 Xem Lịch sử Đặt phòng 73
2.33 Sơ đồ Tuần tự: UC-26 Khởi tạo Thanh toán 74
2.34 Sơ đồ Tuần tự: UC-27 Xử lý Webhook Thanh toán 75
2.35 Sơ đồ Tuần tự: UC-28 Tự động Hoàn tiền 76
2.36 Sơ đồ Tuần tự: UC-29 Xem số dư ví Host 77
2.37 Sơ đồ Tuần tự: UC-30 Cập nhật Thông tin Ngân hàng 78
2.38 Sơ đồ Tuần tự: UC-31 Yêu cầu Rút tiền 79
2.39 Sơ đồ Tuần tự: UC-32 Xem Lịch sử Giao dịch 80
2.40 Sơ đồ Tuần tự: UC-33 Tạo cuộc hội thoại 81
2.41 Sơ đồ Tuần tự: UC-34 Gửi tin nhắn & File 82
2.42 Sơ đồ Tuần tự: UC-35 Đọc tin nhắn 83
2.43 Sơ đồ Tuần tự: UC-36 Lấy Lịch sử Tin nhắn 84
2.44 Sơ đồ Tuần tự: UC-37 Viết Đánh giá 85
2.45 Sơ đồ Tuần tự: UC-38 Sửa/Xóa Đánh giá 86
2.46 Sơ đồ Tuần tự: UC-39 Đăng nhập Admin 87
2.47 Sơ đồ Tuần tự: UC-40 Duyệt Bất động sản 88
2.48 Sơ đồ Tuần tự: UC-41 Khóa/Mở tài khoản 89
2.49 Sơ đồ Tuần tự: UC-42 Quản lý Danh mục Tiện ích 90
2.50 Sơ đồ Tuần tự: UC-43 Xử lý Khiếu nại 91
2.51 Sơ đồ Tuần tự: UC-44 Xem Báo cáo Thống kê 92
2.52 Sơ đồ Tuần tự: UC-45 Cập nhật Cấu hình Nền tảng 93
2.53 Sơ đồ Kiến trúc Tổng thể của hệ thống AirVnV 95
2.54 Sơ đồ Tuần tự minh họa giao tiếp Đồng bộ dữ liệu qua Kafka 96
2.55 Sơ đồ Tuần tự minh họa giao tiếp Đặt phòng qua RabbitMQ 97
2.56 Sơ đồ Lớp (Class Diagram) cấu trúc REPR Pattern 98
2.57 Sơ đồ Lớp (Class Diagram) cấu trúc CQRS sử dụng Mediator 98
2.58 Sơ đồ Tuần tự (Sequence Diagram) minh họa luồng xử lý CQRS 99
2.59 Sơ đồ Tuần tự (Sequence Diagram) cơ chế Transactional Outbox 99
2.60 Sơ đồ Tuần tự (Sequence Diagram) minh họa Saga Pattern giữa Booking và Payment 101
2.61 Sơ đồ Lớp (Class Diagram) mô hình Strategy Pattern tại PaymentService . . 101
2.62 Sơ đồ Lớp (Class Diagram) mô tả Aggregate Root trong DDD 102
2.63 Sơ đồ Lớp (Class Diagram) của Factory Method Pattern 103
2.64 Sơ đồ Triển khai (Deployment Diagram) các Node và Phân vùng mạng . . . 105

3.1 Giao diện Đăng ký tài khoản 110
3.2 Giao diện Đăng nhập 110

3.3 Giao diện Đăng tin bất động sản (Host) 111
3.4 Giao diện Chi tiết bất động sản 111
3.5 Giao diện Tìm kiếm bất động sản với bản đồ Leaflet 112
3.6 Giao diện đã Đặt phòng 113
3.7 Giao diện Thanh toán VNPay 114
3.8 Giao diện Tin nhắn theo ngữ cảnh (Chat) 114
3.9 Giao diện Dashboard Admin 115

Danh sách bảng

2.1 Yêu cầu phi chức năng của hệ thống AirVnV 39

3.1 Ngôn ngữ, Framework và thư viện chính 107
3.2 Phiên bản cơ sở hạ tầng container 108
3.3 Môi trường phát triển và công cụ 108
3.4 Kết quả đo hiệu năng các API chính 116

4.1 Bảng đối chiếu mục tiêu và kết quả thực hiện 119

CHƯƠNG 1
TỔNG QUAN

1.1 Giới thiệu đề tài

1.1.1 Bối cảnh

Trong những năm gần đây, thị trường cho thuê nhà ngắn hạn tại Việt Nam và trên toàn cầu đã chứng kiến sự tăng trưởng vượt bậc nhờ sự phổ biến của các nền tảng trực tuyến như Airbnb, Booking.com hay Agoda. Mô hình này không chỉ tạo ra nguồn thu nhập thụ động cho chủ nhà mà còn cung cấp cho khách du lịch những lựa chọn đa dạng, linh hoạt hơn so với khách sạn truyền thống.

Tuy nhiên, đằng sau giao diện người dùng đơn giản là những bài toán kỹ thuật phức tạp mà các hệ thống này phải giải quyết:

- **Tính toàn vẹn dữ liệu:** Một bất động sản không thể bị đặt trùng bởi hai khách hàng cùng lúc — đây là bài toán xử lý đồng thời (concurrency control) trong hệ thống phân tán.
- **Khả năng tìm kiếm theo địa lý:** Khách cần tìm phòng trong bán kính nhất định so với điểm du lịch — đòi hỏi hệ thống tìm kiếm địa không gian (geo-spatial search) hiệu năng cao.
- **Đồng bộ dữ liệu thời gian thực:** Khi chủ nhà cập nhật thông tin, kết quả tìm kiếm của khách phải phản ánh ngay lập tức mà không cần truy vấn trực tiếp vào cơ sở dữ liệu giao dịch.
- **Giao tiếp theo ngữ cảnh:** Hệ thống chat không đơn thuần là nhắn tin P2P mà phải được gắn chặt với ngữ cảnh nghiệp vụ (một bất động sản hoặc một đơn đặt phòng cụ thể).
- **Xử lý thanh toán an toàn:** Luồng thanh toán phải đảm bảo tính nguyên tử (atomicity) và không block hệ thống — kết quả thanh toán phải được xử lý bất đồng bộ.

Đề tài **AirVnV** ra đời từ nhu cầu nghiên cứu và xây dựng một nền tảng cho thuê nhà ngắn hạn hiện đại, áp dụng kiến trúc phân tán để giải quyết các thách thức nêu trên trong điều kiện thực tế. Đây không phải là bài toán học thuật đơn thuần mà là một bài toán kỹ thuật phần mềm có tính ứng dụng cao.

1.1.2 Mục tiêu

Mục tiêu của đồ án là xây dựng một nền tảng đặt phòng trực tuyến hoàn chỉnh — từ backend đến frontend — với các chức năng cốt lõi gồm: quản lý bất động sản, tìm kiếm theo địa lý, đặt phòng, thanh toán và chat theo ngữ cảnh.

Bên cạnh việc hoàn thiện sản phẩm về mặt chức năng, nhóm tập trung vào việc giải quyết các bài toán kỹ thuật thực tế trong hệ thống phân tán:

- Áp dụng kiến trúc Microservices — mỗi nghiệp vụ là một service độc lập, có thể phát triển và triển khai riêng lẻ.
- Đảm bảo tính nhất quán dữ liệu giữa các service mà không dùng distributed transaction, thông qua Transactional Outbox Pattern và Change Data Capture (CDC).
- Xây dựng hệ thống tìm kiếm địa lý hiệu năng cao bằng Elasticsearch — kết quả tìm kiếm không truy vấn trực tiếp vào database giao dịch.
- Xây dựng hệ thống chat real-time gắn với ngữ cảnh đặt phòng, tự động phát sinh thông báo hệ thống khi có sự kiện nghiệp vụ.
- Thiết kế luồng thanh toán bất đồng bộ — kết quả thanh toán không block luồng chính mà được xử lý qua message broker.

Phạm vi

Đồ án bao gồm ba nhóm người dùng: Khách (Guest), Chủ nhà (Host) và Quản trị viên (Admin). Ứng dụng mobile native nằm ngoài phạm vi của đồ án này.

1.2 Cơ sở lý thuyết

Để giải quyết các bài toán đặt ra, đồ án áp dụng một tập hợp các công nghệ và kiến trúc hiện đại. Phần này trình bày nền tảng lý thuyết của từng thành phần và lý do lựa chọn.

1.2.1 Kiến trúc Microservices

Kiến trúc Microservices phân chia hệ thống thành các service nhỏ, độc lập, mỗi service chịu trách nhiệm về một miền nghiệp vụ duy nhất và có cơ sở dữ liệu riêng (Database-per-Service). Ngược lại với kiến trúc Monolith, Microservices cho phép:

- Triển khai độc lập từng service mà không cần build lại toàn bộ hệ thống.
- Scale theo chiều ngang (horizontal scaling) có chọn lọc — chỉ scale service đang chịu tải cao.
- Cách ly lỗi (fault isolation): lỗi ở một service không làm sập toàn bộ hệ thống.

Trong AirVnV, hệ thống được phân rã thành 7 service: *UserService, PropertyService, SearchService, BookingService, PaymentService, ChatService* và *Gateway*.

1.2.2 API Gateway Pattern

API Gateway là một điểm vào duy nhất (single entry point) cho tất cả các yêu cầu từ client. Gateway đảm nhận các trách nhiệm:

- **Routing:** Định tuyến request đến đúng service.
- **Authentication:** Xác thực JWT token trước khi forward request.
- **Load Balancing:** Phân phối tải đến các instance của cùng một service.

Đồ án sử dụng **YARP** (Yet Another Reverse Proxy) — một thư viện reverse proxy hiệu năng cao được phát triển bởi Microsoft, tích hợp tốt với .NET ecosystem và .NET Aspire.

1.2.3 CQRS và Mediator Pattern

CQRS (Command Query Responsibility Segregation) là pattern tách biệt hoàn toàn luồng ghi dữ liệu (Command) và luồng đọc dữ liệu (Query). Mỗi thao tác nghiệp vụ được biểu diễn bằng một object (Command hoặc Query) và xử lý bởi một Handler tương ứng.

- **Command:** Thay đổi trạng thái hệ thống. Ví dụ: `CreateBookingCommand`, `PublishPropertyCo`
- **Query:** Chỉ đọc dữ liệu, không có side-effect. Ví dụ: `GetPropertyQuery`, `SearchPropertiesQu`

Để triển khai CQRS một cách hiệu quả và giảm thiểu sự phụ thuộc chéo (coupling) giữa các class, đồ án áp dụng **Mediator Pattern** (thông qua thư viện `Mediator.SourceGenerator`). Mediator Pattern đóng vai trò là một ”người điều phối” trung tâm. Thay vì các Endpoint/Controller phải tiêm (inject) hàng loạt các Service/Repository gây ra tình trạng ”God object”, chúng chỉ cần gửi (Send) một `Request` tới Mediator. Mediator sẽ dựa vào kiểu dữ liệu của Request để tự động định tuyến tới đúng `Handler` duy nhất chịu trách nhiệm xử lý.

Sự kết hợp giữa CQRS, Mediator Pattern và **FastEndpoints** giúp hệ thống hiện thực hóa mô hình kiến trúc cắt dọc (**Vertical Slice Architecture**). Theo đó, mỗi Use Case nghiệp vụ được gom cụm độc lập trong một thư mục duy nhất (gồm `Endpoint.cs, Request.cs, Response.cs, Handler.cs`), tối đa hóa tính gắn kết nội bộ (high cohesion) và hạn chế tối đa rủi ro khi thay đổi mã nguồn.

1.2.4 Kiến trúc Hướng sự kiện (Event-Driven Architecture)

Event-Driven Architecture (EDA) là mô hình các service giao tiếp với nhau thông qua việc phát và tiêu thụ sự kiện (events) bất đồng bộ thay vì gọi trực tiếp (synchronous HTTP).

Trong AirVnV, hai message broker được sử dụng với vai trò khác nhau:

- **RabbitMQ:** Dùng cho Domain Events nội bộ — các sự kiện nghiệp vụ ngắn hạn giữa các service (ví dụ: `BookingConfirmedEvent` kích hoạt ChatService inject system message).
- **Apache Kafka:** Dùng cho Data Streaming — luồng dữ liệu lớn, bền vững, hỗ trợ replay. Cụ thể là luồng CDC từ PostgreSQL đến Elasticsearch.

1.2.5 Transactional Outbox Pattern

Một thách thức cốt lõi của Microservices là: làm sao đảm bảo rằng khi một nghiệp vụ thành công (ví dụ: lưu Booking vào DB), sự kiện tương ứng (ví dụ: `BookingCreatedEvent`) luôn được publish đến message broker — kể cả khi ứng dụng crash ngay sau khi lưu DB?

Outbox Pattern giải quyết vấn đề này bằng cách lưu event vào một bảng `OutboxMessage` trong *cùng một transaction* với nghiệp vụ chính. Một background worker riêng biệt sẽ đọc bảng Outbox và publish event đến broker, đảm bảo tính nguyên tử (atomicity) mà không cần distributed transaction.

1.2.6 Change Data Capture (CDC) với Debezium

CDC là kỹ thuật theo dõi các thay đổi trong cơ sở dữ liệu (INSERT, UPDATE, DELETE) ở mức transaction log và phát chúng ra ngoài như các sự kiện.

Đồ án sử dụng **Debezium** — một nền tảng CDC mã nguồn mở — để đọc Write-Ahead Log (WAL) của PostgreSQL và publish các thay đổi vào Kafka. SearchService tiêu thụ luồng Kafka này để cập nhật Elasticsearch, đảm bảo read-model luôn đồng bộ gần thời gian thực với dữ liệu giao dịch.

1.2.7 Tìm kiếm và Chỉ mục với Elasticsearch

Elasticsearch là một công cụ tìm kiếm và phân tích phân tán (Distributed Search Engine) dựa trên Apache Lucene. Thay vì lưu dữ liệu dạng bảng như RDBMS và sử dụng phép toán `LIKE` vốn cực kỳ chậm chạp trên tập dữ liệu lớn, Elasticsearch sử dụng cấu trúc **Inverted Index** (Chỉ mục ngược). Cấu trúc này ánh xạ từng từ khóa (token) ngược lại với các tài liệu (documents) chứa từ khóa đó, giúp tốc độ tìm kiếm văn bản toàn văn (Full-text Search) nhanh gấp hàng trăm lần SQL.

Bên cạnh đó, Elasticsearch hỗ trợ mạnh mẽ kiểu dữ liệu `geo_point`, cho phép thực hiện các truy vấn không gian (Geo-Spatial) phức tạp như:

- **Geo Distance Query:** Tìm tất cả bất động sản trong bán kính r km từ một tọa độ GPS (lat, lon).
- **Geo Bounding Box:** Lọc các bất động sản nằm lọt thỏm trong một vùng khung chữ nhật trên bản đồ.

Thông qua thư viện client `Aspire.Elastic.Clients.Elasticsearch`, `SearchService` thao tác trực tiếp với Elasticsearch dưới dạng Read-Model. Việc tách biệt hoàn toàn cơ sở dữ liệu đọc (Elasticsearch) và ghi (PostgreSQL) giúp hiệu năng tìm kiếm đạt được tốc độ cực cao, $O(1)$, bất chấp sự gia tăng của lượng dữ liệu.

1.2.8 .NET Aspire — Service Discovery và Orchestration

.NET Aspire là nền tảng orchestration của Microsoft dành cho ứng dụng phân tán. Thay vì hardcode connection string và URL giữa các service, Aspire cung cấp:

- **Service Discovery:** Các service tìm thấy nhau bằng tên logic (ví dụ: `http://property-service` thay vì IP/port cứng.
- **Infrastructure Provisioning:** Aspire tự động khởi động các container (PostgreSQL, Redis, Kafka, RabbitMQ, Elasticsearch) khi chạy môi trường phát triển.
- **Observability Dashboard:** Giao diện theo dõi logs, metrics và traces tập trung cho toàn bộ hệ thống.

1.2.9 Hệ quản trị Cơ sở dữ liệu PostgreSQL và ORM

PostgreSQL được lựa chọn làm hệ quản trị cơ sở dữ liệu quan hệ (RDBMS) chính cho các Microservices. Lý do chính bao gồm: tính tuân thủ ACID nghiêm ngặt giúp đảm bảo toàn

vẹn dữ liệu tài chính (Booking, Payment), và khả năng hỗ trợ kiểu dữ liệu `JSONB` mạnh mẽ. `JSONB` cho phép lưu trữ các trường dữ liệu bán cấu trúc (semi-structured data) như danh sách tiện ích (amenities) hoặc bộ lọc tìm kiếm một cách linh hoạt mà không cần chuẩn hóa quá mức (over-normalization).

Để giao tiếp với PostgreSQL, đồ án sử dụng **Entity Framework Core** (EF Core) thông qua thư viện `Npgsql.EntityFrameworkCore.PostgreSQL`. EF Core là một Object-Relational Mapper (ORM) mạnh mẽ, cung cấp cơ chế Code-First Migrations, giúp tự động hóa việc tạo và cập nhật schema cơ sở dữ liệu. Để đảm bảo nguyên tắc *Database-per-service*, mỗi Microservice duy trì một lịch sử Migration độc lập và tự động apply vào Logical Database của riêng nó ngay khi khởi động.

1.2.10 Hệ thống Thời gian thực với SignalR và Redis Backplane

Chức năng nhắn tin (Chat) và thông báo (Notification) đòi hỏi kết nối hai chiều liên tục giữa Server và Client với độ trễ cực thấp. Để đáp ứng nhu cầu này, đồ án sử dụng **SignalR** (`Microsoft.AspNetCore.SignalR`). SignalR tự động quản lý các kết nối WebSocket (và fallback xuống Server-Sent Events hoặc Long Polling nếu cần), cho phép máy chủ đẩy dữ liệu (push) trực tiếp đến các client đang kết nối.

Tuy nhiên, trong môi trường phân tán (Docker/Kubernetes), `ChatService` có thể được nhân bản (scale-out) thành nhiều node. Nếu User A kết nối tới Node 1 và User B kết nối tới Node 2, Node 1 sẽ không biết cách gửi tin nhắn trực tiếp cho User B. Để giải quyết rào cản này, **Redis Backplane** (`StackExchange.Redis`) được tích hợp. Mọi tin nhắn SignalR gửi ra sẽ được đẩy vào kênh Redis Pub/Sub; sau đó Redis sẽ broadcast tin nhắn này tới tất cả các node của `ChatService`, đảm bảo tin nhắn luôn đến đúng người nhận bất kể họ đang duy trì socket với node nào.

1.2.11 Bảo mật, Thiết kế API và Ánh xạ dữ liệu

- **Thiết kế API siêu tốc (FastEndpoints):** Thay vì sử dụng mô hình MVC/Controllers truyền thống vốn cồng kềnh, hệ thống sử dụng thư viện `FastEndpoints`. Thư viện này triển khai kiến trúc REPR (Request-Endpoint-Response), giúp nhóm (group) tất cả logic nghiệp vụ, validation và routing của một API vào một file duy nhất, tối ưu tốc độ thực thi và cực kỳ dễ dàng bảo trì.
- **Bảo mật & Xác thực (Security):** Hệ thống kết hợp **JSON Web Token (JWT)** để xác thực không trạng thái (stateless authentication) qua YARP Gateway. Mật khẩu người dùng được băm (hash) an toàn bằng thuật toán bcrypt qua thư viện `BCrypt.Net-Next`.

Đồng thời, đồ án tích hợp hệ thống đăng nhập một chạm (SSO) qua Google nhờ các thư viện `FirebaseAdmin` và `Google.Apis.Auth`.

- **Ánh xạ dữ liệu (Mapster):** Trong kiến trúc sạch, dữ liệu phải được ánh xạ liên tục giữa Entity và DTO. Thay vì dùng AutoMapper (dựa trên Reflection làm suy giảm hiệu năng), hệ thống sử dụng `Mapster`. Mapster sinh mã trực tiếp lúc biên dịch (code generation), giúp tốc độ chuyển đổi dữ liệu đạt mức tối đa.

1.2.12 Tích hợp Cổng thanh toán Ngoại vi (Payment Gateway)

Luồng thanh toán trong AirVnV không lưu trữ trực tiếp thông tin thẻ tín dụng nhằm giảm tải rủi ro pháp lý và đảm bảo tuân thủ tiêu chuẩn PCI-DSS. Thay vào đó, `PaymentService` tích hợp với nền tảng **Stripe** thông qua thư viện `Stripe.net` (hoặc VNPay).

Khi khách hàng khởi tạo thanh toán, hệ thống sẽ sinh ra một Payment Session và điều hướng người dùng sang giao diện an toàn của đối tác. Mấu chốt của kiến trúc thanh toán phân tán là cơ chế **Webhook**: Hệ thống thanh toán sẽ gọi ngược (callback) về hệ thống AirVnV thông qua một API đặc biệt được bảo mật bằng chữ ký số (Webhook Signature) để báo cáo kết quả giao dịch. Nhờ luồng xử lý này, hệ thống backend không bao giờ bị ”treo” (block) để chờ kết quả giao dịch, đảm bảo tính bất đồng bộ và chống rớt đơn hàng hiệu quả kể cả khi kết nối mạng từ client bị ngắt ngang.

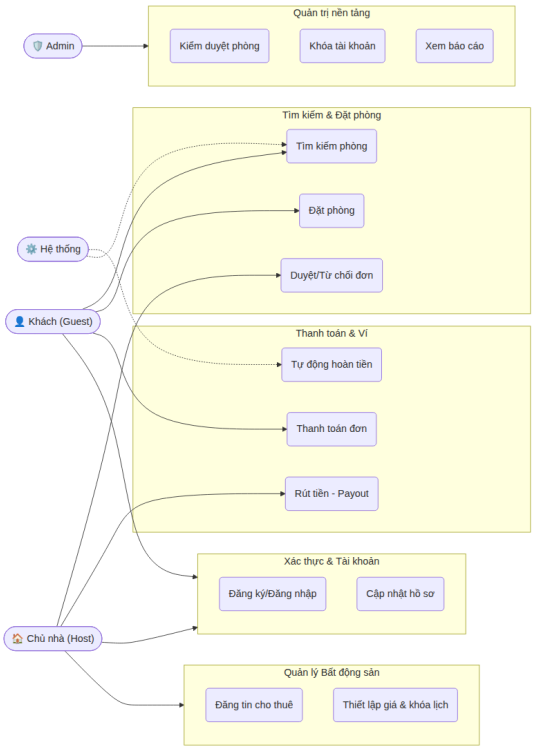
CHƯƠNG 2
THIẾT KẾ HỆ THỐNG

2.1 Phân tích yêu cầu hệ thống

2.1.1 Yêu cầu chức năng (Functional Requirements)

Sơ đồ Use Case Tổng thể (Overview Use Case Diagram)

Sơ đồ dưới đây minh họa các nhóm nghiệp vụ chính của hệ thống AirVnV, được phân bổ cho 4 nhóm Tác nhân (Actors) cốt lõi: Khách (Guest), Chủ nhà (Host), Quản trị viên (Admin), và Hệ thống (System).



Hình 2.1. Sơ đồ Use Case Tổng quát của hệ thống AirVnV

Đặc tả Use Case Chi tiết

Dưới đây là đặc tả chi tiết 45 luồng nghiệp vụ cốt lõi (Use Cases) của hệ thống AirVnV, được ánh xạ trực tiếp từ các tài nguyên API (Endpoints) và được chia thành 7 nhóm chức năng

(Features) độc lập nhằm đáp ứng yêu cầu của nền tảng phân tán.

Mã Usecase	**UC-01**
Tên Usecase	**Đăng ký tài khoản**
Tác nhân (Actors)	Khách (Guest), Chủ nhà (Host)
Mô tả ngắn	Người dùng đăng ký tài khoản mới trên hệ thống AirVnV.
Điều kiện tiên quyết	Người dùng chưa đăng nhập.
Điều kiện đảm bảo	Tài khoản được tạo ở trạng thái Inactive, OTP được gửi qua email.
Luồng sự kiện chính	1. Người dùng nhập thông định cơ bản. 2. Hệ thống kiểm tra email. 3. Hệ thống tạo user và gửi OTP.
Luồng rẽ nhánh	**Email đã tồn tại**: Hệ thống báo lỗi Email already exists.
Yêu cầu chức năng (FR)	◇ **FR_01.1**: Hệ thống từ chối nếu email trùng lặp.

Mã Usecase	**UC-02**
Tên Usecase	**Xác thực Email (Verify)**
Tác nhân (Actors)	Người dùng
Mô tả ngắn	Người dùng nhập mã OTP để kích hoạt tài khoản.
Điều kiện tiên quyết	Tài khoản ở trạng thái Inactive, có mã OTP hợp lệ.
Điều kiện đảm bảo	Tài khoản chuyển sang trạng thái Active.
Luồng sự kiện chính	1. Người dùng nhập OTP. 2. Hệ thống kiểm tra tính hợp lệ. 3. Cập nhật trạng thái Active.
Luồng rẽ nhánh	**Mã hết hạn**: Hệ thống báo lỗi Verification code expired.
Yêu cầu chức năng (FR)	◇ **FR_02.1**: Mã OTP chỉ có hiệu lực trong 15 phút.

Mã Usecase	**UC-03**
Tên Usecase	**Đăng nhập hệ thống**
Tác nhân (Actors)	Người dùng
Mô tả ngắn	Người dùng đăng nhập bằng mật khẩu hoặc Google SSO.
Điều kiện tiên quyết	Tài khoản đã kích hoạt.
Điều kiện đảm bảo	Nhận được JWT Token để truy cập hệ thống.
Luồng sự kiện chính	1. Người dùng nhập email/mật khẩu. 2. Hệ thống kiểm tra hash. 3. Hệ thống cấp phát JWT Token.
Luồng rẽ nhánh	**Sai mật khẩu**: Báo lỗi Invalid email or password.
Yêu cầu chức năng (FR)	◇ **FR_03.1**: Mật khẩu phải được mã hóa trước khi so sánh.

Mã Usecase	**UC-04**
Tên Usecase	**Cập nhật hồ sơ cá nhân**
Tác nhân (Actors)	Người dùng
Mô tả ngắn	Thay đổi thông tin như tên, ảnh đại diện, số điện thoại.
Điều kiện tiên quyết	Người dùng đã đăng nhập.
Điều kiện đảm bảo	Thông tin mới được lưu trữ trong CSDL.
Luồng sự kiện chính	1. Người dùng sửa thông tin. 2. Hệ thống validate. 3. Cập nhật CSDL.
Luồng rẽ nhánh	**Lỗi định dạng**: Báo lỗi Validation Failed (400).
Yêu cầu chức năng (FR)	◇ **FR_04.1**: Số điện thoại phải đúng định dạng quốc gia.

Mã Usecase	**UC-05**
Tên Usecase	**Đổi mật khẩu**
Tác nhân (Actors)	Người dùng
Mô tả ngắn	Người dùng đang đăng nhập muốn đổi mật khẩu mới.
Điều kiện tiên quyết	Đã đăng nhập.
Điều kiện đảm bảo	Mật khẩu được cập nhật, các phiên đăng nhập khác bị đăng xuất.
Luồng sự kiện chính	1. Nhập mật khẩu cũ và mới. 2. Hệ thống kiểm tra. 3. Cập nhật mật khẩu.
Luồng rẽ nhánh	**Sai mật khẩu cũ**: Báo lỗi và từ chối cập nhật.
Yêu cầu chức năng (FR)	◇ **FR_05.1**: Mật khẩu mới phải mạnh (có ký tự đặc biệt, hoa, thường).

Mã Usecase	**UC-06**
Tên Usecase	**Quên mật khẩu**
Tác nhân (Actors)	Người dùng
Mô tả ngắn	Yêu cầu khôi phục mật khẩu qua email.
Điều kiện tiên quyết	Không yêu cầu đăng nhập.
Điều kiện đảm bảo	Gửi link reset kèm token vào email.
Luồng sự kiện chính	1. Người dùng nhập email. 2. Hệ thống tạo Reset Token. 3. Gửi email chứa link khôi phục.
Luồng rẽ nhánh	**Email không tồn tại**: Trả về 200 OK nhưng không gửi mail để tránh lộ thông tin.
Yêu cầu chức năng (FR)	◇ **FR_06.1**: Token reset chỉ có hiệu lực 1 lần duy nhất trong 30 phút.

Mã Usecase	**UC-07**
Tên Usecase	**Làm mới Token (Refresh Token)**
Tác nhân (Actors)	Người dùng
Mô tả ngắn	Lấy JWT mới khi token cũ hết hạn.
Điều kiện tiên quyết	Sở hữu Refresh Token hợp lệ.
Điều kiện đảm bảo	Cấp JWT mới và Refresh Token mới.
Luồng sự kiện chính	1. Client gửi Refresh Token. 2. Hệ thống kiểm tra. 3. Trả về bộ token mới.
Luồng rẽ nhánh	**Token bị thu hồi**: Báo lỗi Token is revoked và yêu cầu đăng nhập lại.
Yêu cầu chức năng (FR)	◇ **FR_07.1**: Cơ chế xoay vòng (Rotation) Refresh Token để chống đánh cắp.

Mã Usecase	**UC-08**
Tên Usecase	**Nhận Thông báo (Notifications)**
Tác nhân (Actors)	Người dùng
Mô tả ngắn	Xem danh sách thông báo in-app và đánh dấu đã đọc.
Điều kiện tiên quyết	Đã đăng nhập.
Điều kiện đảm bảo	Danh sách thông báo được cập nhật trạng thái IsRead.
Luồng sự kiện chính	1. Lấy danh sách thông báo qua API. 2. Hiển thị lên chuông thông báo. 3. Đánh dấu là đã đọc.
Luồng rẽ nhánh	Không có.
Yêu cầu chức năng (FR)	◇ **FR_08.1**: Hệ thống đẩy thông báo realtime qua SignalR.

Mã Usecase	**UC-09**
Tên Usecase	**Đăng tin Bất động sản mới**
Tác nhân (Actors)	Chủ nhà (Host)
Mô tả ngắn	Khởi tạo một Bất động sản mới dưới dạng Draft.
Điều kiện tiên quyết	Chủ nhà đã đăng nhập.
Điều kiện đảm bảo	Bất động sản được tạo với trạng thái Draft.
Luồng sự kiện chính	1. Host nhập tiêu đề, mô tả. 2. Hệ thống validate. 3. Tạo bản ghi trong propdb.
Luồng rẽ nhánh	**Dữ liệu thiếu**: Báo lỗi Validation Failed.
Yêu cầu chức năng (FR)	◇ **FR_09.1**: Tiêu đề không được để trống và tối đa 100 ký tự.

Mã Usecase	**UC-10**
Tên Usecase	**Quản lý hình ảnh & tiện ích**
Tác nhân (Actors)	Chủ nhà (Host)
Mô tả ngắn	Host tải lên hình ảnh và chọn tiện ích (Wifi, Bể bơi).
Điều kiện tiên quyết	Bất động sản thuộc sở hữu của Host.
Điều kiện đảm bảo	Hình ảnh được lưu (Cloudinary/S3), tiện ích được map.
Luồng sự kiện chính	1. Host chọn ảnh/tiện ích. 2. Upload lên Cloud. 3. Lưu URL vào CSDL.
Luồng rẽ nhánh	**Sai định dạng ảnh**: Từ chối upload nếu không phải JPG/PNG.
Yêu cầu chức năng (FR)	◇ **FR_10.1**: Mỗi phòng phải có ít nhất 5 ảnh trước khi submit.

Mã Usecase	**UC-11**
Tên Usecase	**Thiết lập Giá theo ngày (Smart Pricing)**
Tác nhân (Actors)	Chủ nhà (Host)
Mô tả ngắn	Cấu hình giá cao hơn vào cuối tuần hoặc mùa cao điểm.
Điều kiện tiên quyết	Phòng đã được tạo.
Điều kiện đảm bảo	Cập nhật bảng giá tùy chỉnh.
Luồng sự kiện chính	1. Host chọn các ngày đặc biệt. 2. Nhập giá mới. 3. Lưu xuống cơ sở dữ liệu.
Luồng rẽ nhánh	**Giá không hợp lệ**: Báo lỗi nếu giá nhập < 0.
Yêu cầu chức năng (FR)	◇ **FR_11.1**: Hệ thống phải ưu tiên giá tùy chỉnh thay vì giá mặc định.

Mã Usecase	**UC-12**
Tên Usecase	**Khóa lịch rảnh (Block Dates)**
Tác nhân (Actors)	Chủ nhà (Host)
Mô tả ngắn	Host chủ động khóa một khoảng thời gian không cho khách đặt.
Điều kiện tiên quyết	Phòng đã được Publish.
Điều kiện đảm bảo	Khoảng thời gian bị chặn trong lịch Booking.
Luồng sự kiện chính	1. Chọn khoảng ngày. 2. Hệ thống kiểm tra xem có ai đã đặt chưa. 3. Tạo BlockAvailability.
Luồng rẽ nhánh	**Trùng lịch**: Báo lỗi không thể khóa ngày đã có khách đặt.
Yêu cầu chức năng (FR)	◇ **FR_12.1**: Từ chối block nếu ngày đó đang có Booking Pending/Confirmed.

Mã Usecase	**UC-13**
Tên Usecase	**Xóa Bất động sản**
Tác nhân (Actors)	Chủ nhà (Host)
Mô tả ngắn	Host xóa vĩnh viễn bài đăng đang lưu nháp.
Điều kiện tiên quyết	Phòng phải ở trạng thái Draft.
Điều kiện đảm bảo	Dữ liệu bị xóa mềm (Soft Delete).
Luồng sự kiện chính	1. Host chọn phòng nháp. 2. Bấm Xóa. 3. Cập nhật IsDeleted = true.
Luồng rẽ nhánh	**Đã publish**: Báo lỗi Cannot delete published property. Must suspend first.
Yêu cầu chức năng (FR)	◇ **FR_13.1**: Xóa toàn bộ hình ảnh trên Cloudinary tương ứng.

Mã Usecase	**UC-14**
Tên Usecase	**Gửi duyệt Bất động sản (Submit)**
Tác nhân (Actors)	Chủ nhà (Host)
Mô tả ngắn	Chuyển trạng thái từ Draft sang Pending Approval.
Điều kiện tiên quyết	Phòng có đủ ảnh, địa chỉ, tiện ích, giá.
Điều kiện đảm bảo	Trạng thái thành Pending Approval, gửi thông báo cho Admin.
Luồng sự kiện chính	1. Host nhấn Submit. 2. Hệ thống validate toàn diện. 3. Đổi trạng thái.
Luồng rẽ nhánh	**Thiếu dữ liệu**: Từ chối Submit nếu thiếu giá tiền hoặc địa chỉ.
Yêu cầu chức năng (FR)	◇ **FR_14.1**: Trạng thái chỉ chuyển được từ Draft sang Pending Approval.

Mã Usecase	**UC-15**
Tên Usecase	**Cập nhật trạng thái phòng**
Tác nhân (Actors)	Chủ nhà (Host)
Mô tả ngắn	Tạm ngưng (Suspend) hoặc Mở lại (Reinstate) phòng.
Điều kiện tiên quyết	Phòng đang ở trạng thái Published hoặc Suspended.
Điều kiện đảm bảo	Trạng thái phòng được cập nhật.
Luồng sự kiện chính	1. Host chọn thay đổi trạng thái. 2. Cập nhật DB. 3. Gửi Event sang SearchService.
Luồng rẽ nhánh	**Từ Draft**: Báo lỗi Cannot revert to Draft from current status.
Yêu cầu chức năng (FR)	◇ **FR_15.1**: Đồng bộ trạng thái này sang Elasticsearch qua Kafka.

Mã Usecase	**UC-16**
Tên Usecase	**Xem Dashboard Chủ nhà**
Tác nhân (Actors)	Chủ nhà (Host)
Mô tả ngắn	Host xem tổng quan thu nhập và lượt xem.
Điều kiện tiên quyết	Là Host.
Điều kiện đảm bảo	Trả về dữ liệu Analytics.
Luồng sự kiện chính	1. Lấy danh sách booking tháng này. 2. Lấy lượt view từ SearchService. 3. Hiển thị biểu đồ.
Luồng rẽ nhánh	Không có.
Yêu cầu chức năng (FR)	◇ **FR_16.1**: Dữ liệu Dashboard phải được cache trên Redis để tối ưu tốc độ.

Mã Usecase	**UC-17**
Tên Usecase	**Phản hồi đánh giá (Host Reply)**
Tác nhân (Actors)	Chủ nhà (Host)
Mô tả ngắn	Host bình luận lại bên dưới đánh giá của khách.
Điều kiện tiên quyết	Phòng có đánh giá hợp lệ.
Điều kiện đảm bảo	Lưu phản hồi và hiển thị công khai.
Luồng sự kiện chính	1. Host nhập nội dung reply. 2. Lưu vào DB. 3. Gửi thông báo cho khách.
Luồng rẽ nhánh	**Đã phản hồi**: Báo lỗi Host already replied to this review.
Yêu cầu chức năng (FR)	◇ **FR_17.1**: Nội dung phản hồi không chứa từ ngữ thô tục.

Mã Usecase	**UC-18**
Tên Usecase	**Tìm kiếm Bất động sản (Elasticsearch)**
Tác nhân (Actors)	Khách (Guest)
Mô tả ngắn	Tìm kiếm phòng trống theo địa điểm, giá, ngày.
Điều kiện tiên quyết	Không yêu cầu đăng nhập.
Điều kiện đảm bảo	Trả về danh sách phòng trống.
Luồng sự kiện chính	1. Khách nhập tiêu chí tìm kiếm. 2. SearchService truy vấn Elasticsearch. 3. Trả về kết quả phân trang.
Luồng rẽ nhánh	**Lỗi Elasticsearch**: Fallback về PostgreSQL (nếu có) hoặc báo lỗi 500.
Yêu cầu chức năng (FR)	◇ **FR_18.1**: Chỉ trả về các phòng có trạng thái Published.

Mã Usecase	**UC-19**
Tên Usecase	**Tìm kiếm gợi ý (Autocomplete)**
Tác nhân (Actors)	Khách (Guest)
Mô tả ngắn	Gợi ý tên thành phố hoặc địa danh khi khách gõ phím.
Điều kiện tiên quyết	Không yêu cầu đăng nhập.
Điều kiện đảm bảo	Trả về mảng chuỗi địa danh.
Luồng sự kiện chính	1. Khách gõ chữ cái đầu. 2. SearchService tìm gần đúng (Fuzzy Match). 3. Trả về danh sách top 5.
Luồng rẽ nhánh	Không có.
Yêu cầu chức năng (FR)	◇ **FR_19.1**: Thời gian phản hồi autocomplete phải < 50ms.

Mã Usecase	**UC-20**
Tên Usecase	**Xem chi tiết Bất động sản**
Tác nhân (Actors)	Khách (Guest)
Mô tả ngắn	Xem chi tiết phòng, giá, tiện ích và các review.
Điều kiện tiên quyết	Không yêu cầu.
Điều kiện đảm bảo	Dữ liệu chi tiết được hiển thị.
Luồng sự kiện chính	1. Chọn phòng. 2. Lấy dữ liệu từ PropertyService và ReviewService. 3. Hiển thị cho khách.
Luồng rẽ nhánh	**Phòng không tồn tại**: Báo lỗi Property not found (404).
Yêu cầu chức năng (FR)	◇ **FR_20.1**: Nếu phòng bị ẩn, hệ thống trả về 404 Not Found.

Mã Usecase	**UC-21**
Tên Usecase	**Đặt phòng (Create Booking)**
Tác nhân (Actors)	Khách (Guest)
Mô tả ngắn	Tạo yêu cầu đặt phòng cho một khoảng thời gian.
Điều kiện tiên quyết	Đã đăng nhập.
Điều kiện đảm bảo	Tạo đơn Pending, phát sự kiện sang PaymentService.
Luồng sự kiện chính	1. Khách chọn ngày và số người. 2. Hệ thống khóa lịch (Idempotent). 3. Tạo Booking Pending và bắn Event.
Luồng rẽ nhánh	**Ngày đã bị đặt**: Báo lỗi These dates are already booked.
Yêu cầu chức năng (FR)	◇ **FR_21.1**: Tổng chi phí = Số đêm x Giá đêm + Phí dịch vụ.

Mã Usecase	**UC-22**
Tên Usecase	**Chủ nhà Duyệt/Từ chối đặt phòng**
Tác nhân (Actors)	Chủ nhà (Host)
Mô tả ngắn	Host quyết định chấp nhận hoặc từ chối khách.
Điều kiện tiên quyết	Có đơn Booking ở trạng thái Pending Host Approval.
Điều kiện đảm bảo	Booking chuyển sang Approved hoặc Rejected.
Luồng sự kiện chính	1. Host xem đơn. 2. Chọn Duyệt. 3. Cập nhật trạng thái và thông báo Khách.
Luồng rẽ nhánh	**Đơn không tồn tại**: Báo lỗi Booking not found.
Yêu cầu chức năng (FR)	◇ **FR_22.1**: Host chỉ được thao tác trên các đơn thuộc Property của mình.

Mã Usecase	**UC-23**
Tên Usecase	**Khách Hủy đặt phòng**
Tác nhân (Actors)	Khách (Guest)
Mô tả ngắn	Khách chủ động hủy đơn.
Điều kiện tiên quyết	Đơn đang ở trạng thái Pending hoặc Confirmed.
Điều kiện đảm bảo	Booking chuyển sang Cancelled, kích hoạt hoàn tiền nếu cần.
Luồng sự kiện chính	1. Khách nhấn Hủy. 2. Kiểm tra chính sách hủy. 3. Cập nhật trạng thái và bắn RefundEvent.
Luồng rẽ nhánh	**Quá hạn hủy**: Từ chối hủy nếu vi phạm chính sách (ví dụ sát giờ check-in).
Yêu cầu chức năng (FR)	◇ **FR_23.1**: Hệ thống tự động tính số tiền hoàn lại dựa trên Policy.

Mã Usecase	**UC-24**
Tên Usecase	**Chủ nhà Hủy đặt phòng**
Tác nhân (Actors)	Chủ nhà (Host)
Mô tả ngắn	Host hủy đơn trong trường hợp khẩn cấp.
Điều kiện tiên quyết	Đơn đang ở trạng thái Confirmed.
Điều kiện đảm bảo	Hủy đơn, hoàn 100% tiền cho khách và phạt Host.
Luồng sự kiện chính	1. Host chọn hủy. 2. Ghi nhận lý do. 3. Bắn Event phạt Host và Refund cho khách.
Luồng rẽ nhánh	**Đã check-in**: Không thể hủy khi đã qua ngày Check-in.
Yêu cầu chức năng (FR)	◇ **FR_24.1**: Trừ tự động phí phạt (Penalty) vào tài khoản Host.

Mã Usecase	**UC-25**
Tên Usecase	**Xem Lịch sử Đặt phòng**
Tác nhân (Actors)	Khách (Guest)
Mô tả ngắn	Khách xem danh sách chuyến đi sắp tới và quá khứ.
Điều kiện tiên quyết	Đã đăng nhập.
Điều kiện đảm bảo	Trả về danh sách Booking.
Luồng sự kiện chính	1. Khách vào Trips. 2. Query BookingService. 3. Trả về dữ liệu phân trang.
Luồng rẽ nhánh	Không có.
Yêu cầu chức năng (FR)	◇ **FR_25.1**: Các đơn Upcoming phải hiển thị trên cùng.

Mã Usecase	**UC-26**
Tên Usecase	**Khởi tạo Thanh toán**
Tác nhân (Actors)	Hệ thống, Khách
Mô tả ngắn	Tạo Session thanh toán VNPay/Stripe.
Điều kiện tiên quyết	Đã có Booking ID.
Điều kiện đảm bảo	Cấp URL thanh toán cho khách.
Luồng sự kiện chính	1. PaymentService nhận BookingEvent. 2. Khởi tạo Session qua VNPay API. 3. Trả URL cho Client.
Luồng rẽ nhánh	**Đang xử lý**: Báo lỗi Payment is already being processed.
Yêu cầu chức năng (FR)	◇ **FR_26.1**: Phải gắn Idempotency Key để chống tạo trùng hóa đơn.

Mã Usecase	**UC-27**
Tên Usecase	**Xử lý Webhook Thanh toán**
Tác nhân (Actors)	Cổng thanh toán (VNPay)
Mô tả ngắn	Hệ thống nhận kết quả từ bên thứ ba.
Điều kiện tiên quyết	Giao dịch đã được khách thực hiện.
Điều kiện đảm bảo	Cập nhật trạng thái hóa đơn và báo cho BookingService.
Luồng sự kiện chính	1. Nhận payload IPN từ VNPay. 2. Xác thực chữ ký số. 3. Phát PaymentCompletedEvent.
Luồng rẽ nhánh	**Sai chữ ký**: Ghi log cảnh báo bảo mật và bỏ qua payload.
Yêu cầu chức năng (FR)	◇ **FR_27.1**: Bắt buộc kiểm tra Checksum (Secure Hash) trước khi xử lý.

Mã Usecase	**UC-28**
Tên Usecase	**Tự động Hoàn tiền (Refund)**
Tác nhân (Actors)	Hệ thống
Mô tả ngắn	Tự động hoàn tiền khi có lỗi hoặc khách hủy đúng luật.
Điều kiện tiên quyết	Giao dịch gốc đã thành công.
Điều kiện đảm bảo	Tiền được hoàn lại tài khoản khách.
Luồng sự kiện chính	1. Nhận RefundEvent. 2. Gọi API Hoàn tiền của VNPay. 3. Cập nhật trạng thái Refunded.
Luồng rẽ nhánh	**Cổng VNPay lỗi**: Lưu vào Outbox để retry sau.
Yêu cầu chức năng (FR)	◇ **FR_28.1**: Lưu trữ đầy đủ Transaction ID đối soát.

Mã Usecase	**UC-29**
Tên Usecase	**Xem số dư ví Host**
Tác nhân (Actors)	Chủ nhà (Host)
Mô tả ngắn	Host xem doanh thu và số tiền có thể rút.
Điều kiện tiên quyết	Đã đăng nhập.
Điều kiện đảm bảo	Trả về dữ liệu ví.
Luồng sự kiện chính	1. Truy vấn bảng Balance. 2. Hiển thị tổng thu nhập và Available Payout. 3. Trả về Client.
Luồng rẽ nhánh	Không có.
Yêu cầu chức năng (FR)	◇ **FR_29.1**: Chỉ cộng tiền vào ví sau khi khách Check-out thành công.

Mã Usecase	**UC-30**
Tên Usecase	**Cập nhật Thông tin Ngân hàng**
Tác nhân (Actors)	Chủ nhà (Host)
Mô tả ngắn	Host thêm số tài khoản ngân hàng để nhận Payout.
Điều kiện tiên quyết	Đã đăng nhập.
Điều kiện đảm bảo	Lưu thông tin thanh toán (đã mã hóa).
Luồng sự kiện chính	1. Host nhập STK, Tên ngân hàng. 2. Validate format. 3. Lưu vào DB.
Luồng rẽ nhánh	**Số tài khoản sai**: Báo lỗi Invalid Bank Account Format.
Yêu cầu chức năng (FR)	◇ **FR_30.1**: Số tài khoản phải được mã hóa mã hóa chuẩn AES256.

Mã Usecase	**UC-31**
Tên Usecase	**Yêu cầu Rút tiền (Payout)**
Tác nhân (Actors)	Chủ nhà, Admin
Mô tả ngắn	Host yêu cầu rút tiền và Admin duyệt chuyển khoản.
Điều kiện tiên quyết	Số dư lớn hơn mức tối thiểu.
Điều kiện đảm bảo	Tiền bị trừ khỏi ví, lệnh chuyển khoản được tạo.
Luồng sự kiện chính	1. Host tạo lệnh Payout. 2. Admin xem và duyệt. 3. Hệ thống MarkPayoutCompleted và trừ ví.
Luồng rẽ nhánh	**Số dư không đủ**: Hệ thống từ chối tạo lệnh.
Yêu cầu chức năng (FR)	◇ **FR_31.1**: Lệnh rút tiền phải sinh ra mã PayoutReference để đối soát.

Mã Usecase	**UC-32**
Tên Usecase	**Xem Lịch sử Giao dịch**
Tác nhân (Actors)	Chủ nhà (Host)
Mô tả ngắn	Hiển thị lịch sử cộng/trừ tiền trong ví.
Điều kiện tiên quyết	Đã đăng nhập.
Điều kiện đảm bảo	Trả về List Transaction.
Luồng sự kiện chính	1. Host mở Transaction History. 2. Query PaymentService DB. 3. Hiển thị phân trang.
Luồng rẽ nhánh	Không có.
Yêu cầu chức năng (FR)	◇ **FR_32.1**: Lịch sử phải không thể xóa hoặc sửa đổi (Immutable).

Mã Usecase	**UC-33**
Tên Usecase	**Tạo cuộc hội thoại**
Tác nhân (Actors)	Khách, Chủ nhà
Mô tả ngắn	Khách gửi yêu cầu chat với Host về một căn phòng.
Điều kiện tiên quyết	Khách đã đăng nhập.
Điều kiện đảm bảo	Tạo Conversation_Id trong MongoDB/Postgres.
Luồng sự kiện chính	1. Khách bấm Liên hệ Chủ nhà. 2. Tạo Conversation map với PropertyId. 3. Chuyển sang màn hình Chat.
Luồng rẽ nhánh	**Chat với chính mình**: Host cannot start a conversation as a guest for their own property.
Yêu cầu chức năng (FR)	◇ **FR_33.1**: Nếu Conversation giữa 2 user cho Property này đã có, trả về Id cũ.

Mã Usecase	**UC-34**
Tên Usecase	**Gửi tin nhắn & File**
Tác nhân (Actors)	Người dùng
Mô tả ngắn	Gửi text hoặc ảnh qua WebSocket.
Điều kiện tiên quyết	Nằm trong Conversation.
Điều kiện đảm bảo	Tin nhắn được lưu và push realtime qua SignalR/Socket.IO.
Luồng sự kiện chính	1. Nhập tin nhắn. 2. Lưu DB. 3. Broadcast tới người kia.
Luồng rẽ nhánh	**Không có quyền**: Báo lỗi You are not a participant in this conversation.
Yêu cầu chức năng (FR)	◇ **FR_34.1**: Tin nhắn hệ thống (System message) không thể được gửi bởi user.

Mã Usecase	**UC-35**
Tên Usecase	**Đọc tin nhắn (Mark As Read)**
Tác nhân (Actors)	Người dùng
Mô tả ngắn	Đánh dấu tin nhắn đã được xem.
Điều kiện tiên quyết	Có tin nhắn chưa đọc.
Điều kiện đảm bảo	Huy hiệu Unread bị xóa.
Luồng sự kiện chính	1. Người dùng mở hộp thoại. 2. Gọi API MarkAsRead. 3. Cập nhật DB.
Luồng rẽ nhánh	Không có.
Yêu cầu chức năng (FR)	◇ **FR_35.1**: Phải cập nhật LastReadTimestamp.

Mã Usecase	**UC-36**
Tên Usecase	**Lấy Lịch sử Tin nhắn**
Tác nhân (Actors)	Người dùng
Mô tả ngắn	Tải các tin nhắn cũ theo cơ chế Infinite Scroll.
Điều kiện tiên quyết	Đang trong Conversation.
Điều kiện đảm bảo	Trả về phân trang.
Luồng sự kiện chính	1. Cuộn lên trên. 2. Gửi request LastMessageId. 3. Lấy 20 tin nhắn tiếp theo.
Luồng rẽ nhánh	Không có.
Yêu cầu chức năng (FR)	◇ **FR_36.1**: Sử dụng Keyset Pagination (Cursor) để tránh trôi tin nhắn.

Mã Usecase	**UC-37**
Tên Usecase	**Viết Đánh giá (Review)**
Tác nhân (Actors)	Khách (Guest)
Mô tả ngắn	Khách đánh giá phòng sau khi ở.
Điều kiện tiên quyết	Booking ở trạng thái Completed.
Điều kiện đảm bảo	Lưu Review và cập nhật Rating tổng của phòng.
Luồng sự kiện chính	1. Nhập số sao và bình luận. 2. Lưu Review. 3. Phát Event cập nhật PropertyService.
Luồng rẽ nhánh	**Chưa hoàn tất chuyến đi**: Báo lỗi Booking must be Confirmed or Completed to review.
Yêu cầu chức năng (FR)	◇ **FR_37.1**: Mỗi Booking chỉ được phép viết 1 Review duy nhất.

Mã Usecase	**UC-38**
Tên Usecase	**Sửa/Xóa Đánh giá**
Tác nhân (Actors)	Khách (Guest)
Mô tả ngắn	Sửa hoặc xóa review đã viết.
Điều kiện tiên quyết	Review do chính khách viết.
Điều kiện đảm bảo	Rating tổng của phòng được tính toán lại.
Luồng sự kiện chính	1. Chọn Sửa/Xóa. 2. Cập nhật DB. 3. Phát Event để tính lại Rating trung bình.
Luồng rẽ nhánh	**Phòng bị xóa**: Báo lỗi Property not found.
Yêu cầu chức năng (FR)	◇ **FR_38.1**: Khi xóa Review, hệ thống phải cập nhật lại sao trung bình của phòng.

Mã Usecase	**UC-39**
Tên Usecase	**Đăng nhập Admin**
Tác nhân (Actors)	Admin
Mô tả ngắn	Đăng nhập vào hệ thống quản trị.
Điều kiện tiên quyết	Tài khoản có Role = Admin/Moderator.
Điều kiện đảm bảo	Cấp JWT Token quyền quản trị.
Luồng sự kiện chính	1. Nhập credentials. 2. Kiểm tra quyền. 3. Cấp Token.
Luồng rẽ nhánh	**Thiếu quyền**: Báo lỗi Access denied. Admin or Moderator role required.
Yêu cầu chức năng (FR)	◇ **FR_39.1**: Mật khẩu Admin phải được mã hóa theo chuẩn bcrypt.

Mã Usecase	**UC-40**
Tên Usecase	**Duyệt Bất động sản**
Tác nhân (Actors)	Admin
Mô tả ngắn	Duyệt tin đăng của Host trước khi hiển thị công khai.
Điều kiện tiên quyết	Phòng ở trạng thái Pending Approval.
Điều kiện đảm bảo	Phòng được duyệt thành Published hoặc bị Reject.
Luồng sự kiện chính	1. Admin xem tin. 2. Bấm Approve. 3. Cập nhật trạng thái và bắn event lên Kafka.
Luồng rẽ nhánh	**Thiếu thông tin**: Admin chọn Reject kèm lý do.
Yêu cầu chức năng (FR)	◇ **FR_40.1**: Sau khi Approve, dữ liệu phải xuất hiện trên Elasticsearch qua luồng CDC.

Mã Usecase	**UC-41**
Tên Usecase	**Khóa/Mở tài khoản**
Tác nhân (Actors)	Admin
Mô tả ngắn	Khóa user vi phạm chính sách.
Điều kiện tiên quyết	User tồn tại.
Điều kiện đảm bảo	Trạng thái user = Suspended, vô hiệu hóa mọi JWT đang có.
Luồng sự kiện chính	1. Admin chọn User. 2. Bấm Suspend. 3. Cập nhật DB và phát Event.
Luồng rẽ nhánh	**User không tồn tại**: Báo lỗi User not found.
Yêu cầu chức năng (FR)	◇ **FR_41.1**: Hệ thống tự động hủy các phiên đăng nhập (RevokeSessions) của user bị khóa.

Mã Usecase	**UC-42**
Tên Usecase	**Quản lý Danh mục Tiện ích**
Tác nhân (Actors)	Admin
Mô tả ngắn	Thêm, Sửa, Xóa các danh mục tiện ích cho toàn hệ thống.
Điều kiện tiên quyết	Quyền Admin.
Điều kiện đảm bảo	Danh mục tiện ích được cập nhật.
Luồng sự kiện chính	1. Admin nhập tên tiện ích mới. 2. Upload icon. 3. Lưu vào Amenity DB.
Luồng rẽ nhánh	**Trùng tên**: Báo lỗi Amenity already exists.
Yêu cầu chức năng (FR)	◇ **FR_42.1**: Không cho phép xóa tiện ích nếu đang có Property sử dụng nó.

Mã Usecase	**UC-43**
Tên Usecase	**Xử lý Khiếu nại (Disputes)**
Tác nhân (Actors)	Admin
Mô tả ngắn	Admin tham gia giải quyết tranh chấp giữa Host và Guest.
Điều kiện tiên quyết	Booking có tranh chấp.
Điều kiện đảm bảo	Tranh chấp được giải quyết (Refund hoặc phạt).
Luồng sự kiện chính	1. Xem bằng chứng của 2 bên. 2. Ra quyết định Refund. 3. Đóng Dispute.
Luồng rẽ nhánh	Không có.
Yêu cầu chức năng (FR)	◇ **FR_43.1**: Quyết định của Admin là cuối cùng và đóng băng Booking đó.

Mã Usecase	**UC-44**
Tên Usecase	**Xem Báo cáo Thống kê**
Tác nhân (Actors)	Admin
Mô tả ngắn	Xem biểu đồ doanh thu và lượng người dùng tăng trưởng.
Điều kiện tiên quyết	Đã đăng nhập Admin.
Điều kiện đảm bảo	Trả về dữ liệu mảng để vẽ Chart.
Luồng sự kiện chính	1. Chọn khoảng thời gian. 2. Truy vấn Analytics DB. 3. Trả về JSON.
Luồng rẽ nhánh	Không có.
Yêu cầu chức năng (FR)	◇ **FR_44.1**: Dữ liệu báo cáo phải được gom nhóm theo Ngày/Tháng/Năm (Group By).

Mã Usecase	**UC-45**
Tên Usecase	**Cập nhật Cấu hình Nền tảng**
Tác nhân (Actors)	Admin
Mô tả ngắn	Thay đổi phần trăm phí dịch vụ hoặc phí hoa hồng.
Điều kiện tiên quyết	Quyền Root Admin.
Điều kiện đảm bảo	Cấu hình mới được áp dụng ngay lập tức cho các giao dịch sau.
Luồng sự kiện chính	1. Admin nhập cấu hình mới. 2. Lưu vào bảng PlatformSettings. 3. Xóa Cache.
Luồng rẽ nhánh	Không có.
Yêu cầu chức năng (FR)	◇ **FR_45.1**: Các giao dịch tạo trước thời điểm cập nhật vẫn giữ nguyên mức phí cũ.

2.1.2 Yêu cầu phi chức năng (Non-Functional Requirements)

Bảng 2.1 tổng hợp các yêu cầu phi chức năng của hệ thống AirVnV, chia theo 6 thuộc tính chất lượng.

Bảng 2.1. Yêu cầu phi chức năng của hệ thống AirVnV

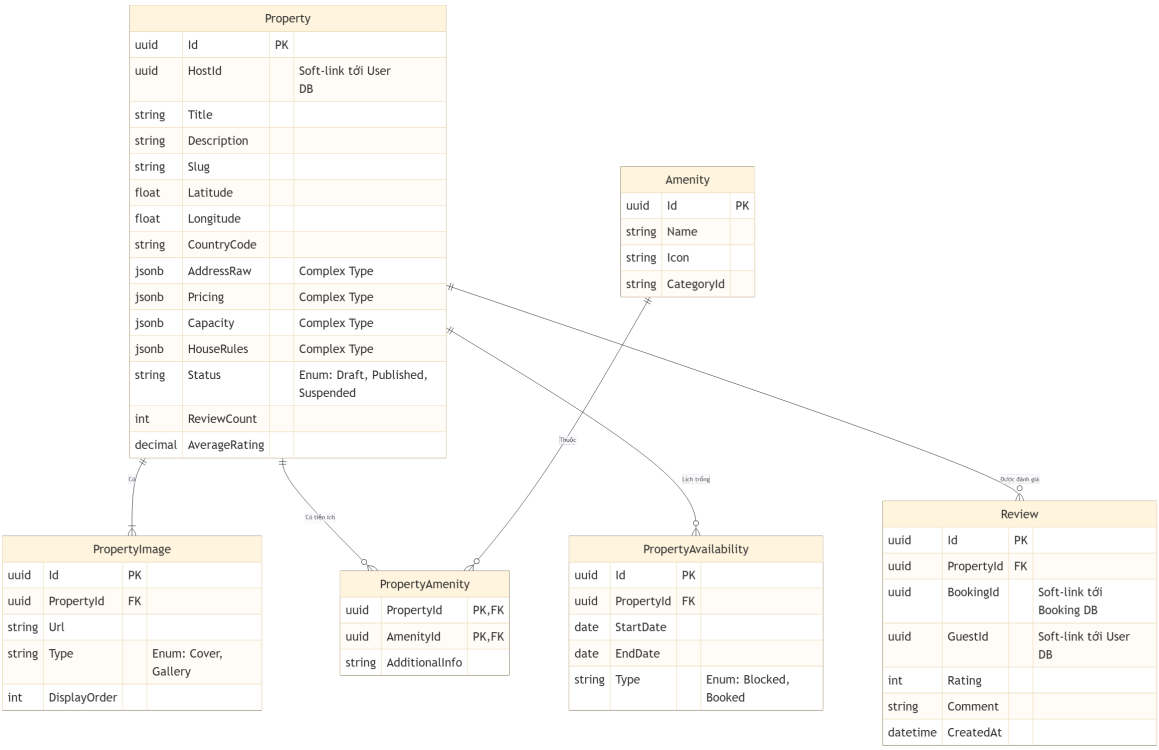
Thuộc tính	**Yêu cầu**	**Giải pháp kỹ thuật**
Hiệu năng	Response time $\leq$ 500ms (p95) cho các API nghiệp vụ chính. Kết quả tìm kiếm trả về $\leq$ 200ms.	SearchService đọc từ Elasticsearch (read-model), không truy vấn DB giao dịch.
Khả năng mở rộng	Từng service có thể scale-out độc lập mà không ảnh hưởng đến service khác.	Kiến trúc Microservices + .NET Aspire Service Discovery. Mỗi service là một container riêng biệt.
Tính nhất quán dữ liệu	Dữ liệu giữa các service đạt trạng thái nhất quán trong thời gian ngắn (Eventual Consistency). Không bao giờ mất event nghiệp vụ.	Transactional Outbox Pattern + CDC (Debezium) + Kafka. Event được lưu cùng transaction, đảm bảo at-least-once delivery.
Bảo mật	Mọi API (trừ đăng ký / đăng nhập) phải được xác thực. Mật khẩu không được lưu dưới dạng plaintext.	JWT Bearer token, xác thực tập trung tại YARP API Gateway. Mật khẩu mã hóa với BCrypt.
Tính sẵn sàng	Lỗi ở một service không làm sập toàn bộ hệ thống.	Fault isolation theo Microservices. Các service giao tiếp bất đồng bộ qua RabbitMQ / Kafka — không phụ thuộc lẫn nhau.
Bảo trì & vận hành	Schema database chỉ được thay đổi qua migration, không sửa trực tiếp. Log phải có cấu trúc, truy vết được.	EF Core Migrations cho tất cả service. Structured Logging (ILogger) tích hợp với .NET Aspire Dashboard.

2.2 Thiết kế lược đồ dữ liệu tích hợp

2.2.1 Thiết kế Dữ liệu Property Service

Property Service là domain cốt lõi (Core Domain) của hệ thống AirVnV, quản lý toàn bộ vòng đời của một căn hộ cho thuê. Cơ sở dữ liệu được thiết kế tập trung quanh Aggregate Root là `Property`, sử dụng PostgreSQL với khả năng hỗ trợ kiểu dữ liệu JSONB mạnh mẽ.

- **Property (Aggregate Root):** Lưu trữ toàn bộ thông tin cơ bản của căn hộ. Điểm đặc biệt trong thiết kế là việc sử dụng kiểu **JSONB** cho các Value Objects phức tạp như `AddressRaw`, `Pricing`, `Capacity`, và `HouseRules`. Việc gom nhóm (Embed) các object này vào cùng một bảng thay vì tạo bảng riêng lẻ giúp loại bỏ hoàn toàn các câu lệnh `JOIN` tốn kém, tối ưu hóa tốc độ truy vấn đọc (Read-heavy) đặc trưng của domain này.
- **PropertyImage & PropertyAvailability:** Các Entity con (Child Entities) có vòng đời phụ thuộc hoàn toàn vào Property. `PropertyAvailability` lưu trữ các khoảng thời gian bị khóa (Blocked) hoặc đã được đặt (Booked) để phục vụ việc kiểm tra lịch trống.
- **Amenity & PropertyAmenity:** Thiết kế bảng trung gian để map quan hệ N-N giữa Tiện ích và Căn hộ. Bảng `Amenity` đóng vai trò là danh mục (Catalog) chuẩn hóa toàn hệ thống.
- **Review:** Quản lý đánh giá của người dùng. Mặc dù Review liên quan đến Booking và Guest, nhưng theo tư duy Domain-Driven Design (DDD), Review thuộc về Property vì nó tác động trực tiếp đến `ReviewCount` và `AverageRating` của căn hộ. Thông tin người dùng (`GuestId`) và đơn đặt (`BookingId`) chỉ được lưu dưới dạng Soft-link.



Hình 2.2. Sơ đồ Thực thể Liên kết (ERD) của Property Service

2.2.2 Thiết kế Dữ liệu Booking Service

Booking Service là dịch vụ đảm nhận luồng xử lý giao dịch quan trọng nhất của hệ thống: Đặt phòng và Thanh toán. Vì đặc thù của giao dịch phân tán, dữ liệu của Booking Service được thiết kế cực kỳ tối giản, chỉ tập trung vào một Aggregate Root duy nhất.

- **Booking (Aggregate Root):** Lưu trữ toàn bộ dữ liệu của một phiên đặt phòng. Các ID liên quan như `PropertyId`, `HostId`, và `GuestId` được trỏ sang các dịch vụ khác thông qua Soft-link. Bảng này quản lý chặt chẽ trạng thái đặt phòng (`Status`) qua các giai đoạn: *Pending, AwaitingApproval, Confirmed, Cancelled*. Các trường liên quan đến tài chính (`BasePricePerNight`, `CleaningFee`, `TaxAmount`, `TotalPrice`) được tính toán và lưu cứng ngay tại thời điểm tạo để tránh sai lệch dữ liệu nếu giá căn hộ thay đổi sau này.
- **ProcessedEvent:** Bảng đặc thù sinh ra từ Kiến trúc Hướng sự kiện (Event-Driven). Do hệ thống sử dụng Kafka làm Message Broker, việc nhận lặp tin nhắn (Duplicate Messages) là không thể tránh khỏi (cơ chế At-least-once delivery). Bảng `ProcessedEvent` hoạt động như một cơ chế **Idempotency Guard**, lưu lại UUID của các Event đã xử lý để đảm bảo một giao dịch (ví dụ: Thanh toán thành công) không bao giờ bị thực thi hai lần.

Booking			
uuid	Id	PK	
uuid	PropertyId		Soft-link tới Property DB
uuid	HostId		Soft-link tới User DB
uuid	GuestId		Soft-link tới User DB
string	CountryCode		
string	BookingMode		InstantBook / RequestToBook
date	CheckIn		
date	CheckOut		
int	GuestCount		
int	NightCount		
decimal	BasePricePerNight		
decimal	CleaningFee		
decimal	ServiceFee		
decimal	TaxAmount		
decimal	TotalPrice		
string	CurrencyCode		
string	Status		Enum: Pending, AwaitingApproval, Confirmed, Cancelled
uuid	CancelledBy		Nullable
datetime	CreatedAt		

ProcessedEvent			
uuid	EventId	PK	Idempotency Key
string	EventType		
datetime	ProcessedAt		

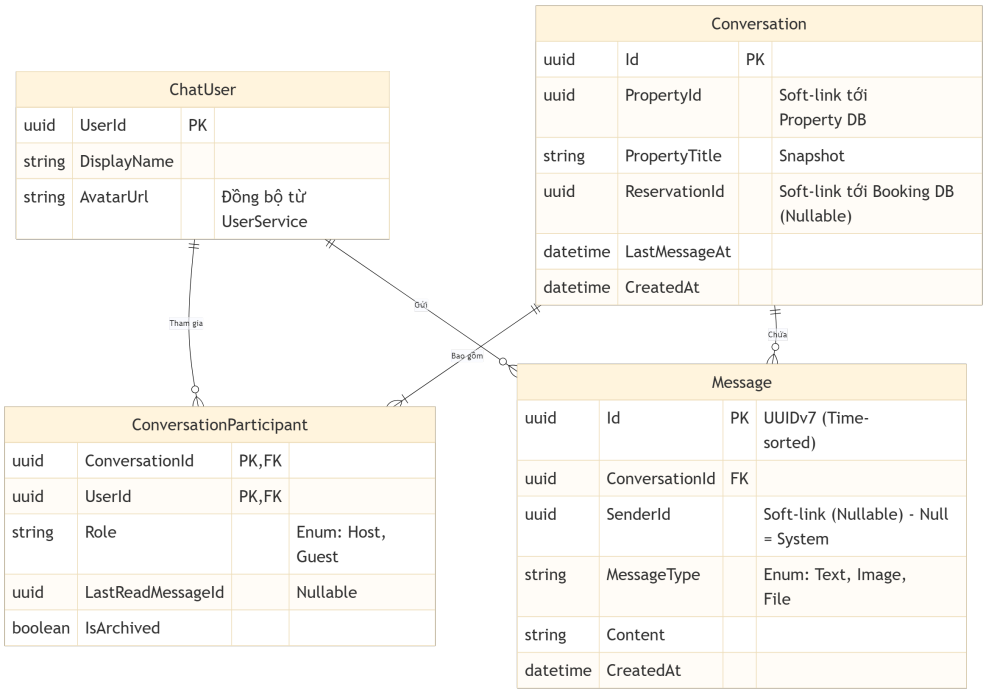
Hình 2.3. Sơ đồ Thực thể Liên kết (ERD) của Booking Service

2.2.3 Thiết kế Dữ liệu Chat Service

Cơ sở dữ liệu của Chat Service được thiết kế tập trung vào việc quản lý hội thoại theo ngữ cảnh (Property/Booking) và tối ưu hóa việc phân phối tin nhắn theo thời gian thực. Theo nguyên lý Database-per-Service, lược đồ bao gồm 4 thực thể chính hoàn toàn độc lập:

- **ChatUser:** Quản lý thông tin hồ sơ cơ bản (DisplayName, AvatarUrl). Dữ liệu này được nhân bản (Data Duplication) và đồng bộ từ UserService thông qua Message Broker (RabbitMQ) mỗi khi người dùng cập nhật profile. Việc này giúp Chat Service không cần gọi API đồng bộ sang UserService mỗi khi hiển thị tin nhắn, loại bỏ hiện tượng thắt cổ chai (Bottleneck).
- **Conversation (Aggregate Root):** Đại diện cho một cuộc trò chuyện. Thay vì sử dụng Khóa ngoại (Foreign Key), bảng này chỉ lưu trữ `PropertyId` và `ReservationId` dưới dạng tham chiếu mềm (Soft-link) sang CSDL của PropertyService và BookingService. Trường `PropertyTitle` được lưu dưới dạng Snapshot để tăng tốc độ load danh sách Inbox.
- **ConversationParticipant:** Bảng trung gian giải quyết mối quan hệ n-n giữa Người dùng và Cuộc trò chuyện. Nó lưu trữ vai trò (`Role`), trạng thái lưu trữ (`IsArchived`), và đặc biệt là `LastReadMessageId` để hệ thống tự động tính toán số lượng tin nhắn chưa đọc (Unread Count) mà không cần query duyệt toàn bộ bảng tin nhắn.

- **Message:** Quản lý nội dung tin nhắn. Cột `SenderId` có thể mang giá trị `NULL` để đại diện cho Tin nhắn hệ thống (System Message). Khóa chính `Id` sử dụng **UUIDv7** thay vì UUIDv4, giúp các bản ghi tự động sắp xếp theo thứ tự thời gian (Time-sorted) ngay tại tầng Database Index, tối ưu hóa đáng kể truy vấn phân trang (Cursor Pagination).



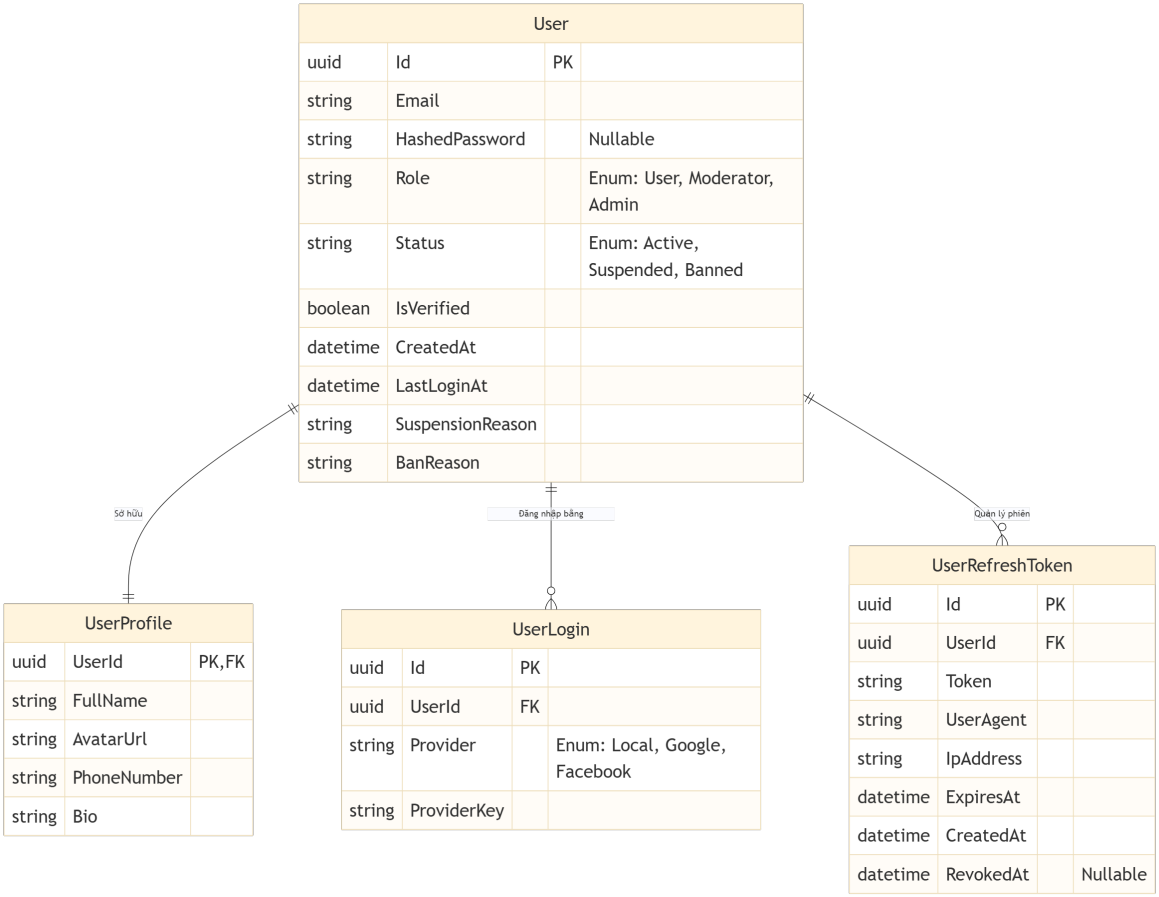
Hình 2.4. Sơ đồ Thực thể Liên kết (ERD) của Chat Service

2.2.4 Thiết kế Dữ liệu User Service

User Service đóng vai trò là ”Identity Provider” (Nhà cung cấp định danh) cho toàn bộ hệ thống. Dữ liệu được thiết kế tập trung vào tính năng xác thực (Authentication), phân quyền (Authorization) và quản lý phiên đăng nhập (Session Management).

- **User (Aggregate Root):** Lưu trữ thông tin tài khoản cốt lõi như `Email`, `HashedPassword`, `Role` và `Status`. Trường `HashedPassword` có thể `NULL` để hỗ trợ linh hoạt các phương thức đăng nhập bằng mạng xã hội (Social Login) mà không vi phạm tính toàn vẹn dữ liệu.
- **UserProfile:** Bảng lưu trữ thông tin hiển thị của người dùng. Trong Entity Framework Core, thực thể này được cấu hình dưới dạng **Owned Entity Type** (Bảng phụ thuộc hoàn toàn vào Aggregate Root) theo quan hệ 1-1 khắt khe.

- **UserLogin:** Giải quyết bài toán đa phương thức đăng nhập (Multi-authentication). Bảng này cho phép một `User` liên kết với nhiều nền tảng định danh khác nhau (Google, Facebook, Local) thông qua `Provider` và `ProviderKey`.
- **UserRefreshToken:** Đảm nhận trách nhiệm cấp phát và vô hiệu hóa (Revoke) phiên đăng nhập. Nó lưu trữ `Token`, thông tin thiết bị (`UserAgent`, `IpAddress`) và `ExpiresAt` để hỗ trợ cơ chế Refresh Token xoay vòng (Token Rotation) nhằm tăng cường bảo mật.

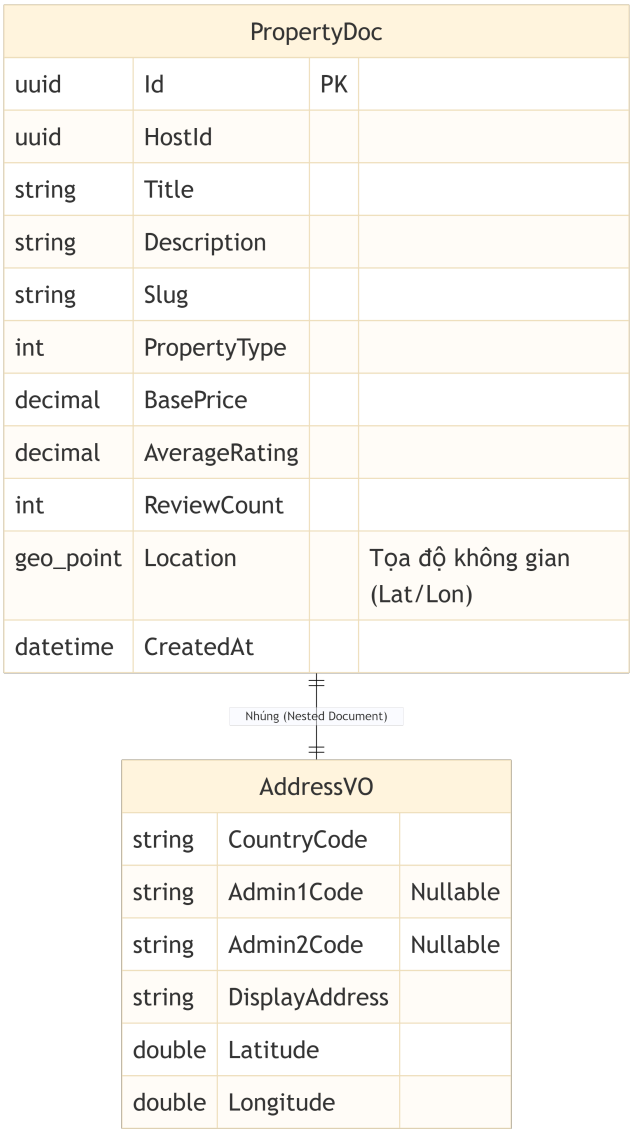


Hình 2.5. Sơ đồ Thực thể Liên kết (ERD) của User Service

2.2.5 Thiết kế Dữ liệu Search Service (Elasticsearch)

Đặc thù của Search Service là không sử dụng Cơ sở dữ liệu quan hệ (PostgreSQL) mà sử dụng **Elasticsearch** đóng vai trò là một Read-Model (Mô hình phục vụ việc Đọc). Lược đồ dữ liệu ở đây không phải là Table (Bảng) mà là các Document (Tài liệu JSON) được thiết kế giải chuẩn hóa hoàn toàn nhằm tối ưu hóa cho tốc độ truy vấn phức tạp và tìm kiếm theo tọa độ không gian (Geo-spatial Search).

- **PropertyDoc (Document Root):** Lưu trữ thông tin tổng hợp của một căn hộ, được đồng bộ tự động theo thời gian thực từ Property Service qua Debezium và Kafka. Dữ liệu này gom sẵn các trường `BasePrice`, `AverageRating`, và `ReviewCount` để phục vụ thao tác Lọc (Filtering) và Sắp xếp (Sorting) trực tiếp tại engine của Elasticsearch.
- **Location (geo_point):** Kiểu dữ liệu đặc thù của Elasticsearch, định dạng `"lat, lon"`. Trường này cho phép Search Service thực thi siêu tốc các truy vấn theo bán kính địa lý (Geo-Distance Queries) như: "Tìm căn hộ trong bán kính 5km quanh tọa độ X".
- **AddressVO (Nested Document):** Một Object được nhúng (Embed) trực tiếp bên trong tài liệu gốc. Nó giúp phân mảnh các yếu tố của địa chỉ (`CountryCode`, `DisplayAddress`) thành các token để hỗ trợ tìm kiếm toàn văn bản (Full-text Search).

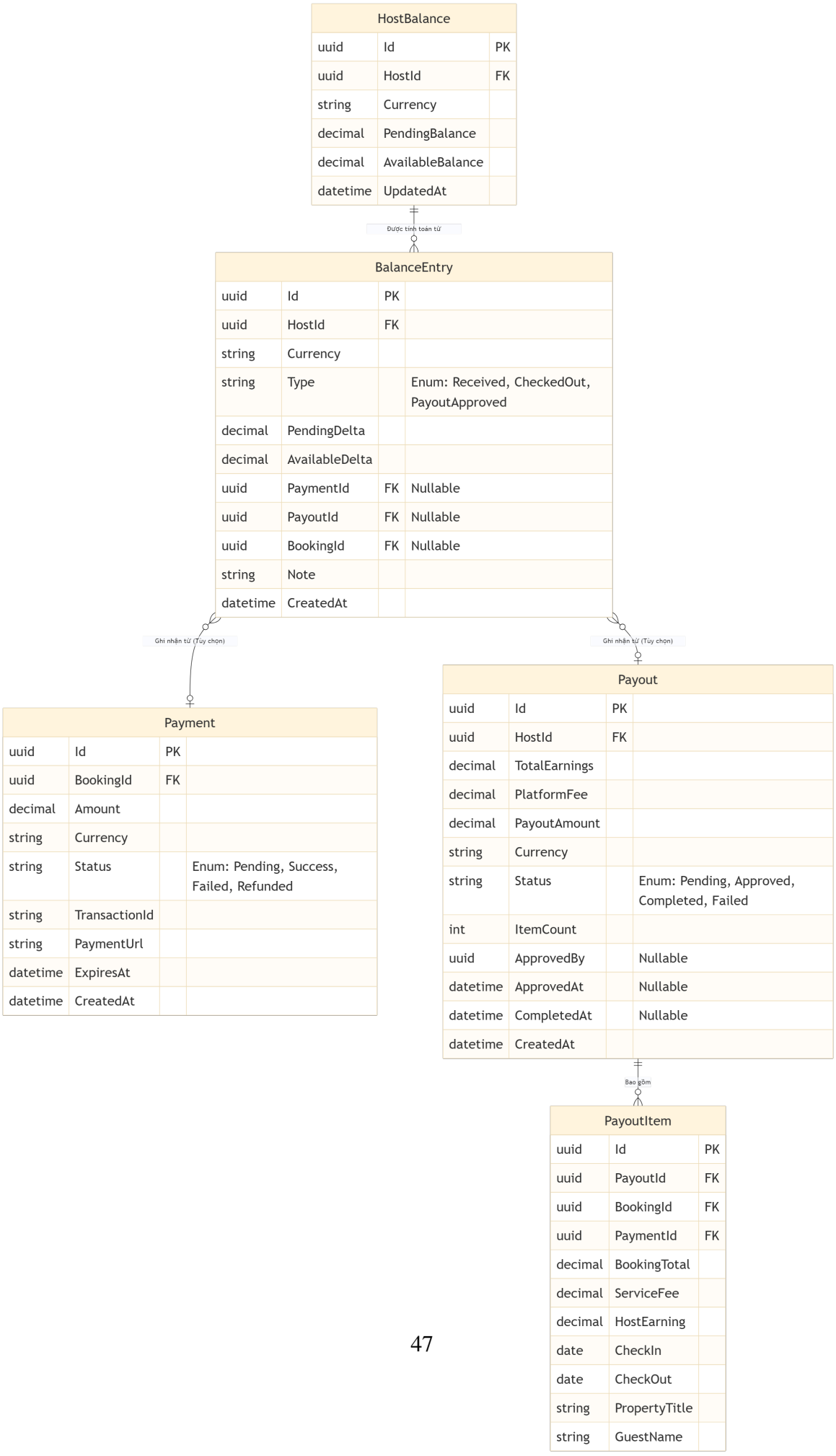


Hình 2.6. Sơ đồ Lược đồ Tài liệu (Document Schema) của Search Service

2.2.6 Thiết kế Dữ liệu Payment Service

Payment Service quản lý vòng đời thanh toán của người dùng và đối soát doanh thu cho chủ nhà (Host). Lược đồ CSDL được thiết kế theo dạng Event Sourcing thu nhỏ và Ledger (Sổ cái) để đảm bảo không bao giờ thất thoát dòng tiền.

- **Payment:** Quản lý các giao dịch thanh toán từ Guest (Inbound). Nó theo dõi `Amount`, `Currency`, và `Status` (Pending, Success, Failed, Refunded). Thực thể này không liên kết cứng với Booking mà chỉ lưu trữ `BookingId` dưới dạng tham chiếu mềm (Soft-link).
- **HostBalance & BalanceEntry:** Cặp thực thể đóng vai trò như Sổ cái (Ledger). `HostBalance` lưu trữ số dư `PendingBalance` (tiền tạm giữ) và `AvailableBalance` (tiền có thể rút). Bất kỳ thay đổi nào về số dư đều phải được ghi nhận thành một bản ghi bất biến (Immutable Record) trong `BalanceEntry` nhằm phục vụ truy vết (Audit) và tính toán lại đối soát (Recompute) khi cần thiết.
- **Payout & PayoutItem:** Quản lý quy trình trả tiền cho Host (Outbound). `Payout` là lệnh tổng hợp số dư để giải ngân. `PayoutItem` là các bản ghi giải chuẩn hóa (Denormalized) lưu lại snapshot của giao dịch (gồm `BookingTotal`, `ServiceFee`, và `HostEarning`) nhằm giữ nguyên lịch sử kế toán ngay cả khi thông tin căn hộ bị xóa hoặc thay đổi giá trong tương lai.


HostBalance
uuid Id PK
uuid HostId FK
string Currency
decimal PendingBalance
decimal AvailableBalance
datetime UpdatedAt
Được tính toán từ
BalanceEntry
uuid Id PK
uuid HostId FK
string Currency
string Type Enum: Received, CheckedOut, PayoutApproved
decimal PendingDelta
decimal AvailableDelta
uuid PaymentId FK Nullable
uuid PayoutId FK Nullable
uuid BookingId FK Nullable
string Note
datetime CreatedAt
Ghi nhận từ (Tùy chọn)
Ghi nhận từ (Tùy chọn)
Payment
uuid Id PK
uuid BookingId FK
decimal Amount
string Currency
string Status Enum: Pending, Success, Failed, Refunded
string TransactionId
string PaymentUrl
datetime ExpiresAt
datetime CreatedAt
Payout
uuid Id PK
uuid HostId FK
decimal TotalEarnings
decimal PlatformFee
decimal PayoutAmount
string Currency
string Status Enum: Pending, Approved, Completed, Failed
int ItemCount
uuid ApprovedBy Nullable
datetime ApprovedAt Nullable
datetime CompletedAt Nullable
datetime CreatedAt
Bao gồm
PayoutItem
uuid Id PK
uuid PayoutId FK
uuid BookingId FK
uuid PaymentId FK
decimal BookingTotal
decimal ServiceFee
decimal HostEarning
date CheckIn
date CheckOut
string PropertyTitle
string GuestName

2.2.7 Cơ sở Dữ liệu Phân tán & Cấu trúc Outbox / Inbox Pattern

Do áp dụng kiến trúc Microservices, hệ thống đối mặt với bài toán đảm bảo tính nhất quán dữ liệu (Data Consistency) giữa việc lưu trạng thái xuống Database và việc gửi/nhận sự kiện (Domain Events) qua Message Broker. Để giải quyết triệt để rủi ro này, dự án sử dụng bộ 3 bảng tiêu chuẩn của mô hình Outbox (điển hình như MassTransit) trên toàn bộ các Service:

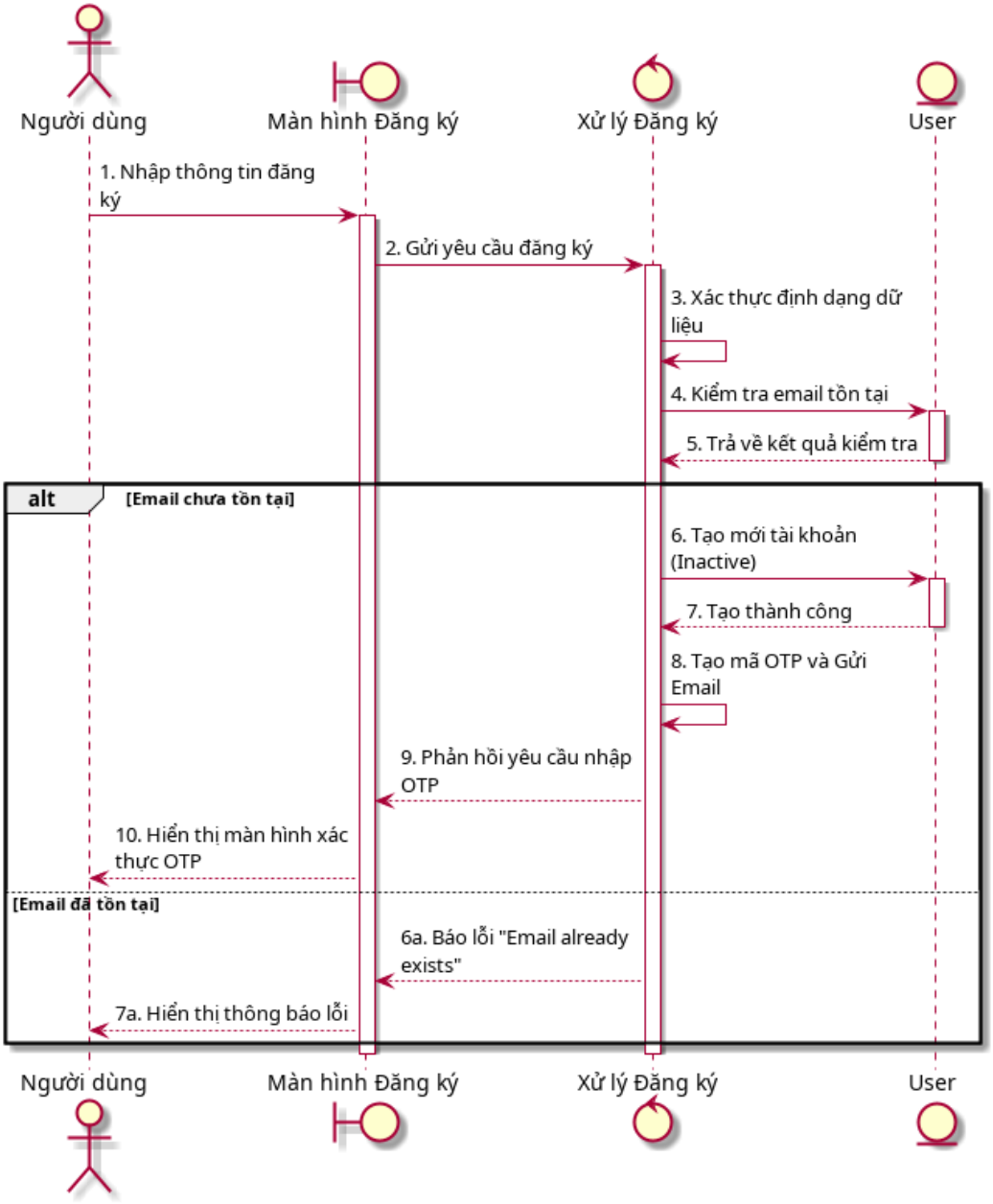
- **OutboxMessage:** Thay vì gửi Event trực tiếp (có rủi ro gửi thành công nhưng lỗi lưu DB), hệ thống lưu Event vào bảng này trong cùng một Transaction (Giao dịch ACID) với nghiệp vụ chính. Một tiến trình nền (Background Worker) sau đó sẽ quét bảng này và đẩy an toàn lên Message Broker.
- **OutboxState:** Bảng hỗ trợ lưu trữ trạng thái (State) của các tiến trình gửi tin nhắn ở chiều đi. Bảng này giúp quản lý việc gửi theo lô (Batching) và khóa (Row Locking) để tiến trình quét không gửi trùng lặp hoặc sót tin nhắn khi chạy nhiều Instance (Scale-out).
- **InboxState:** Bảng đặc thù giải quyết bài toán **Idempotency** (Tính lũy đẳng) ở chiều nhận (Consumer). Do Message Broker luôn có rủi ro gửi lặp tin nhắn (At-least-once delivery), bảng này lưu lại `MessageId` của các sự kiện đã tiêu thụ thành công. Nếu một sự kiện bị gửi lại, hệ thống sẽ kiểm tra `InboxState` và tự động bỏ qua, đảm bảo mỗi giao dịch (ví dụ: Thanh toán) chỉ được thực thi đúng một lần duy nhất.

Việc tích hợp đồng bộ 3 bảng `InboxState`, `OutboxMessage`, `OutboxState` vào CSDL của từng Service giúp kiến trúc phân tán đạt được tính **Eventually Consistent** (Nhất quán cuối) một cách tự động và bền bỉ mà không cần dùng đến các cơ chế đắt đỏ như Two-Phase Commit (2PC).

2.3 Sơ đồ Tuần tự Nghiệp vụ (Sequence Diagrams)

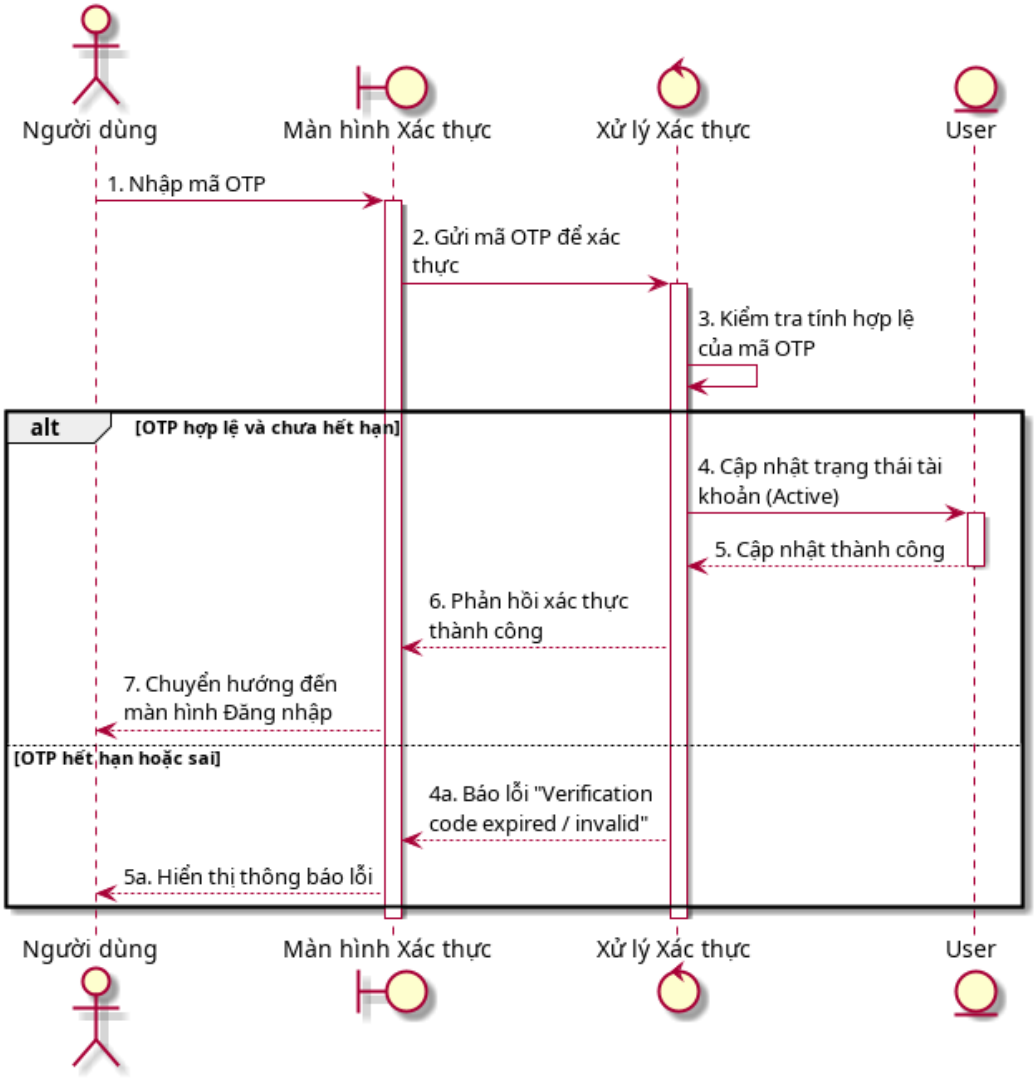
Phần này trình bày luồng tương tác dữ liệu (Data Flow) của các Use Case trong hệ thống theo mô hình Boundary - Control - Entity (BCE). Các sơ đồ tập trung vào tầng Phân tích (Analysis Level), không bao gồm các chi tiết triển khai vật lý.

1. UC-01: Đăng ký tài khoản



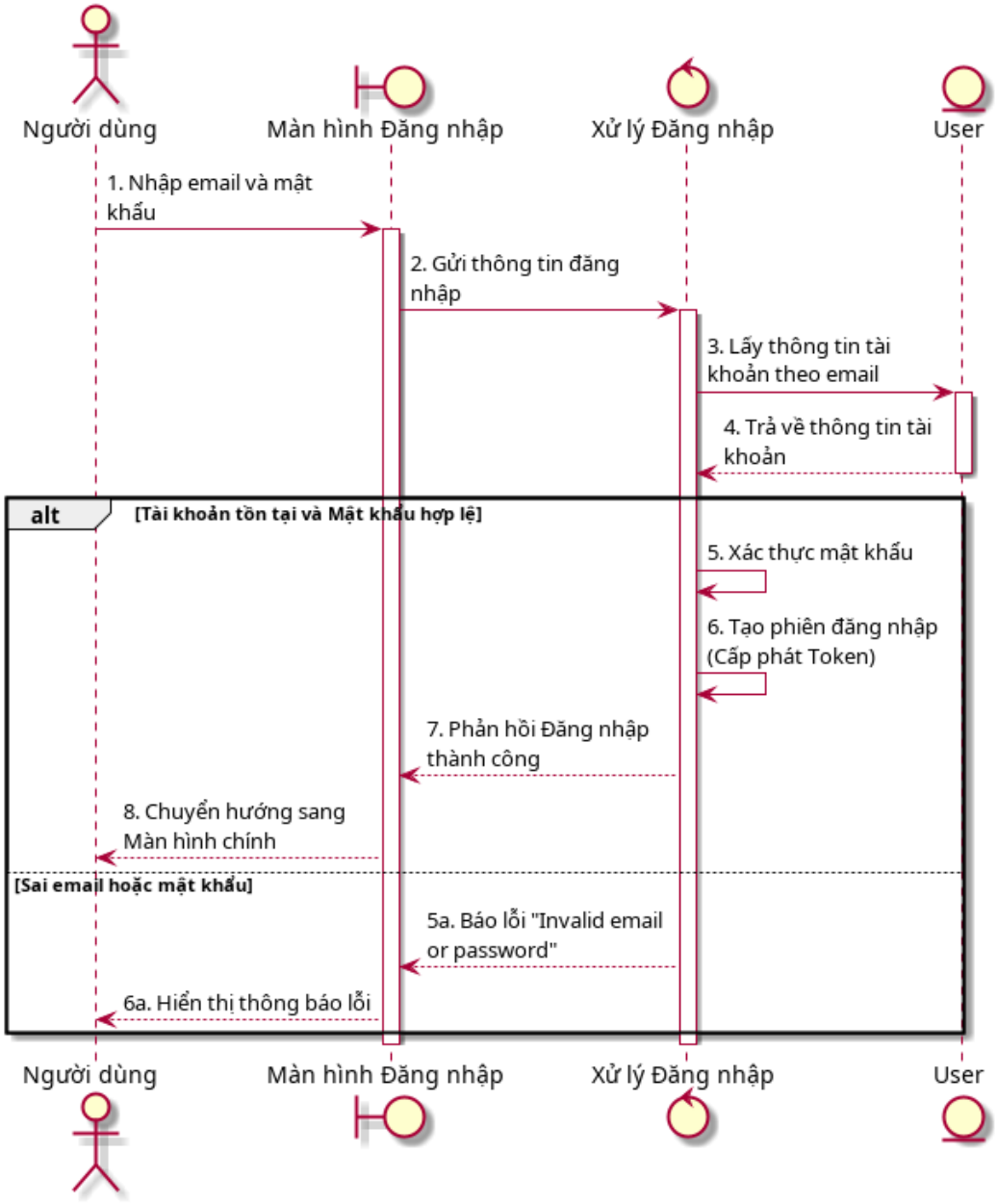
Hình 2.8. Sơ đồ Tuần tự: UC-01 Đăng ký tài khoản

2. UC-02: Xác thực Email (Verify)



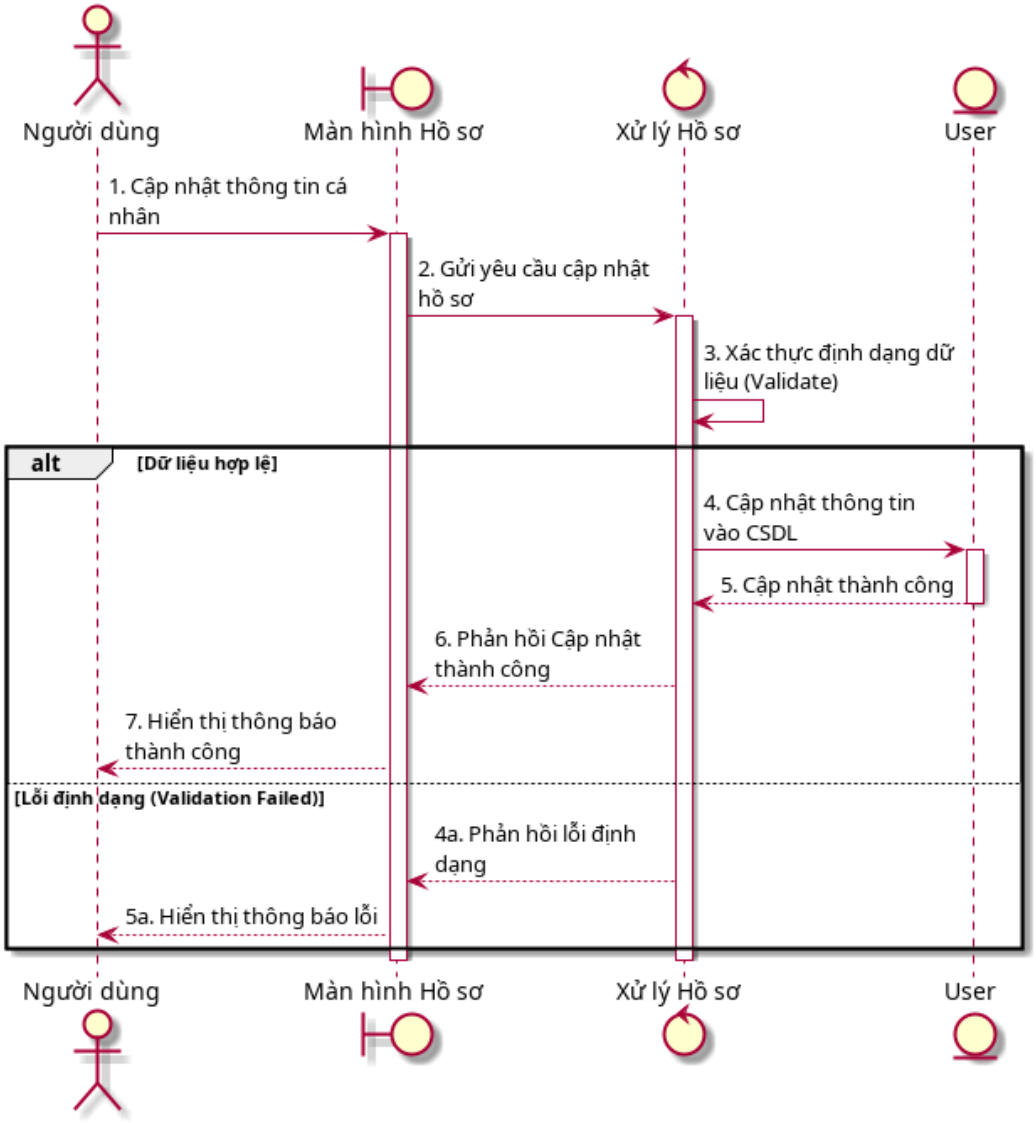
Hình 2.9. Sơ đồ Tuần tự: UC-02 Xác thực Email

3. UC-03: Đăng nhập hệ thống



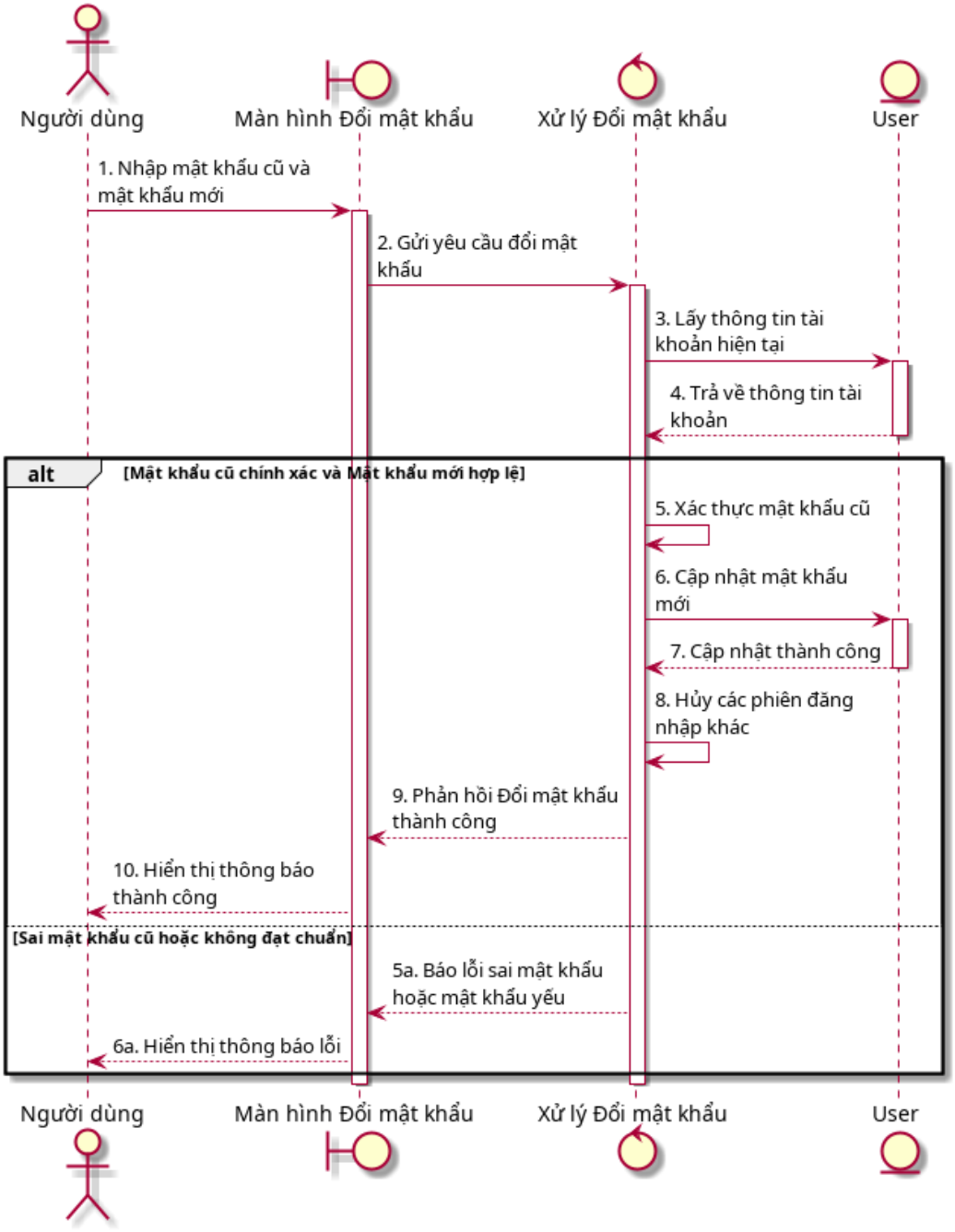
Hình 2.10. Sơ đồ Tuần tự: UC-03 Đăng nhập hệ thống

4. UC-04: Cập nhật hồ sơ cá nhân



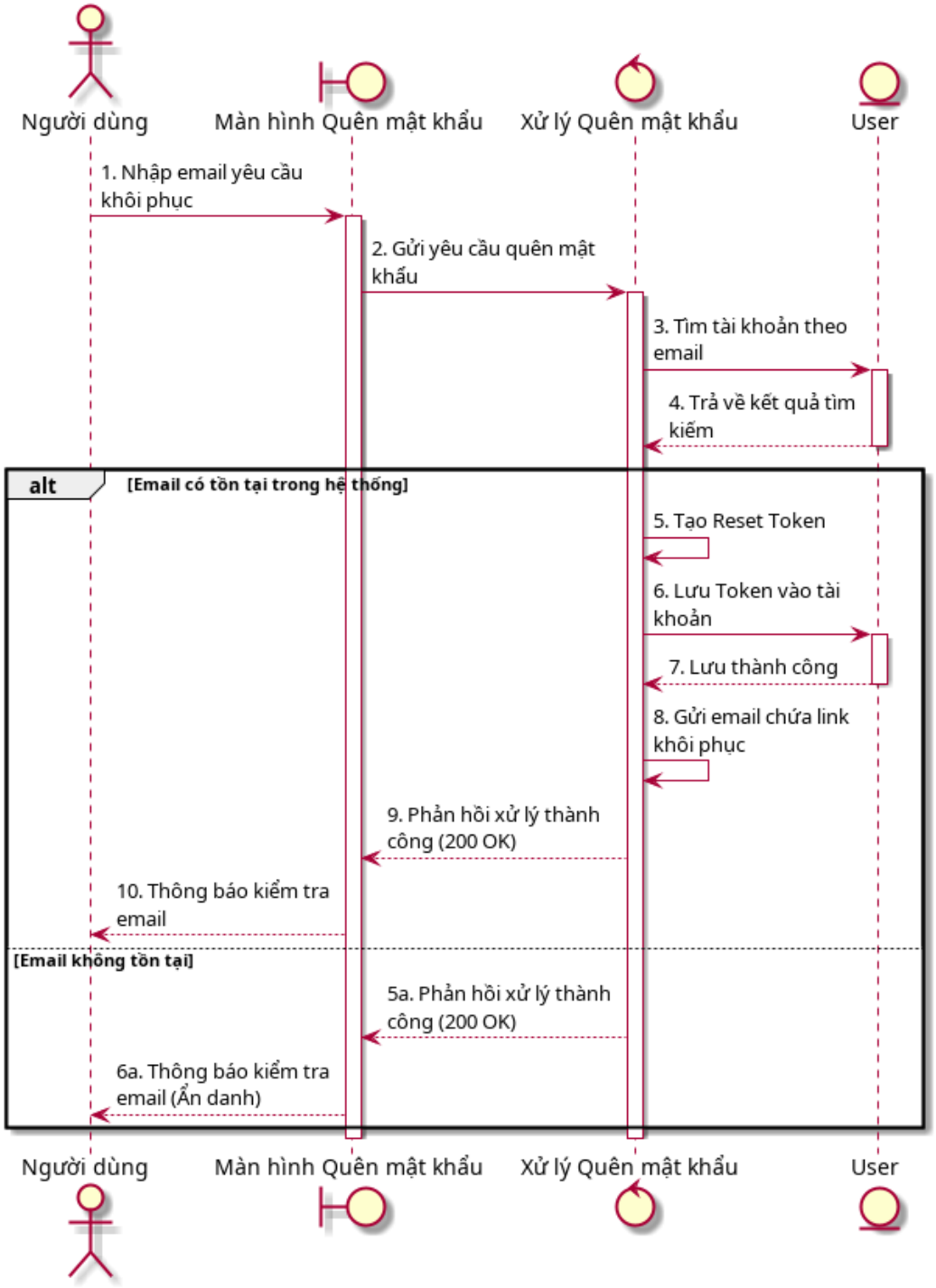
Hình 2.11. Sơ đồ Tuần tự: UC-04 Cập nhật hồ sơ cá nhân

5. UC-05: Đổi mật khẩu



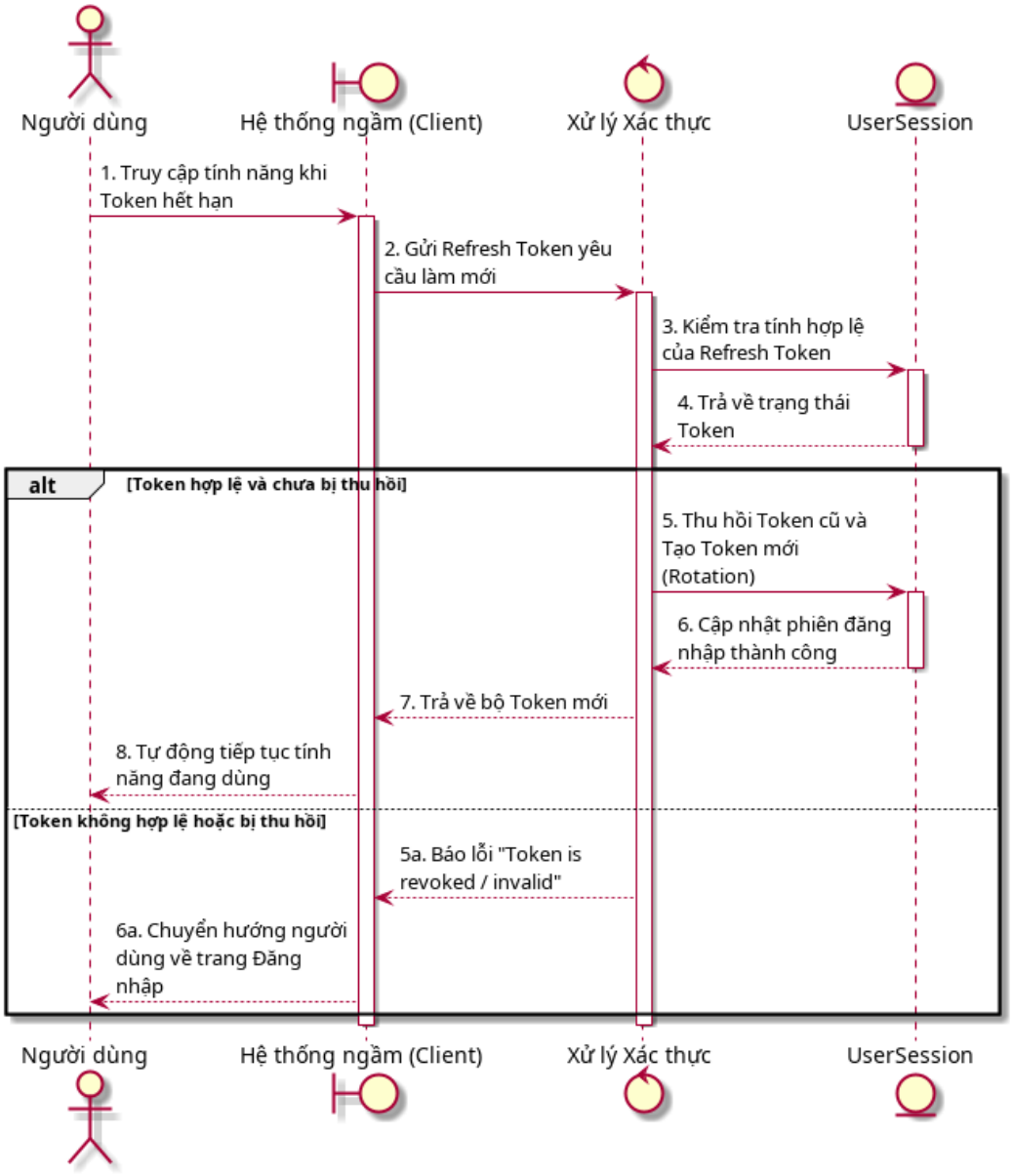
Hình 2.12. Sơ đồ Tuần tự: UC-05 Đổi mật khẩu

6. UC-06: Quên mật khẩu



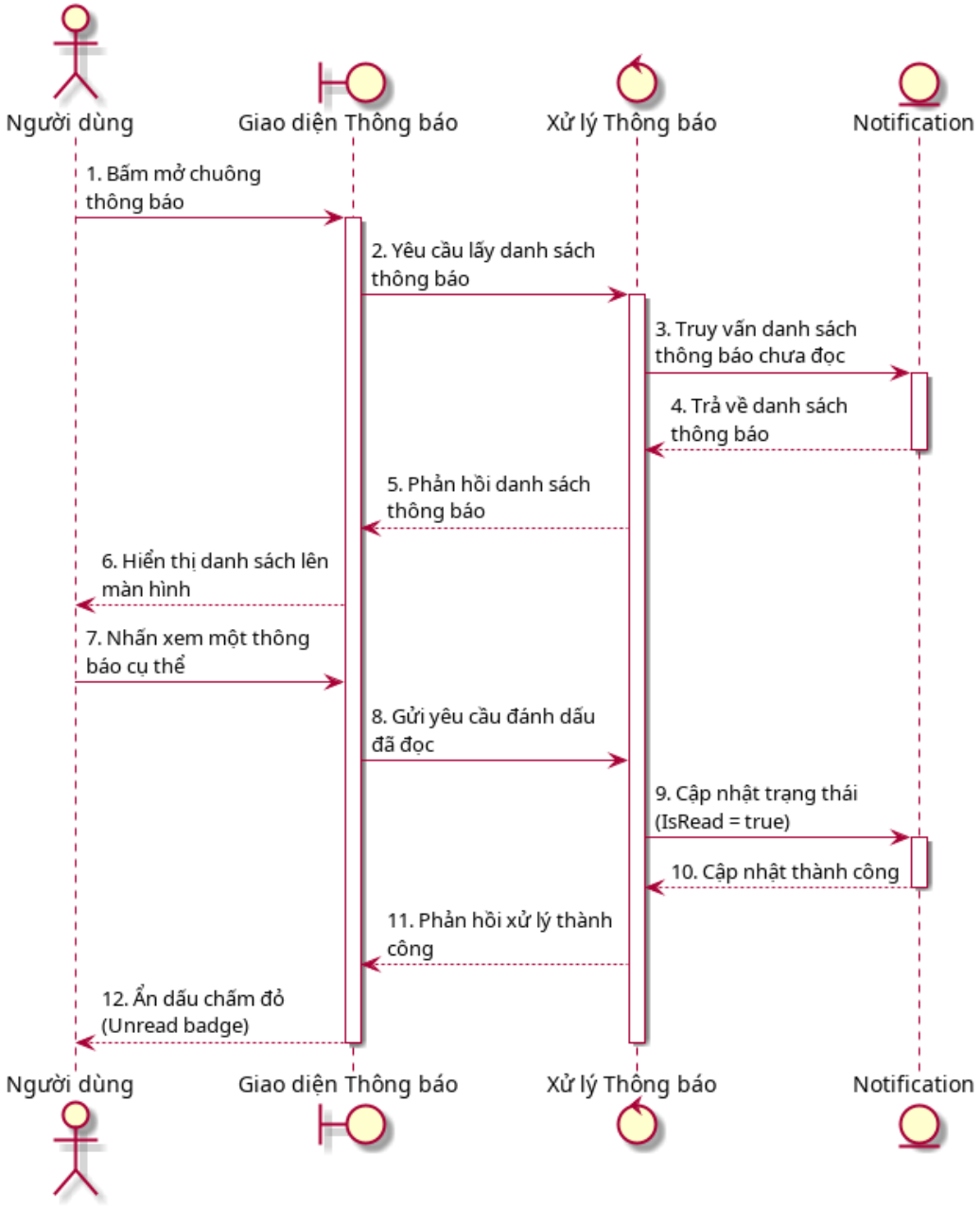
Hình 2.13. Sơ đồ Tuần tự: UC-06 Quên mật khẩu

7. UC-07: Làm mới Token (Refresh Token)



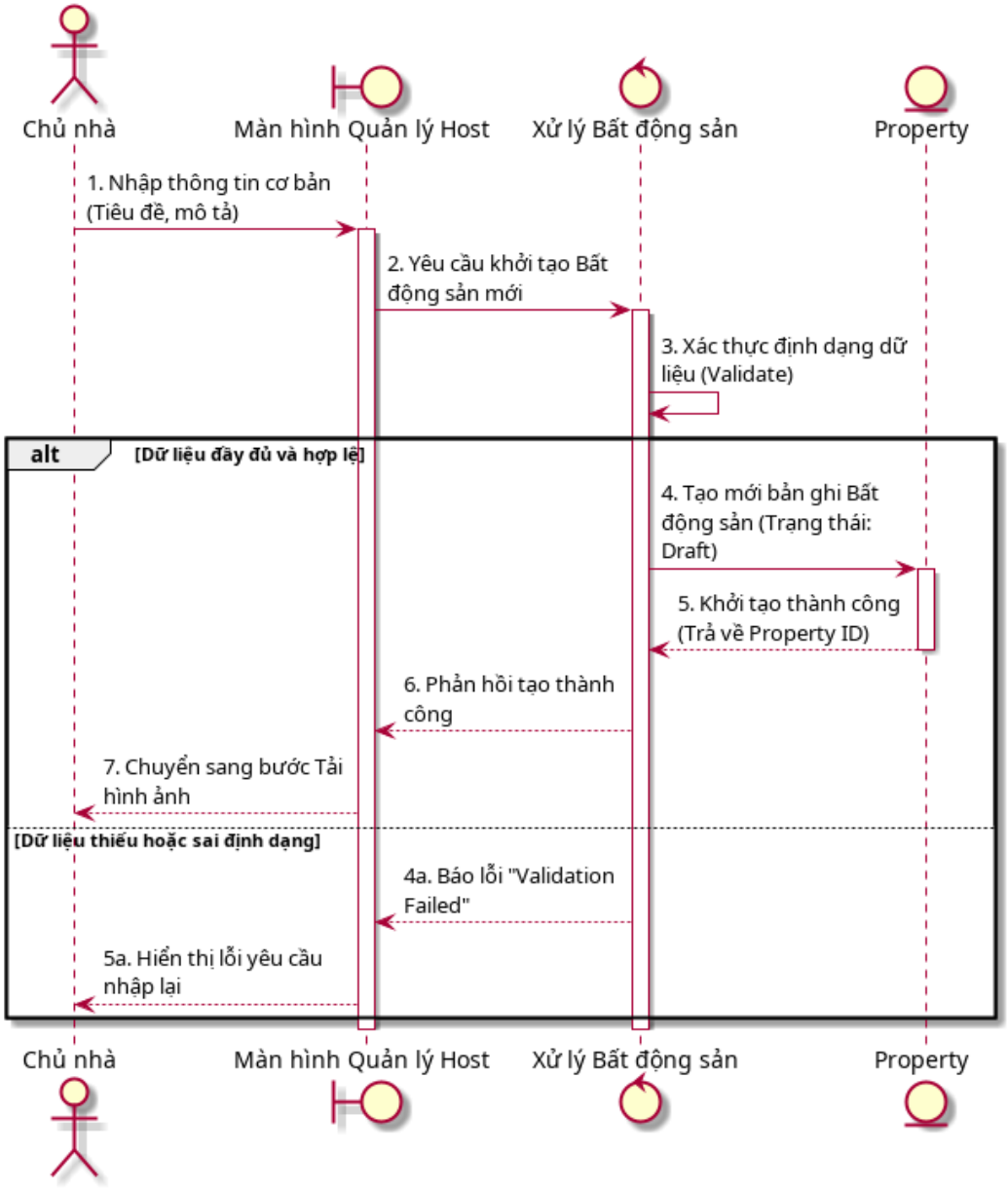
Hình 2.14. Sơ đồ Tuần tự: UC-07 Làm mới Token

8. UC-08: Nhận Thông báo (Notifications)



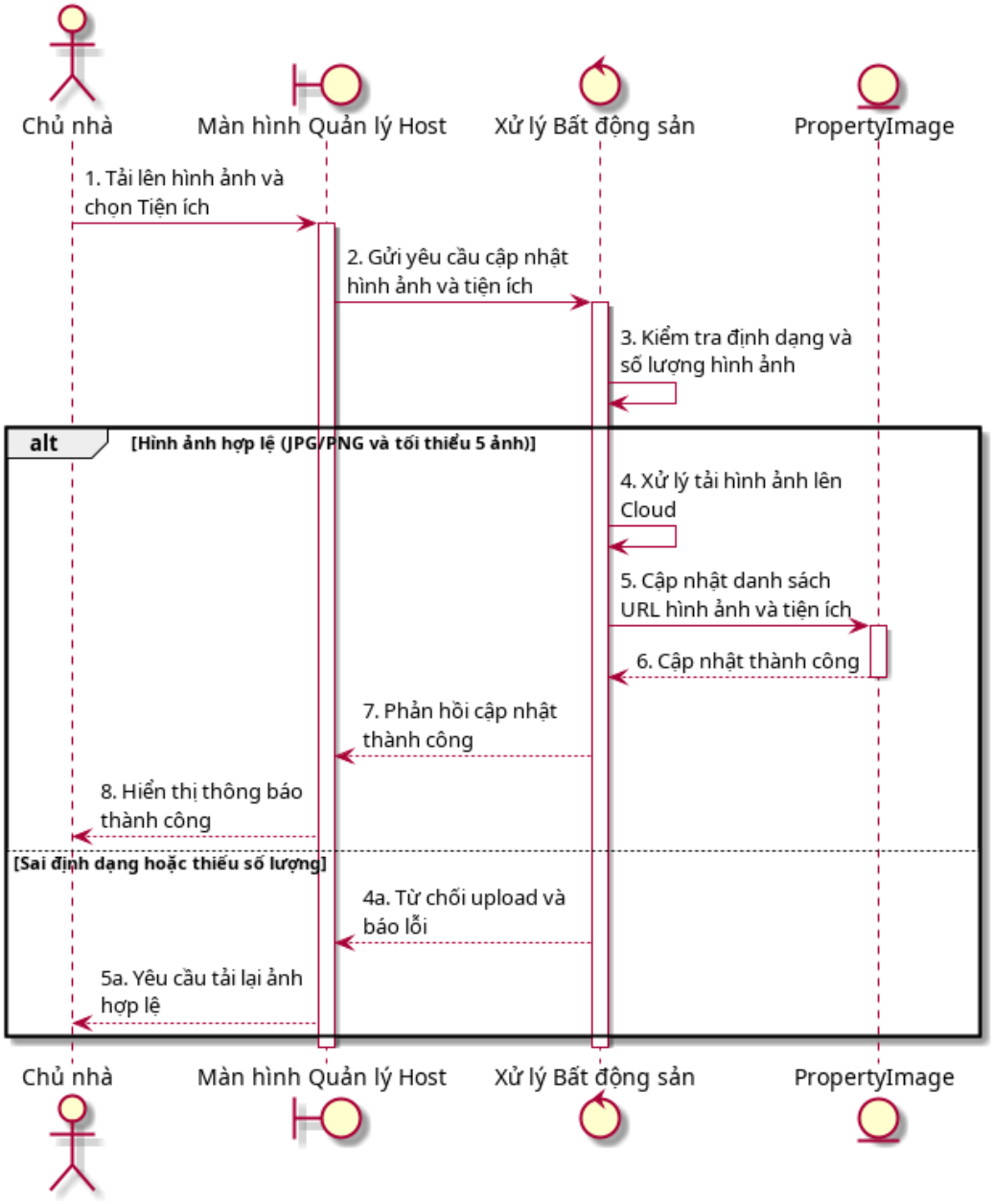
Hình 2.15. Sơ đồ Tuần tự: UC-08 Nhận Thông báo

9. UC-09: Đăng tin Bất động sản mới



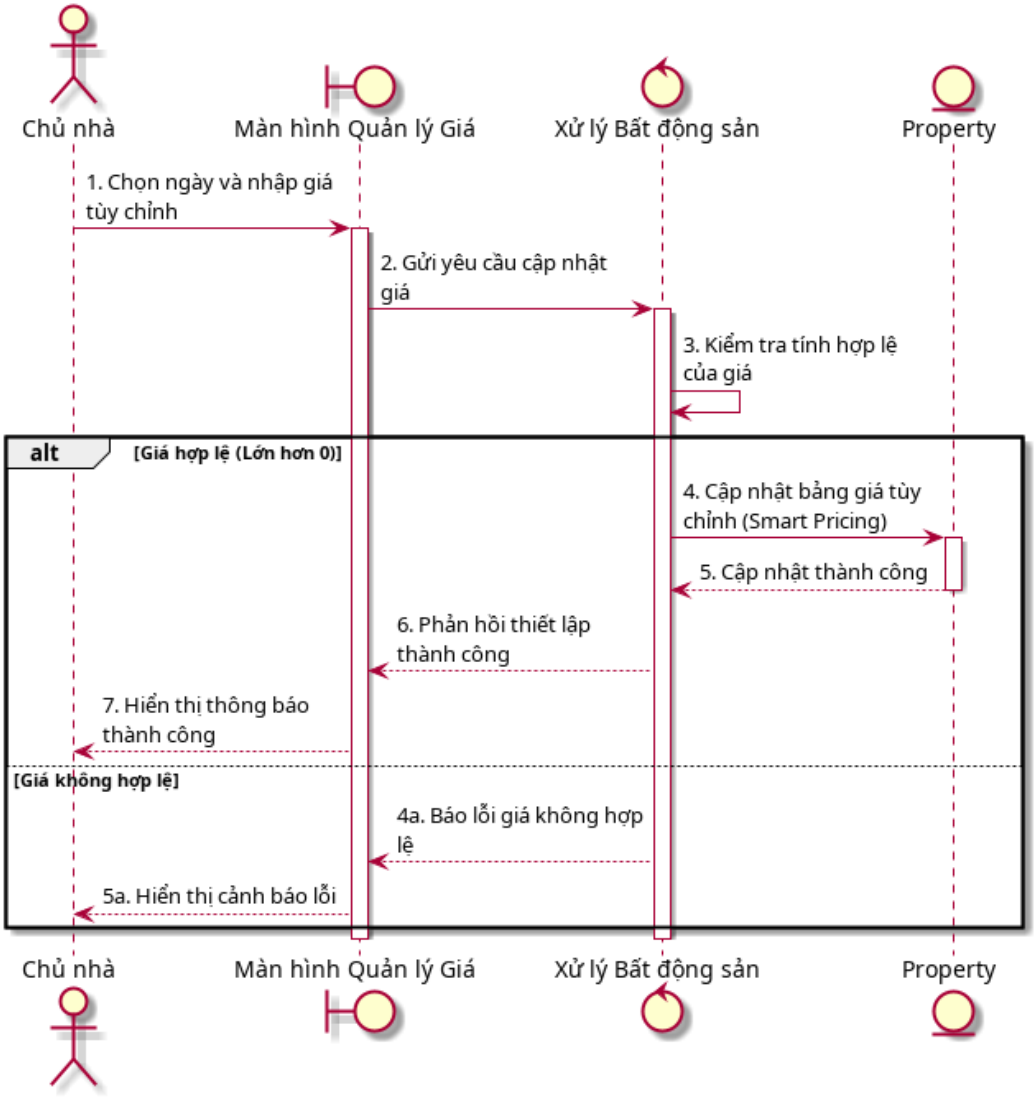
Hình 2.16. Sơ đồ Tuần tự: UC-09 Đăng tin Bất động sản mới

10. UC-10: Quản lý hình ảnh & tiện ích



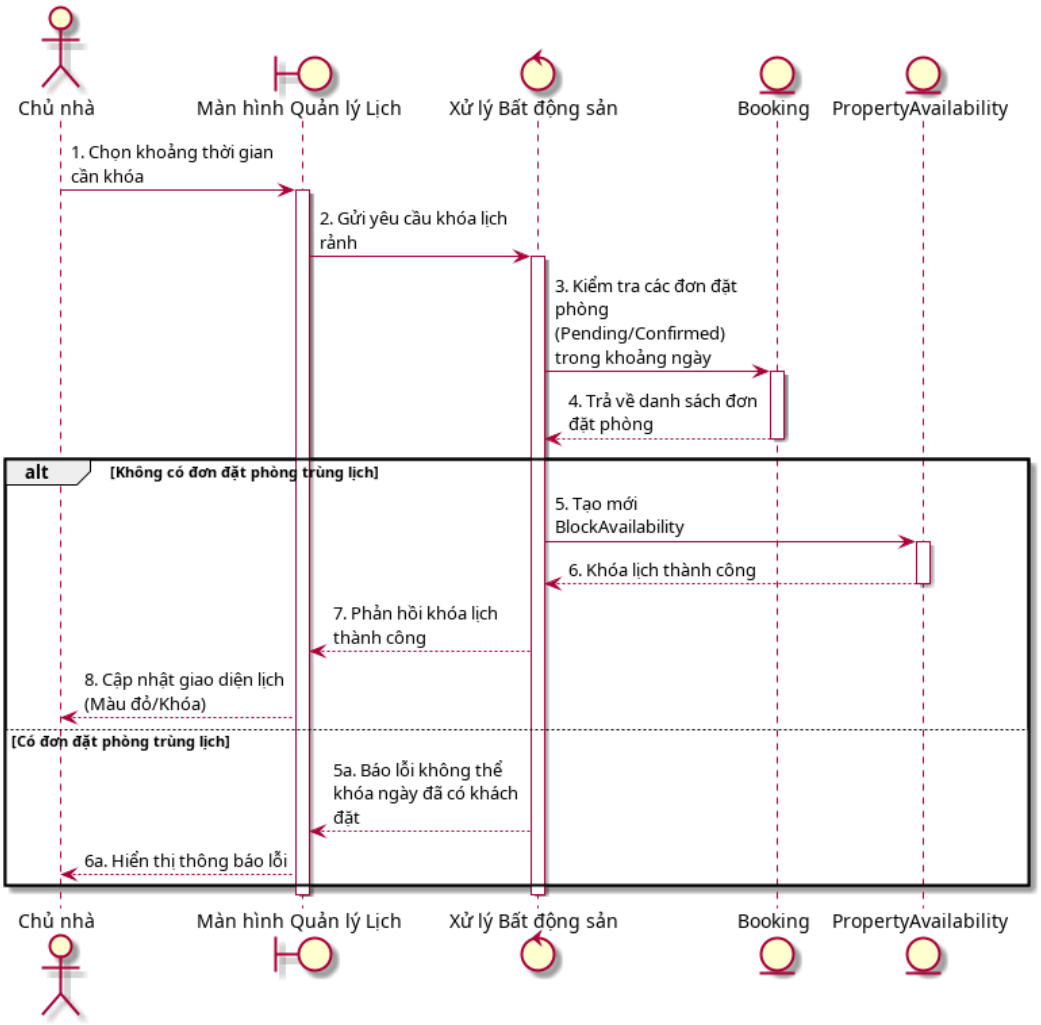
Hình 2.17. Sơ đồ Tuần tự: UC-10 Quản lý hình ảnh & tiện ích

11. UC-11: Thiết lập Giá theo ngày (Smart Pricing)



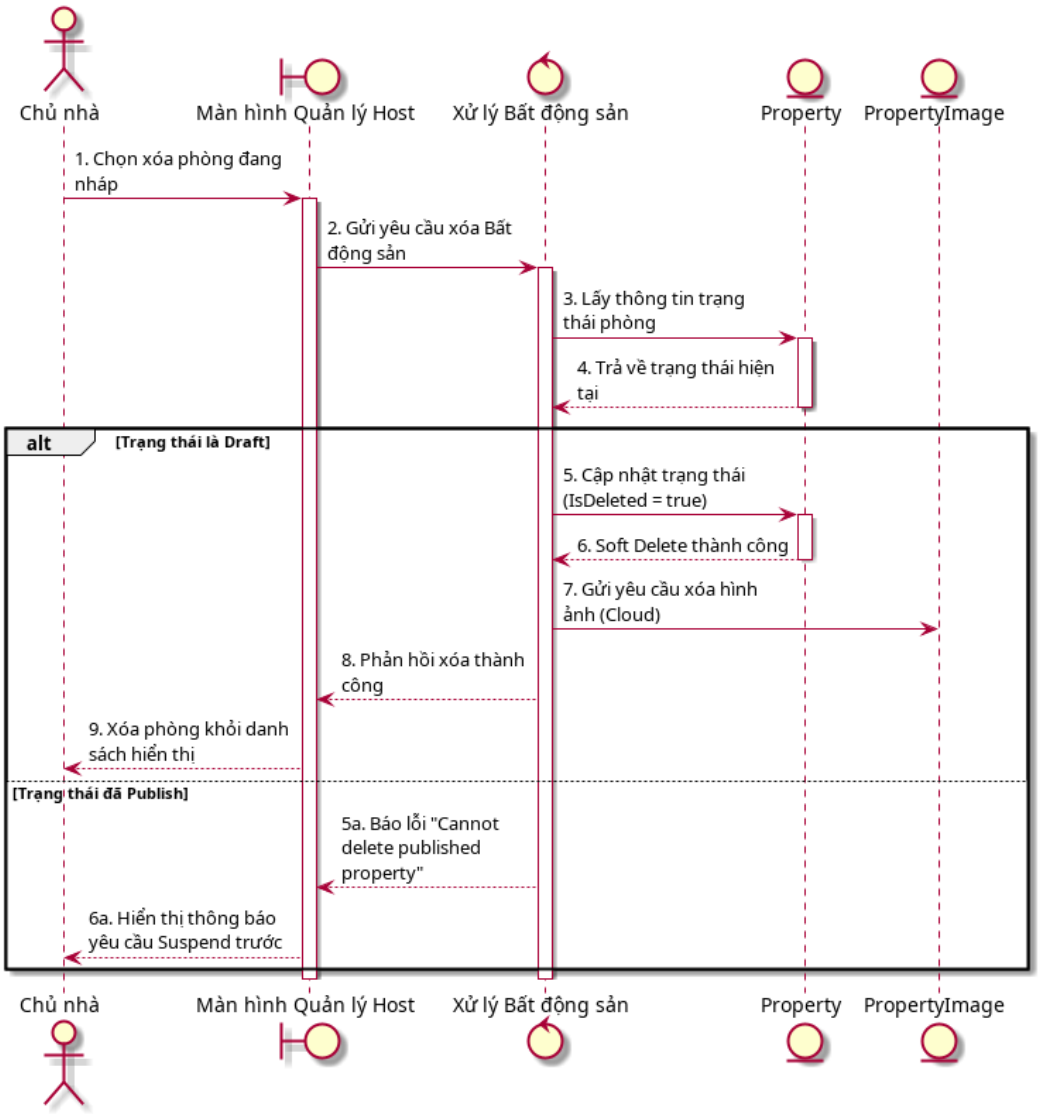
Hình 2.18. Sơ đồ Tuần tự: UC-11 Thiết lập Giá theo ngày

12. UC-12: Khóa lịch rảnh (Block Dates)



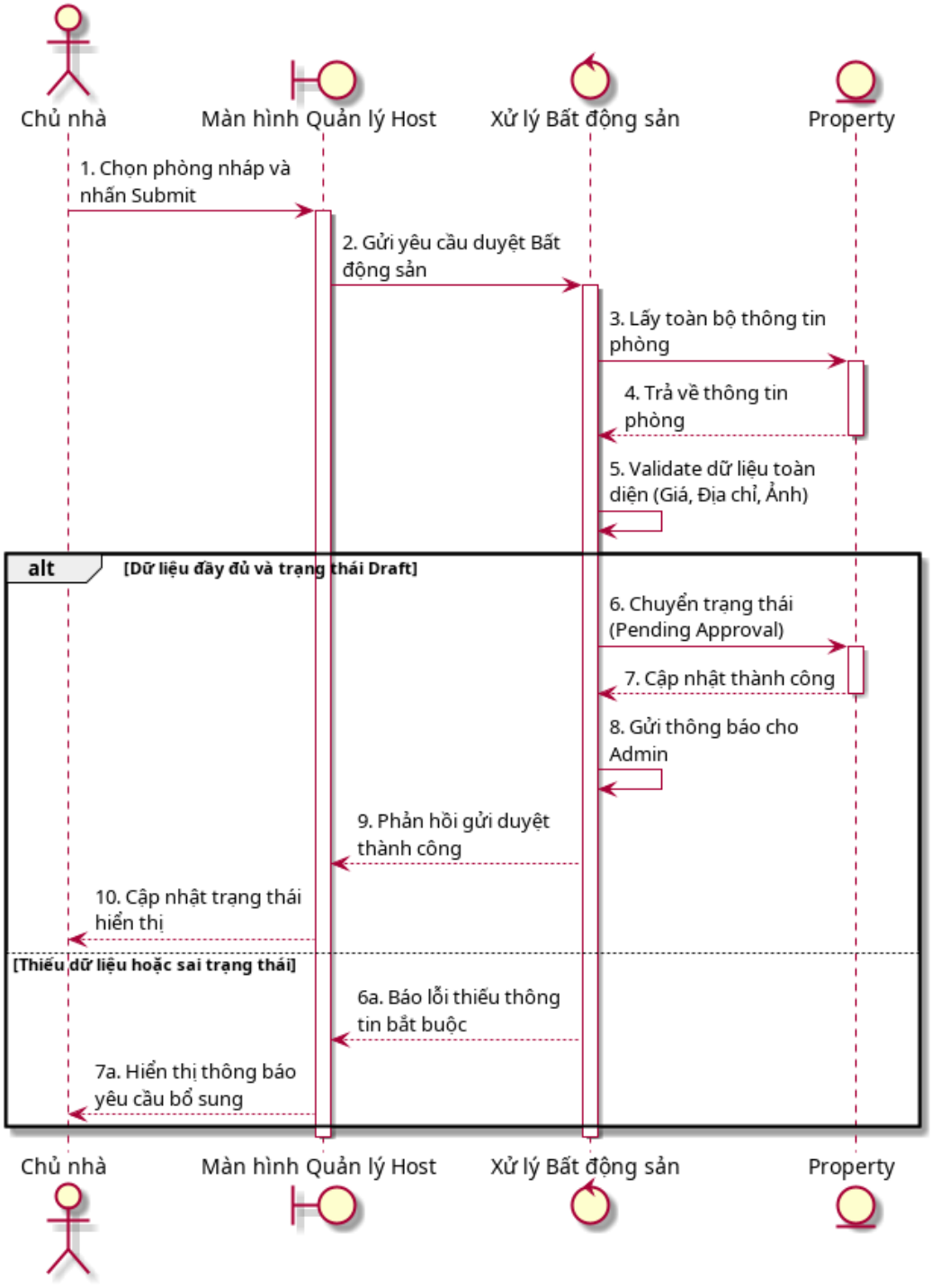
Hình 2.19. Sơ đồ Tuần tự: UC-12 Khóa lịch rảnh

13. UC-13: Xóa Bất động sản



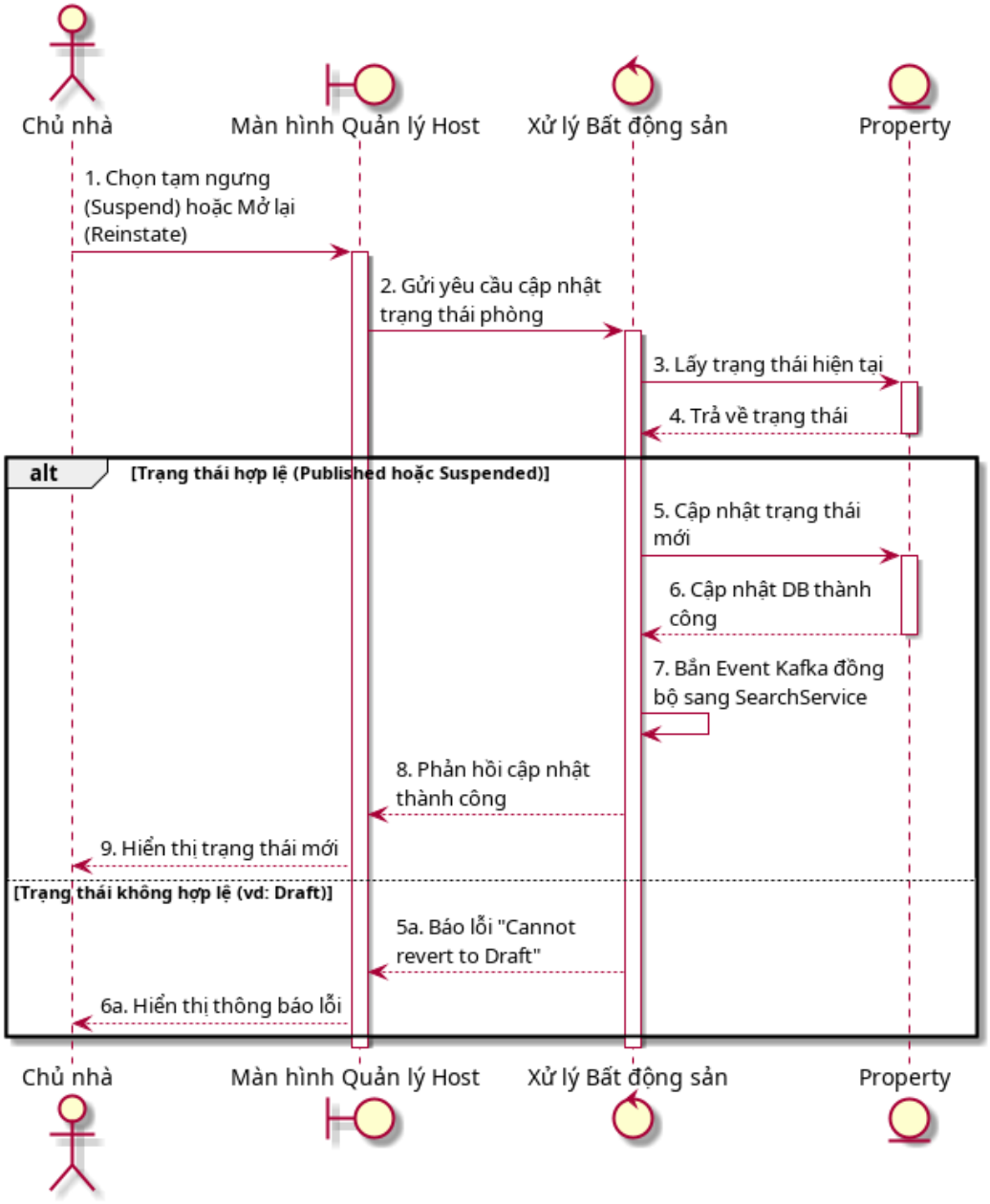
Hình 2.20. Sơ đồ Tuần tự: UC-13 Xóa Bất động sản

14. UC-14: Gửi duyệt Bất động sản (Submit)



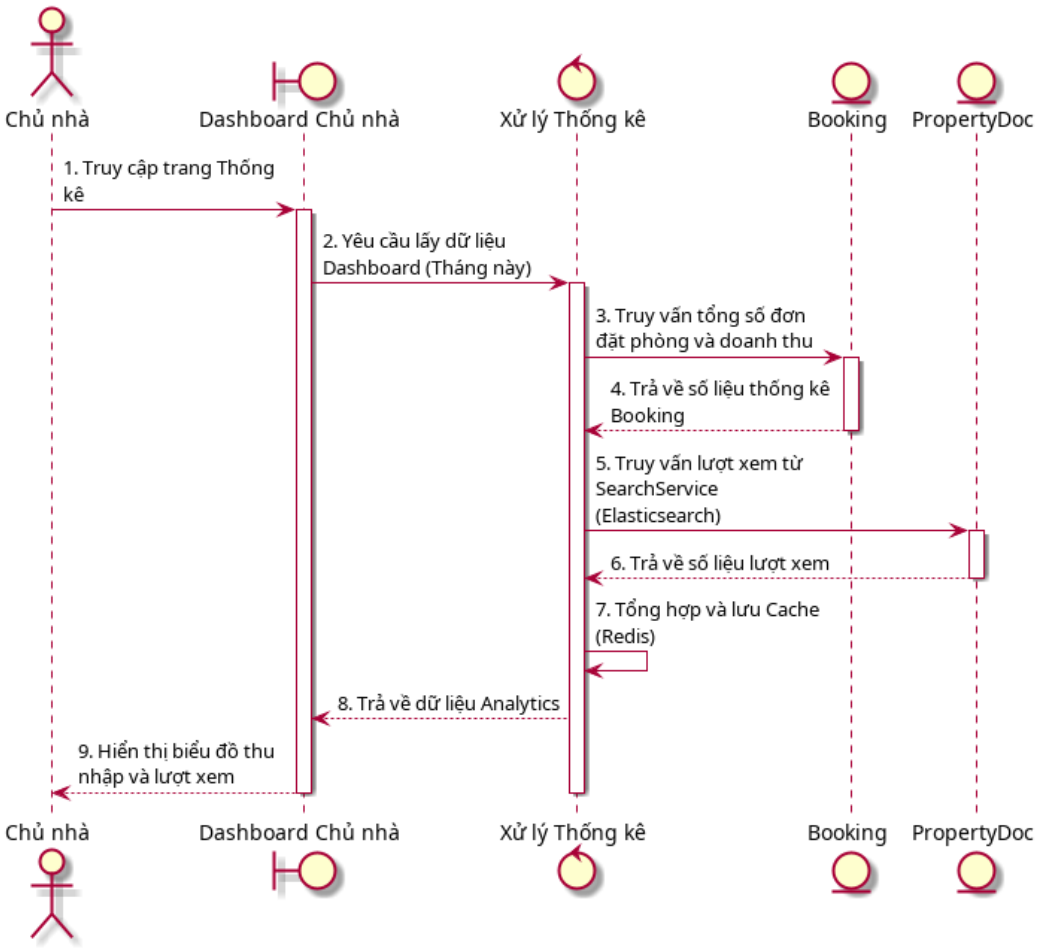
Hình 2.21. Sơ đồ Tuần tự: UC-14 Gửi duyệt Bất động sản

15. UC-15: Cập nhật trạng thái phòng



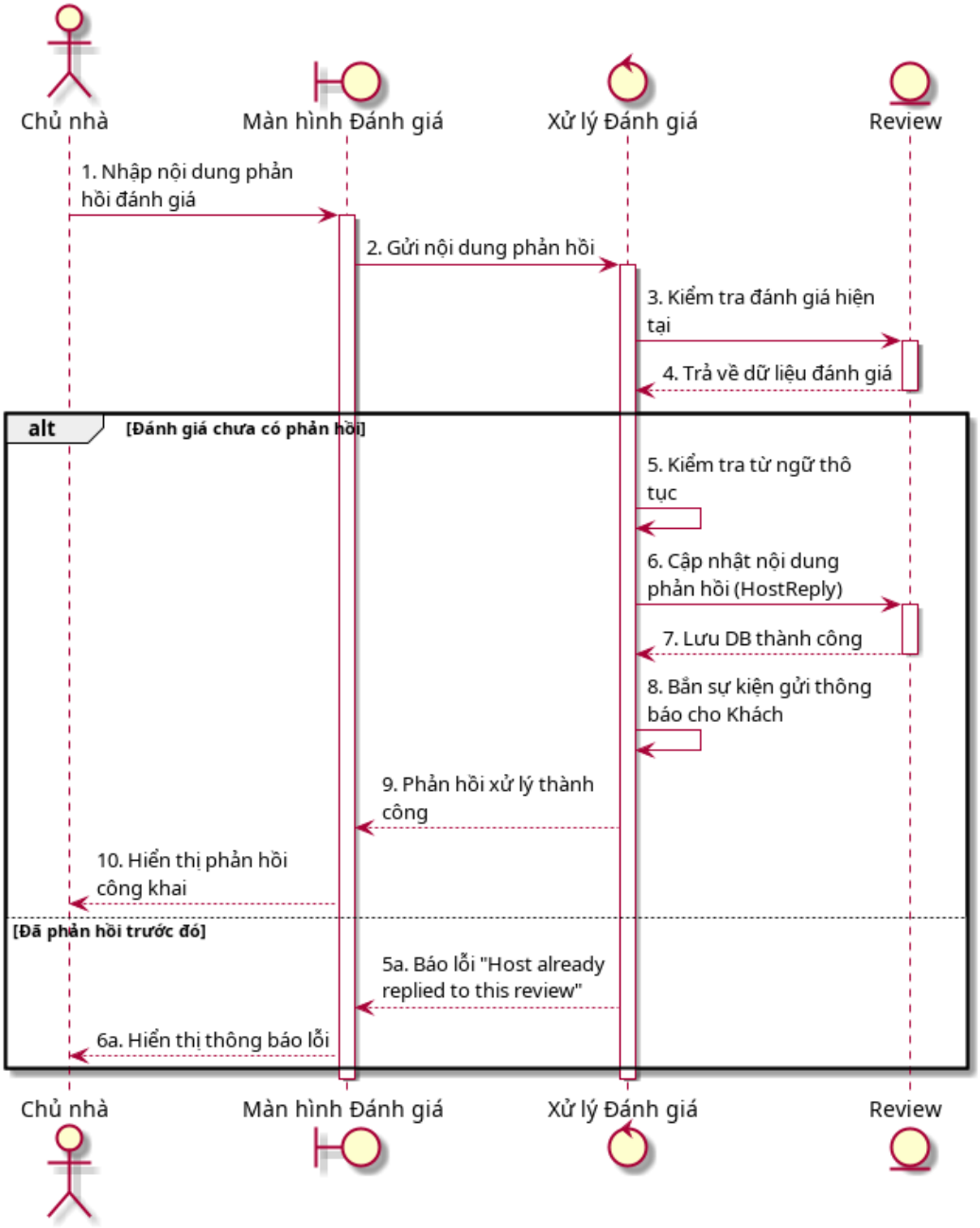
Hình 2.22. Sơ đồ Tuần tự: UC-15 Cập nhật trạng thái phòng

16. UC-16: Xem Dashboard Chủ nhà



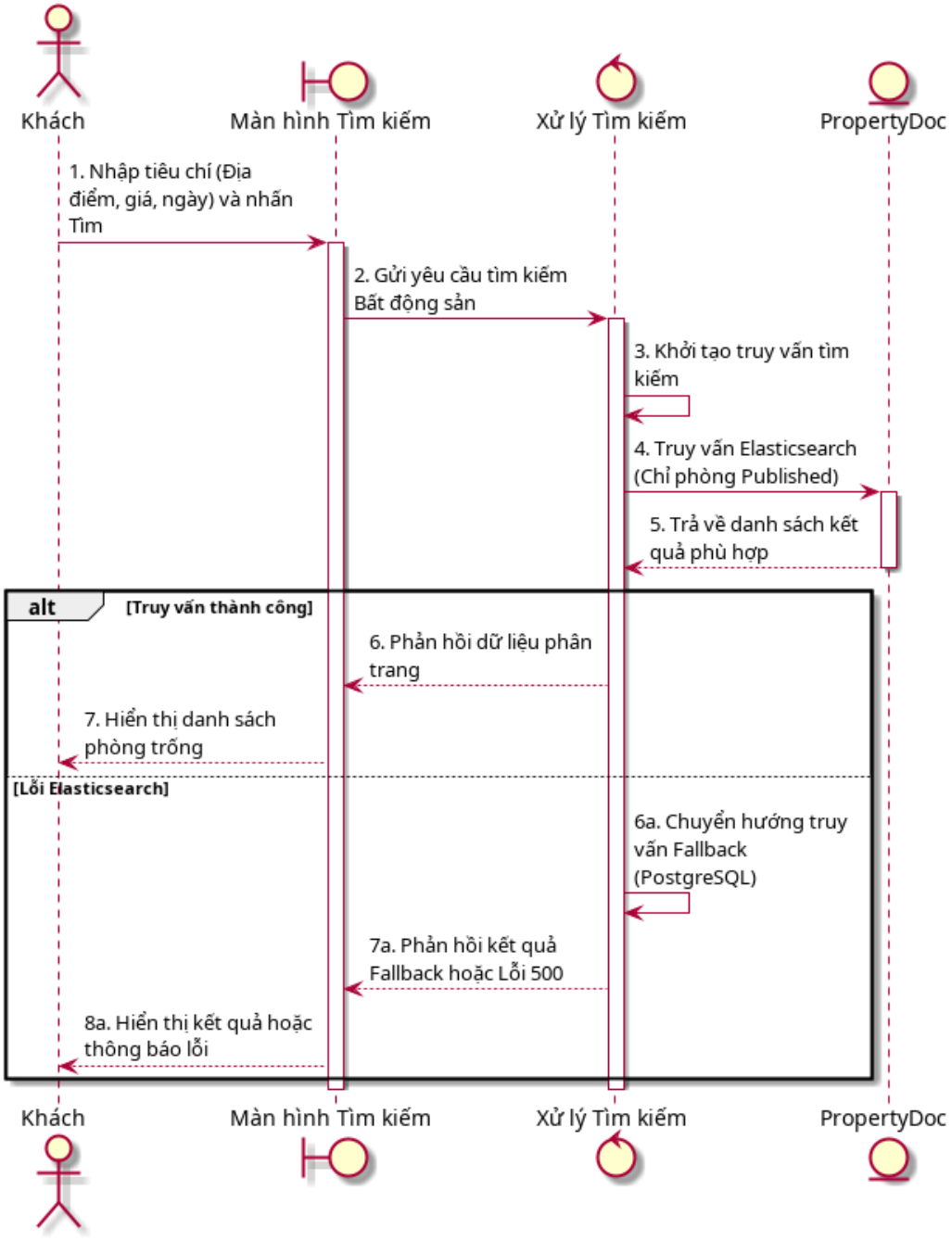
Hình 2.23. Sơ đồ Tuần tự: UC-16 Xem Dashboard Chủ nhà

17. UC-17: Phản hồi đánh giá (Host Reply)



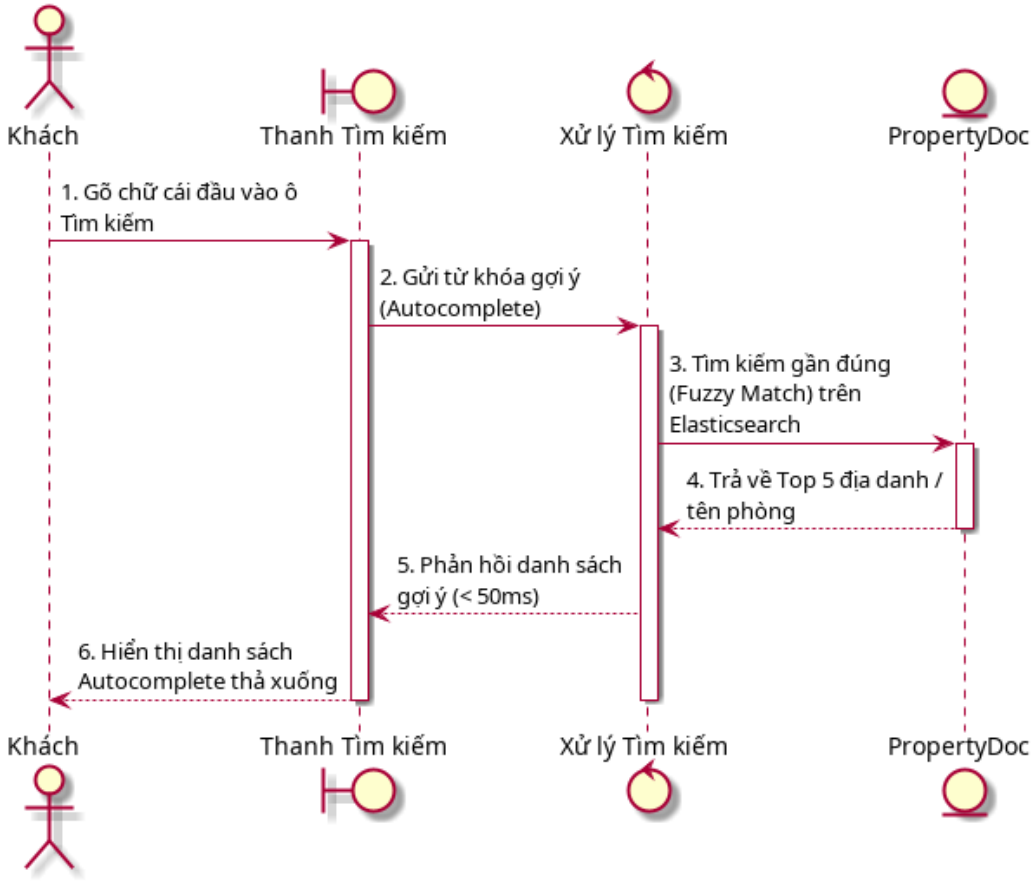
Hình 2.24. Sơ đồ Tuần tự: UC-17 Phản hồi đánh giá

18. UC-18: Tìm kiếm Bất động sản (Elasticsearch)



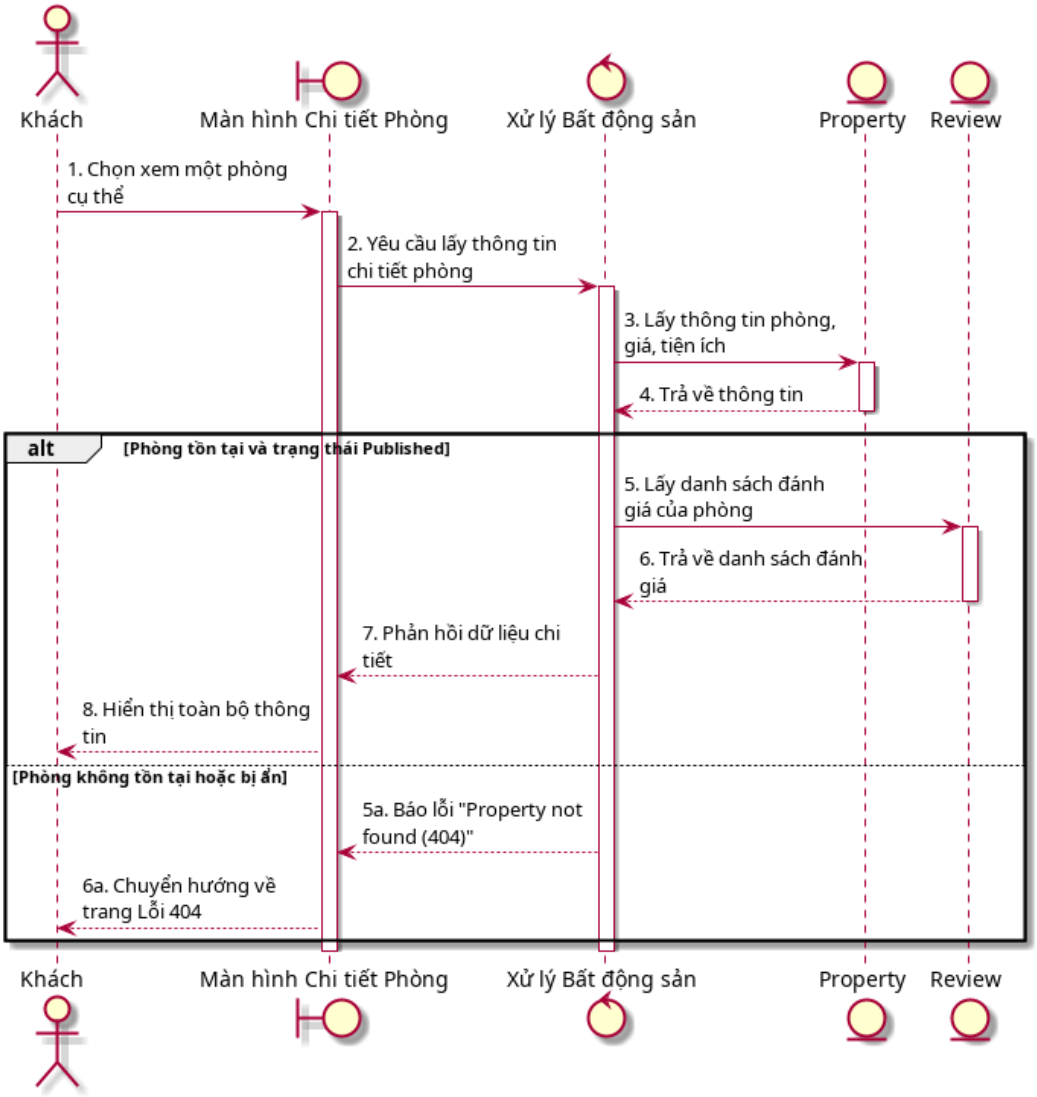
Hình 2.25. Sơ đồ Tuần tự: UC-18 Tìm kiếm Bất động sản

19. UC-19: Tìm kiếm gợi ý (Autocomplete)



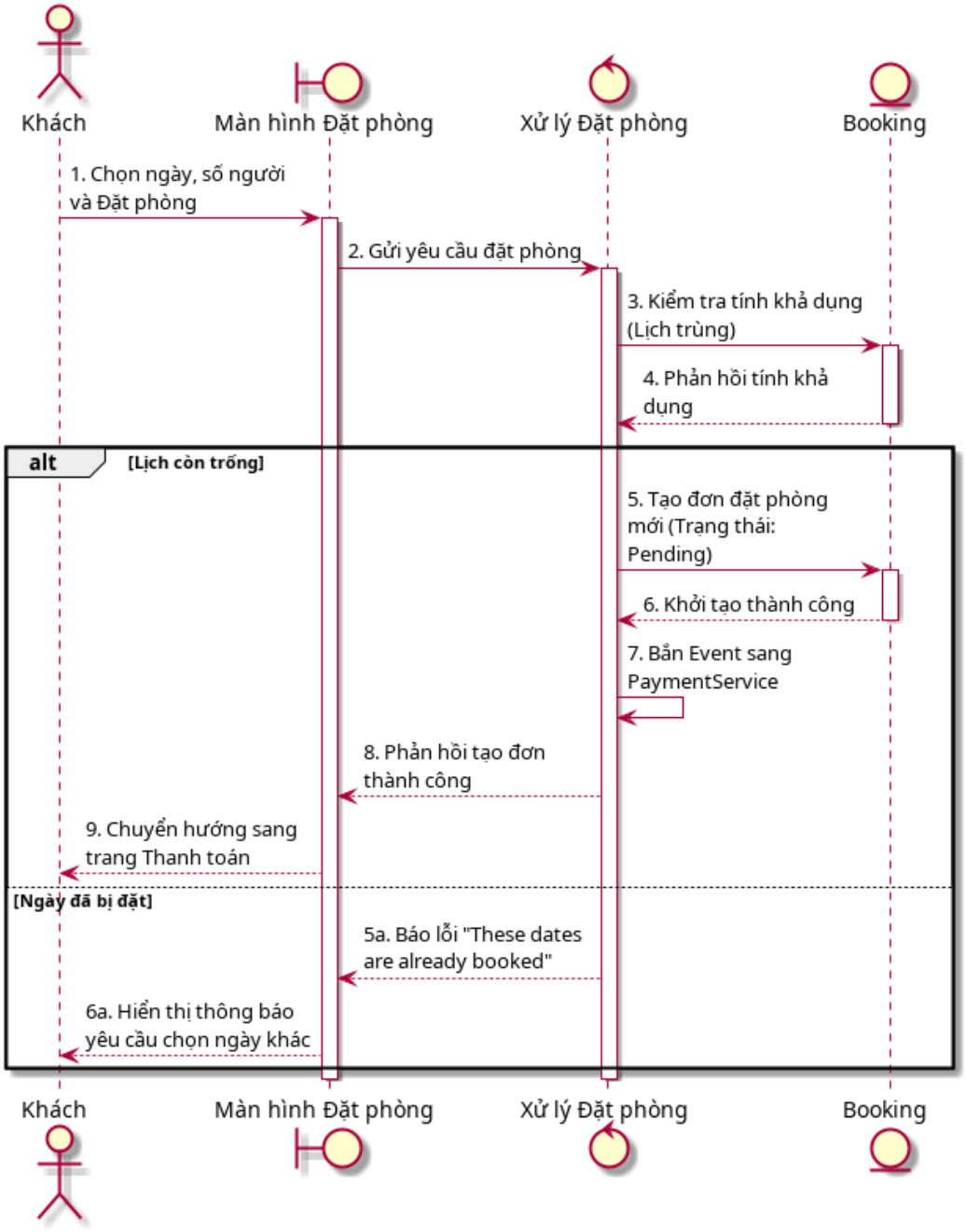
Hình 2.26. Sơ đồ Tuần tự: UC-19 Tìm kiếm gợi ý

20. UC-20: Xem chi tiết Bất động sản



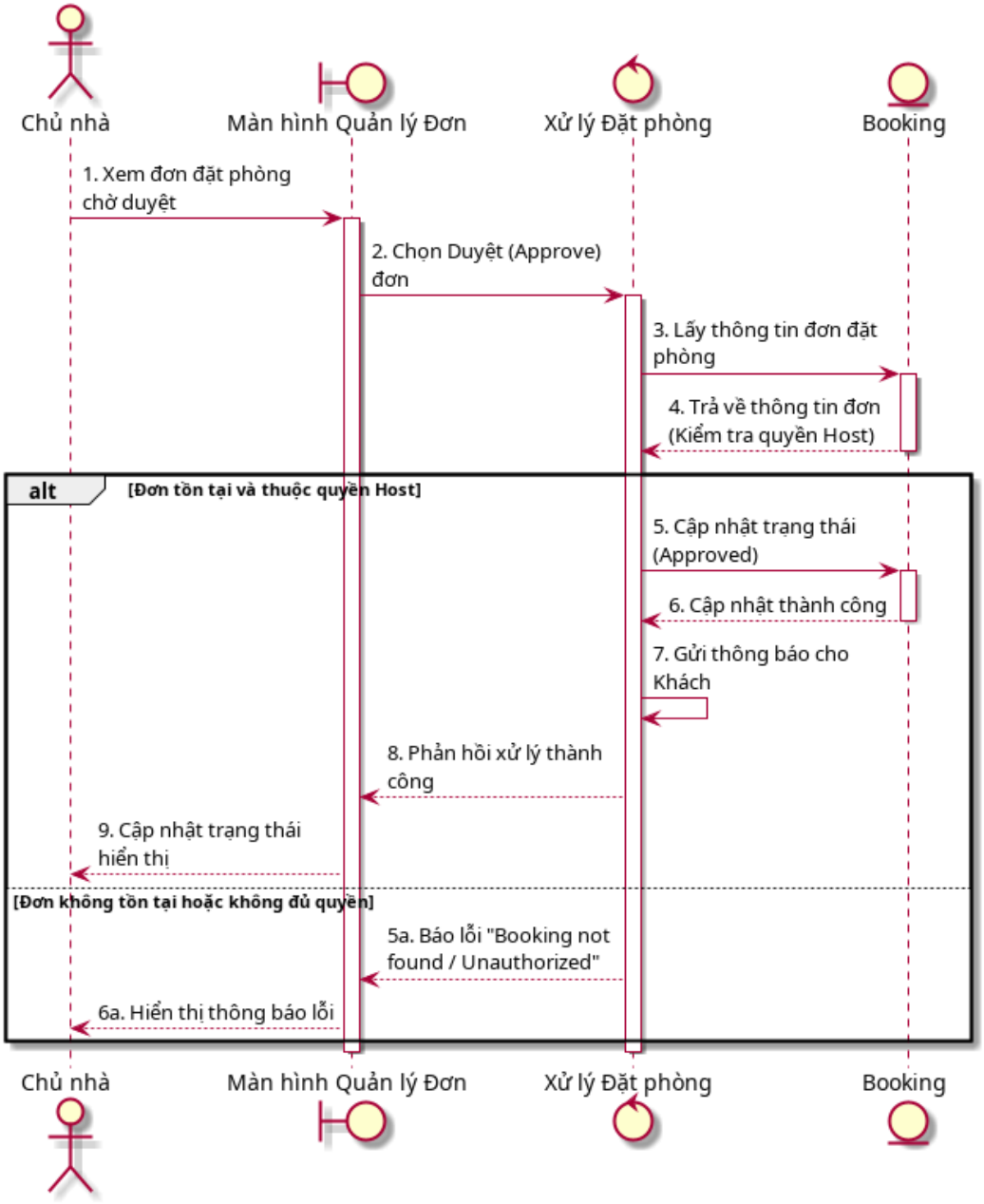
Hình 2.27. Sơ đồ Tuần tự: UC-20 Xem chi tiết Bất động sản

21. UC-21: Đặt phòng (Create Booking)



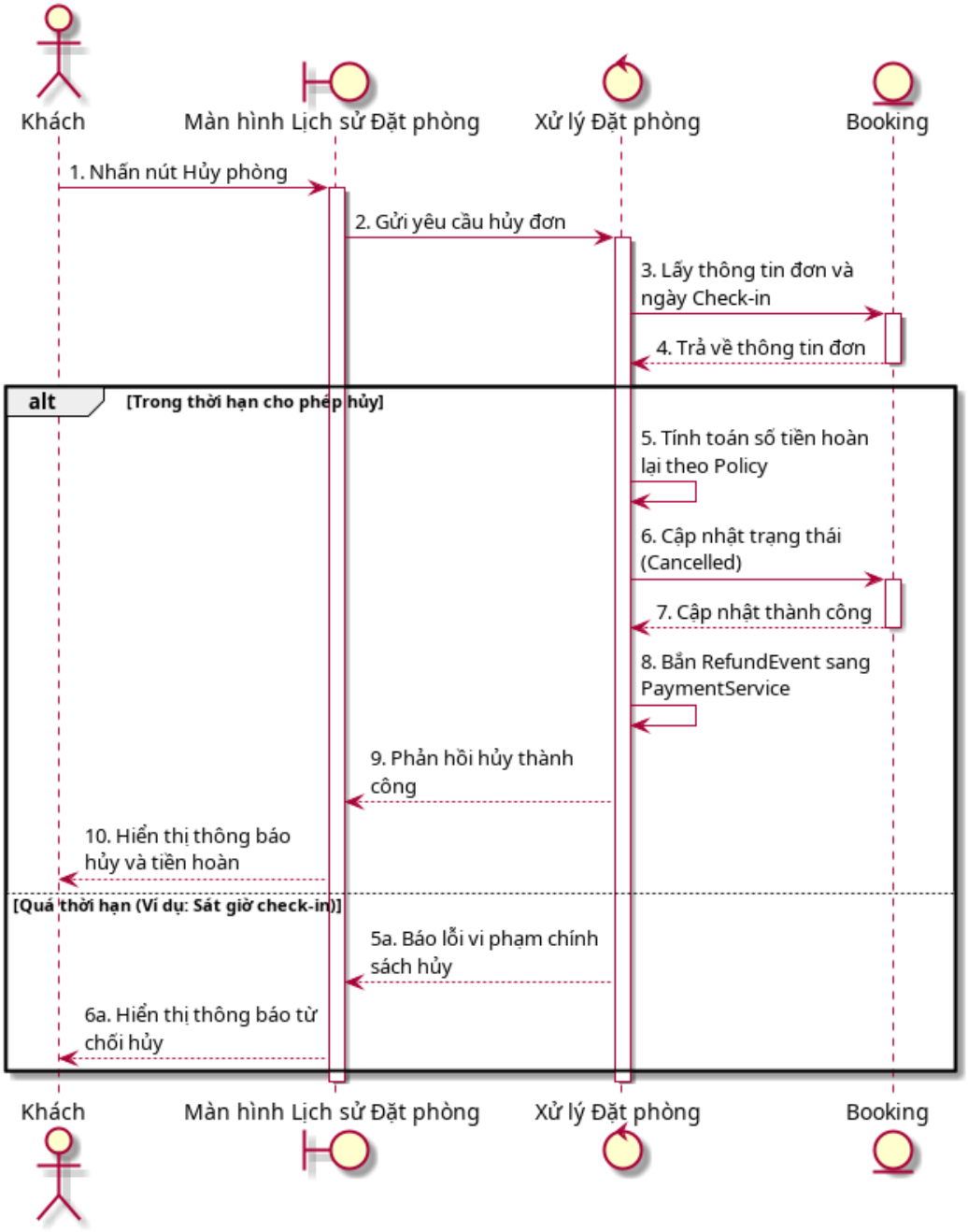
Hình 2.28. Sơ đồ Tuần tự: UC-21 Đặt phòng

22. UC-22: Chủ nhà Duyệt/Từ chối đặt phòng



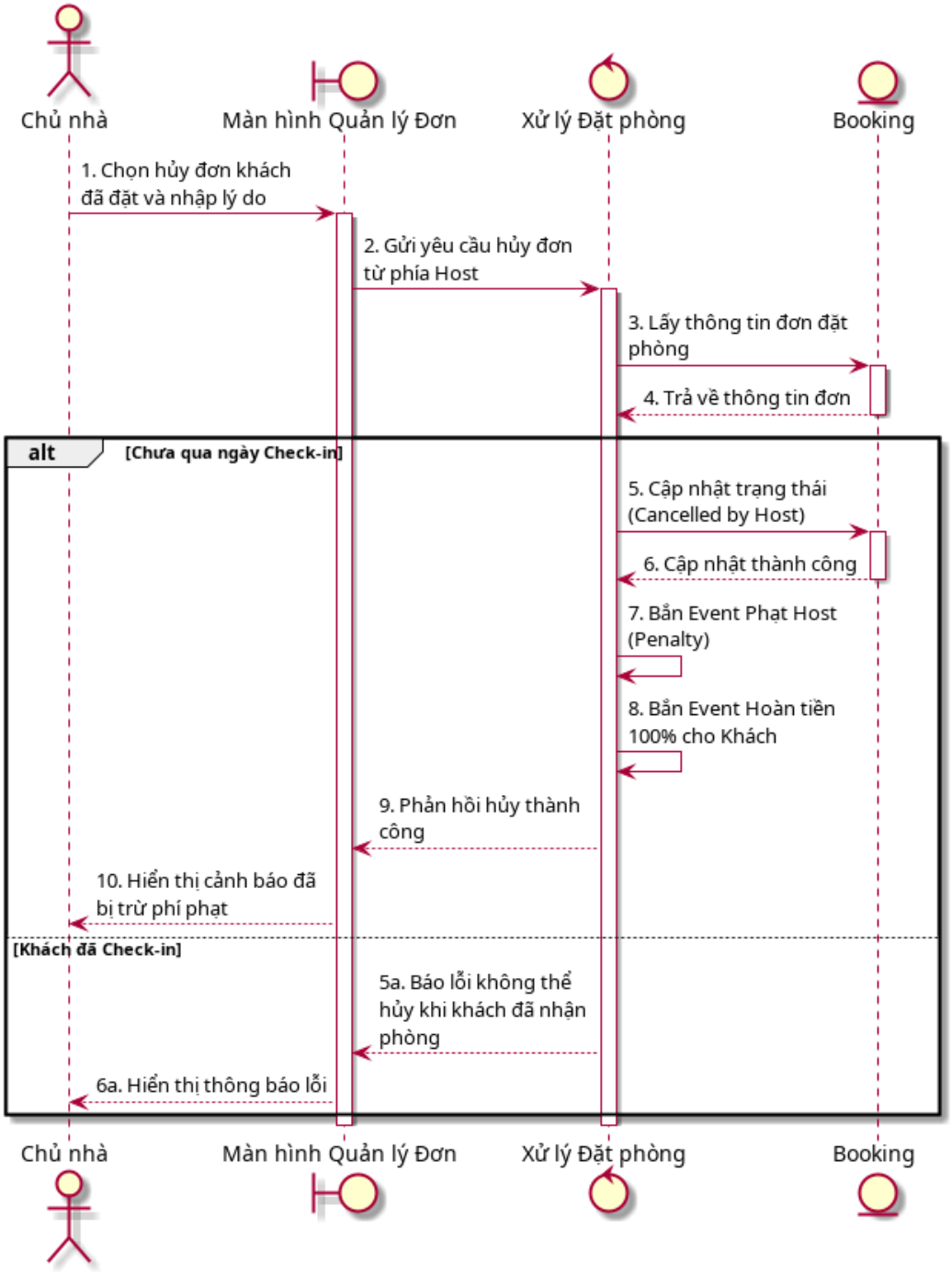
Hình 2.29. Sơ đồ Tuần tự: UC-22 Chủ nhà Duyệt/Từ chối đặt phòng

23. UC-23: Khách Hủy đặt phòng



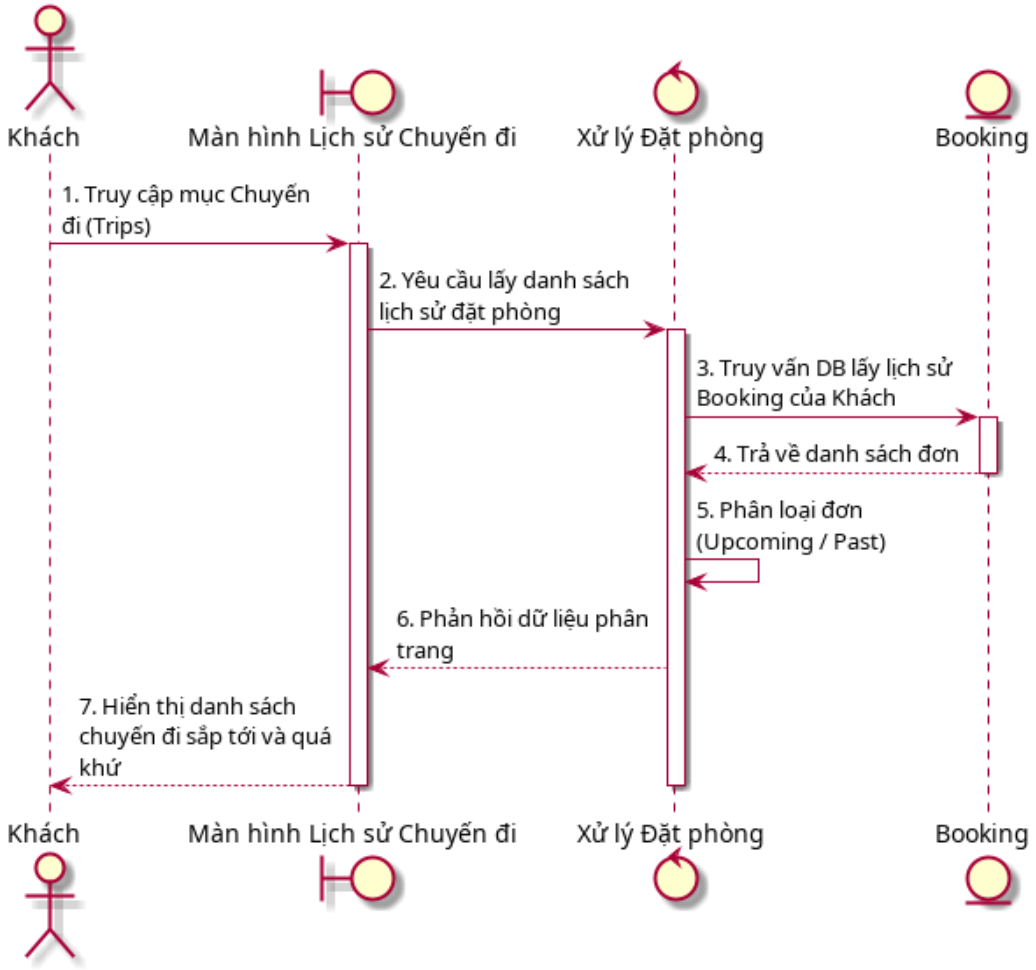
Hình 2.30. Sơ đồ Tuần tự: UC-23 Khách Hủy đặt phòng

24. UC-24: Chủ nhà Hủy đặt phòng



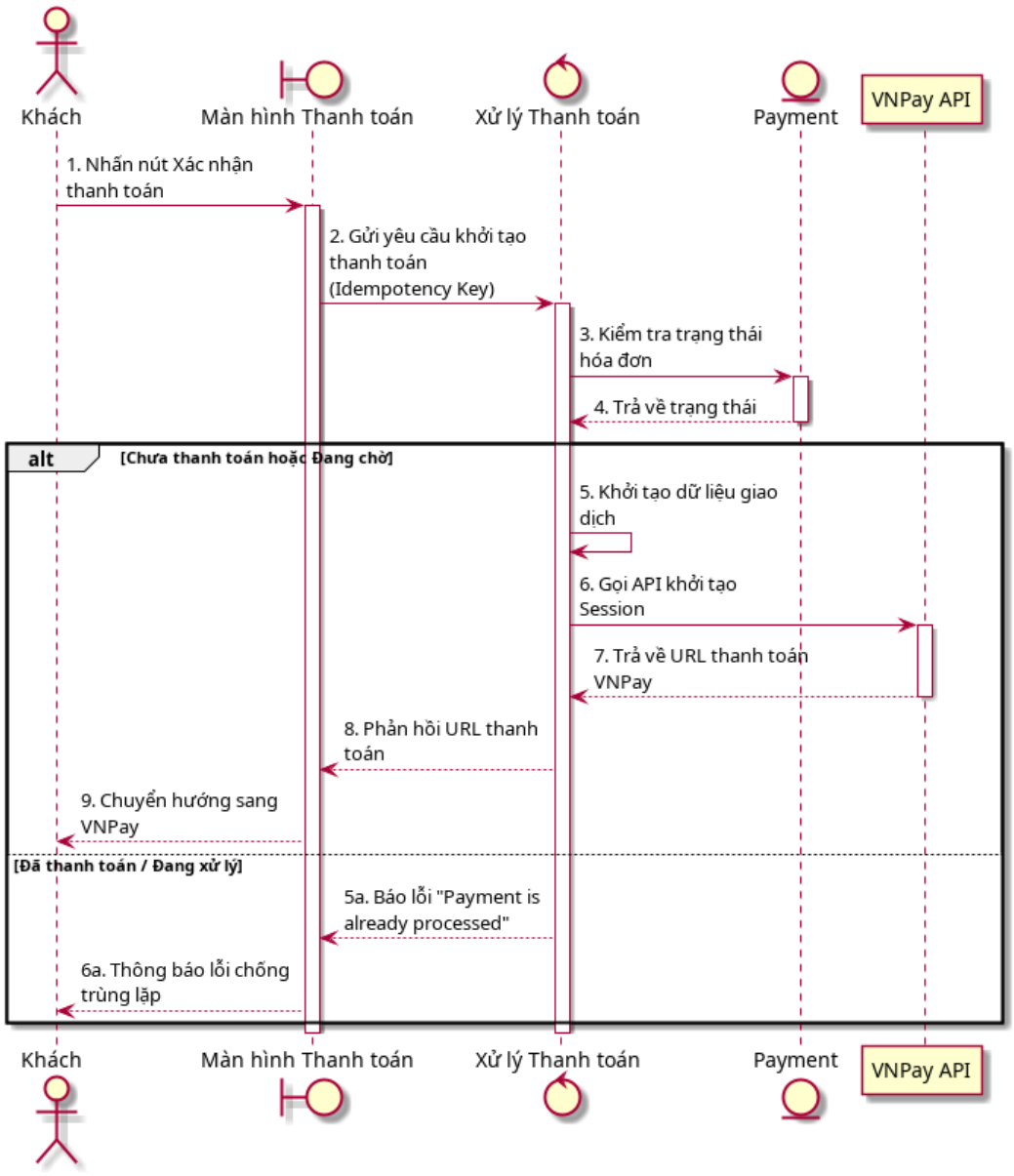
Hình 2.31. Sơ đồ Tuần tự: UC-24 Chủ nhà Hủy đặt phòng

25. UC-25: Xem Lịch sử Đặt phòng



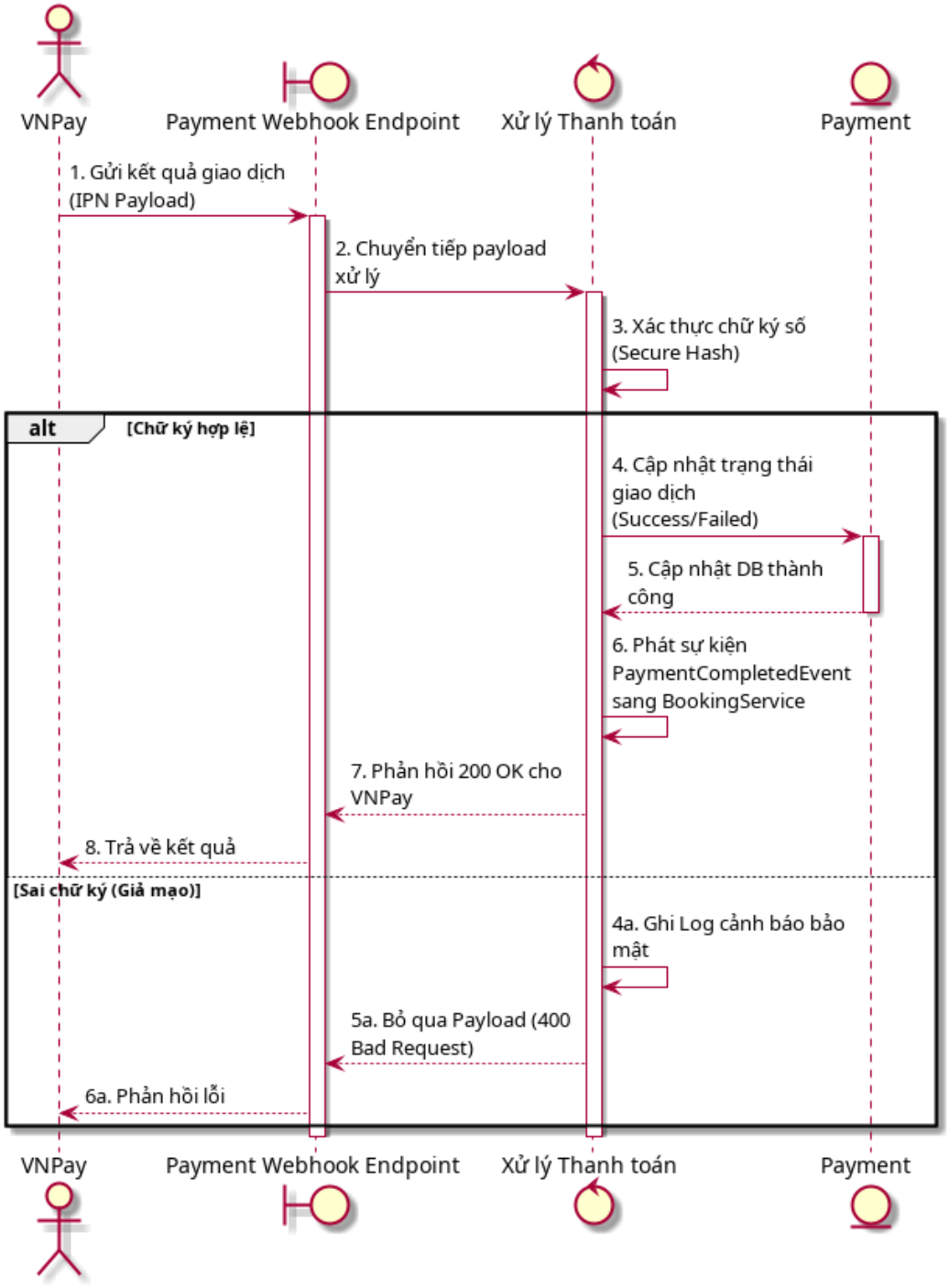
Hình 2.32. Sơ đồ Tuần tự: UC-25 Xem Lịch sử Đặt phòng

26. UC-26: Khởi tạo Thanh toán



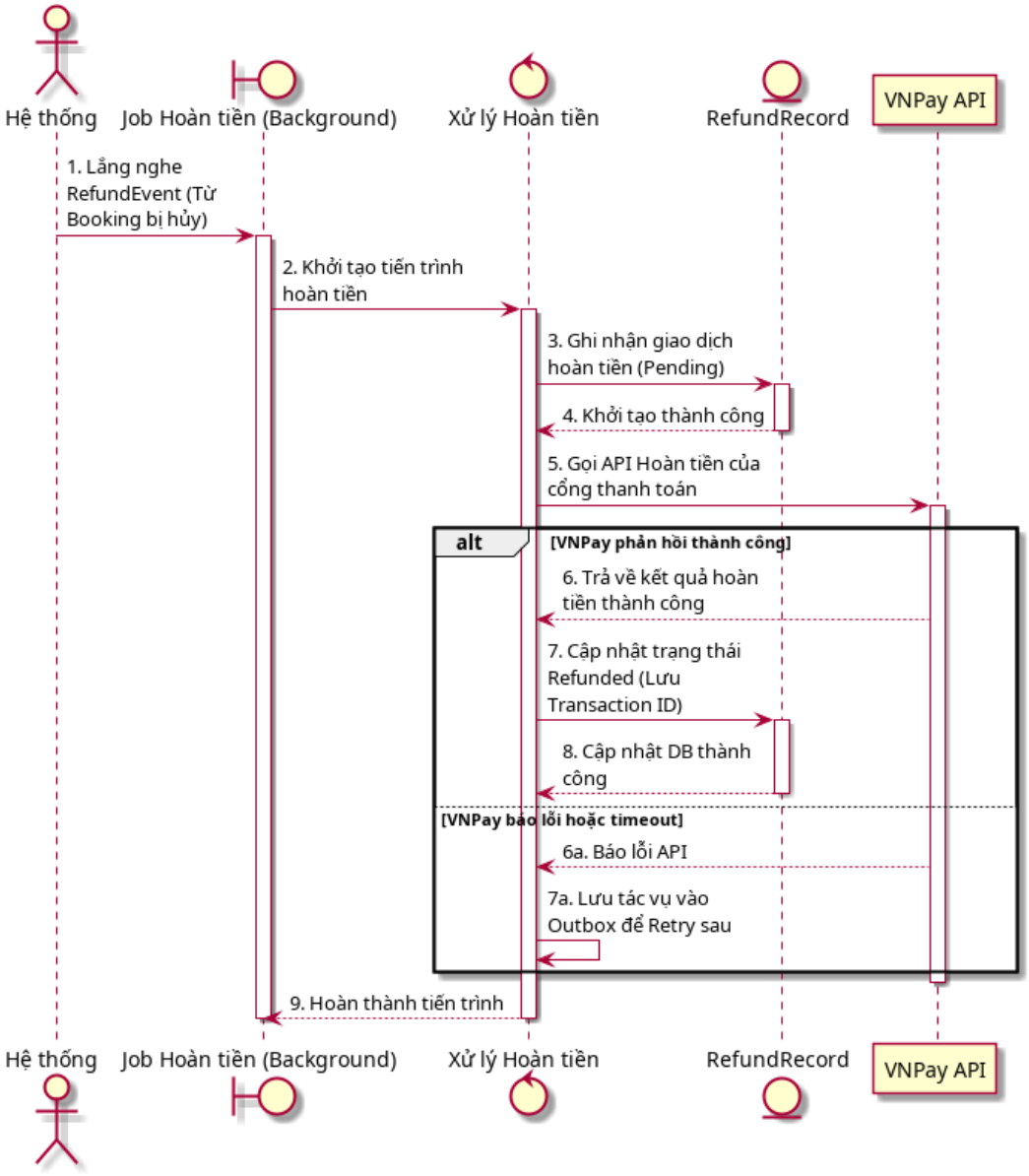
Hình 2.33. Sơ đồ Tuần tự: UC-26 Khởi tạo Thanh toán

27. UC-27: Xử lý Webhook Thanh toán



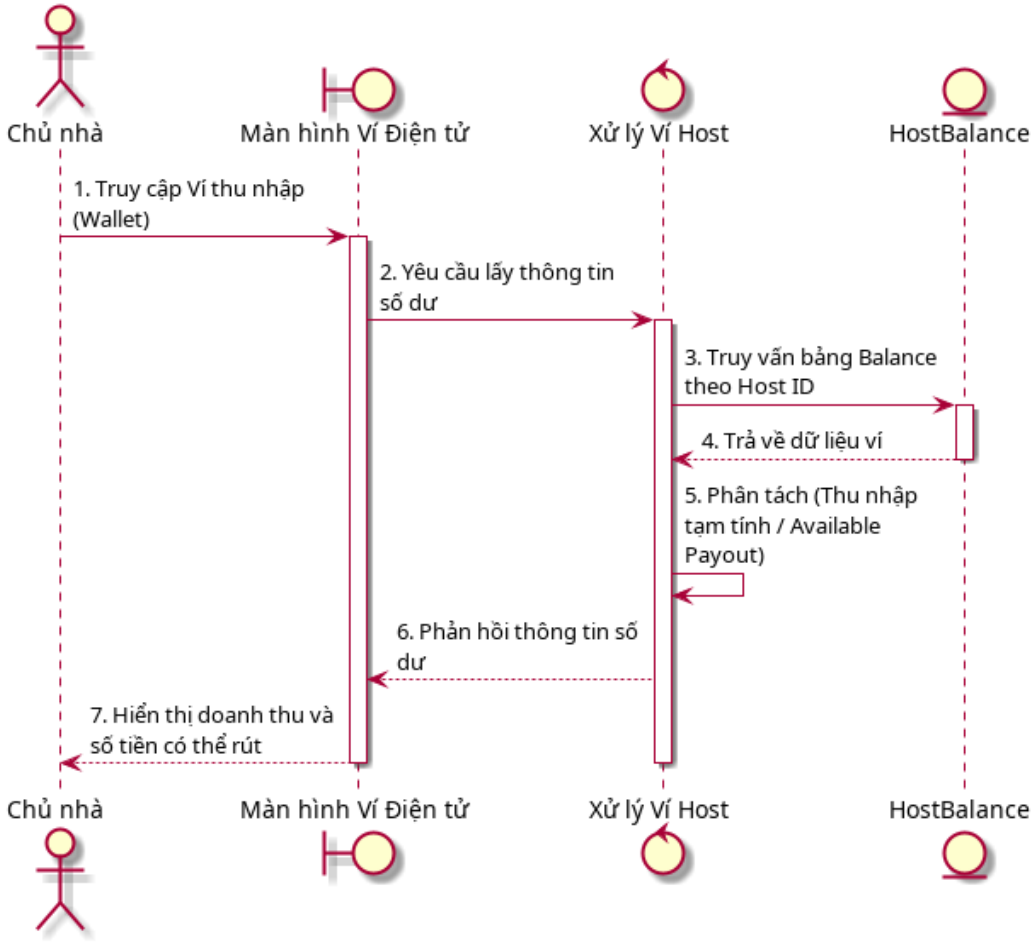
Hình 2.34. Sơ đồ Tuần tự: UC-27 Xử lý Webhook Thanh toán

28. UC-28: Tự động Hoàn tiền (Refund)



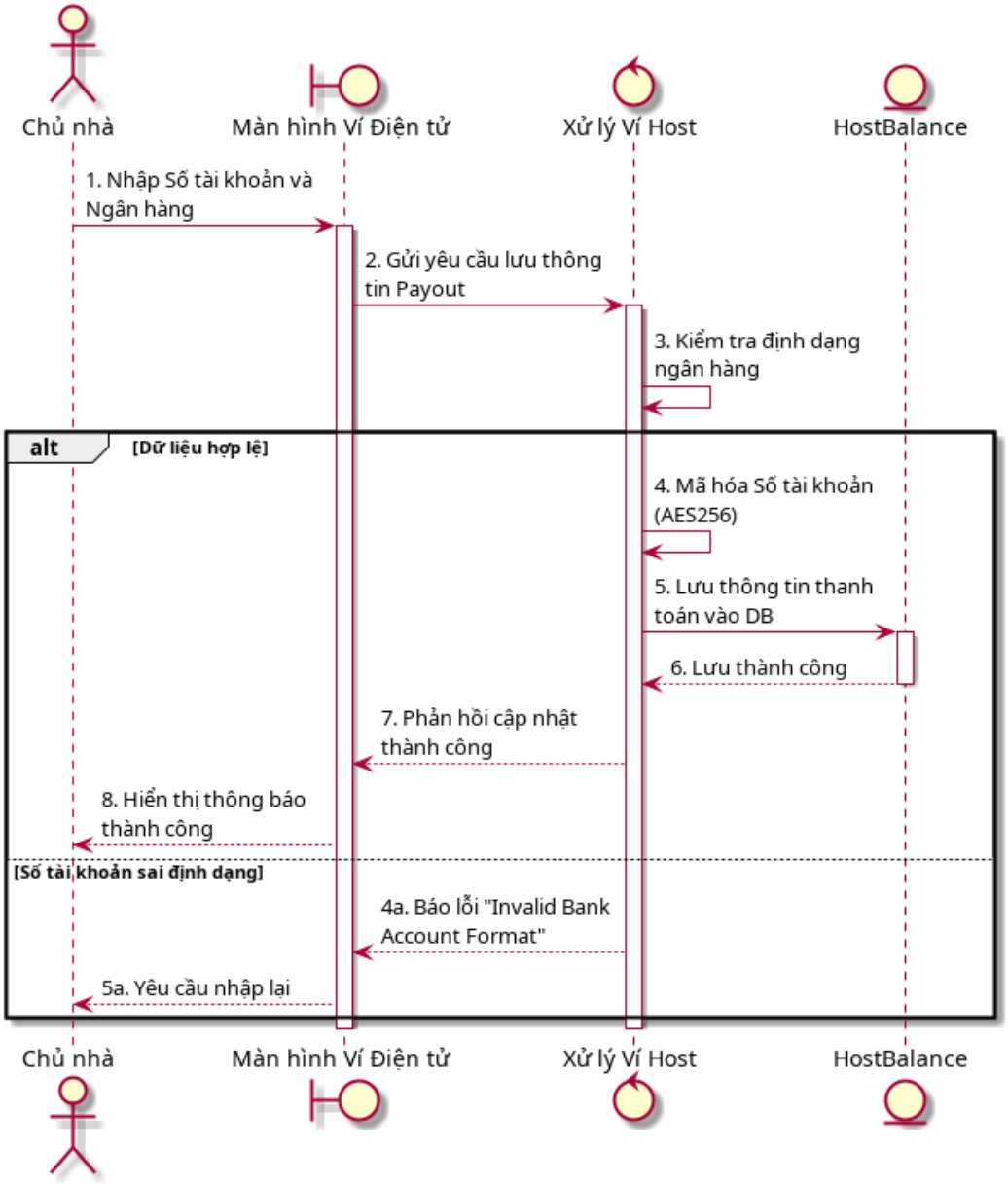
Hình 2.35. Sơ đồ Tuần tự: UC-28 Tự động Hoàn tiền

29. UC-29: Xem số dư ví Host



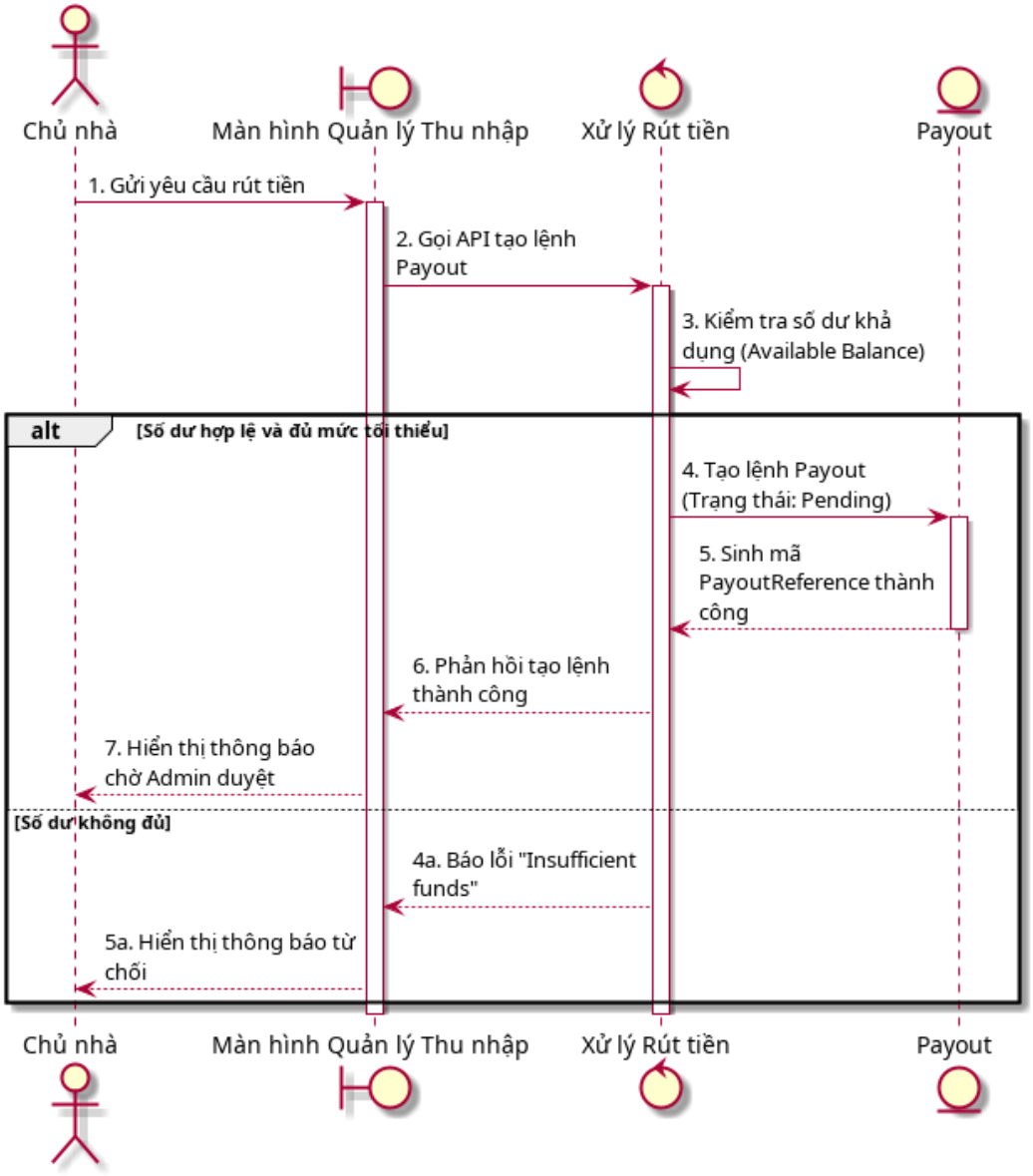
Hình 2.36. Sơ đồ Tuần tự: UC-29 Xem số dư ví Host

30. UC-30: Cập nhật Thông tin Ngân hàng



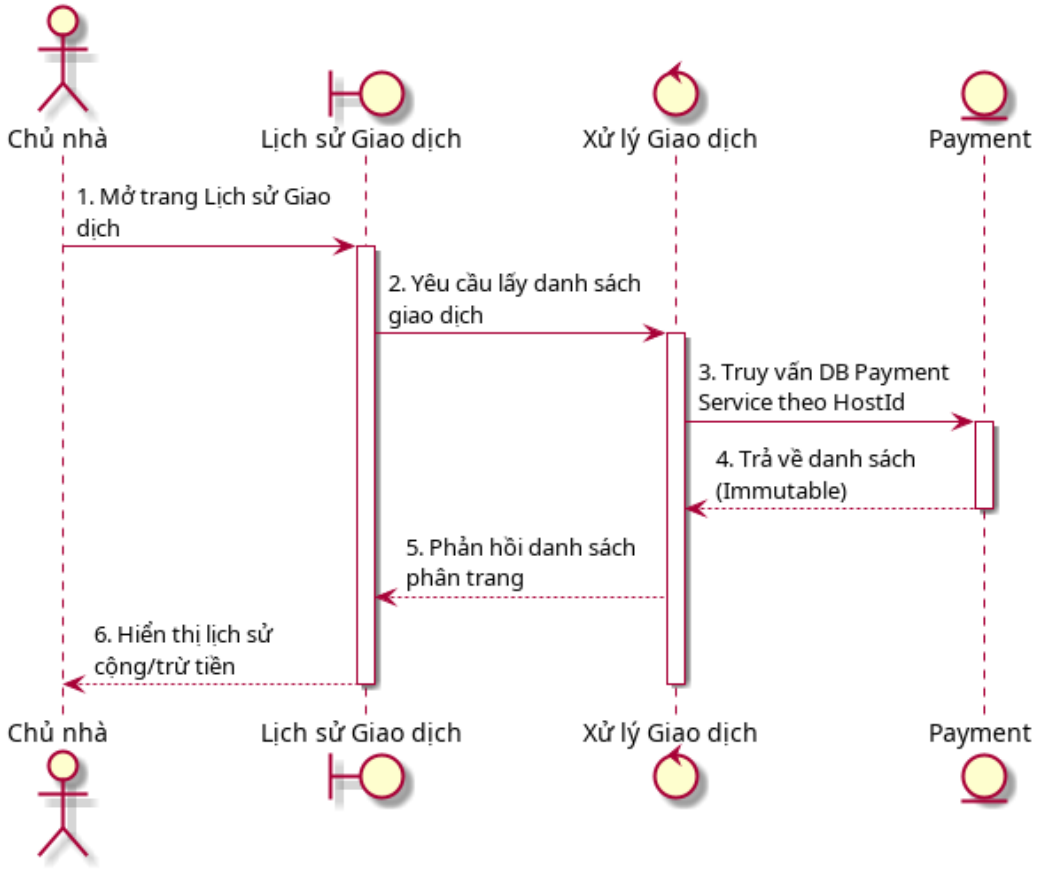
Hình 2.37. Sơ đồ Tuần tự: UC-30 Cập nhật Thông tin Ngân hàng

31. UC-31: Yêu cầu Rút tiền (Payout)



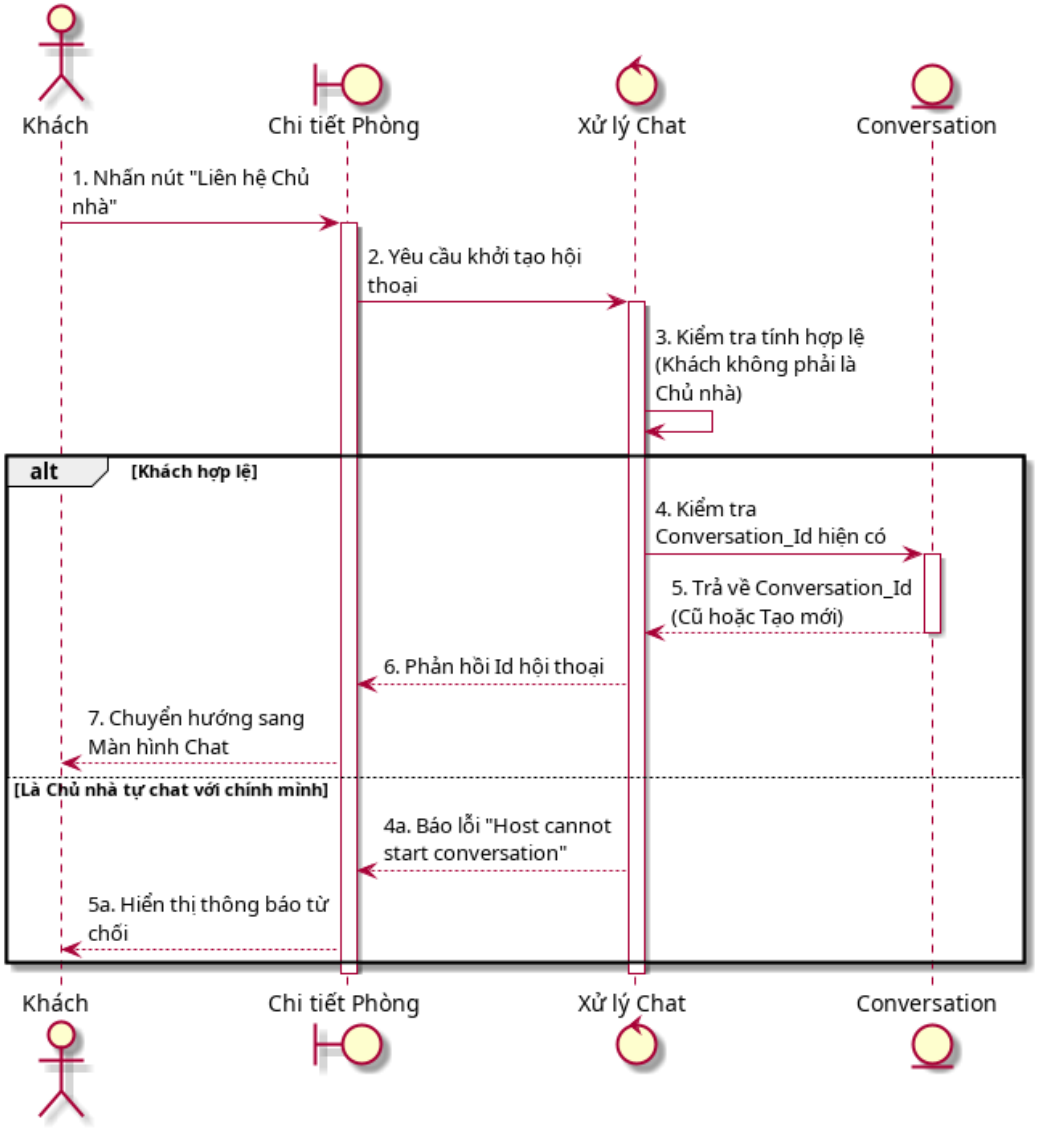
Hình 2.38. Sơ đồ Tuần tự: UC-31 Yêu cầu Rút tiền

32. UC-32: Xem Lịch sử Giao dịch



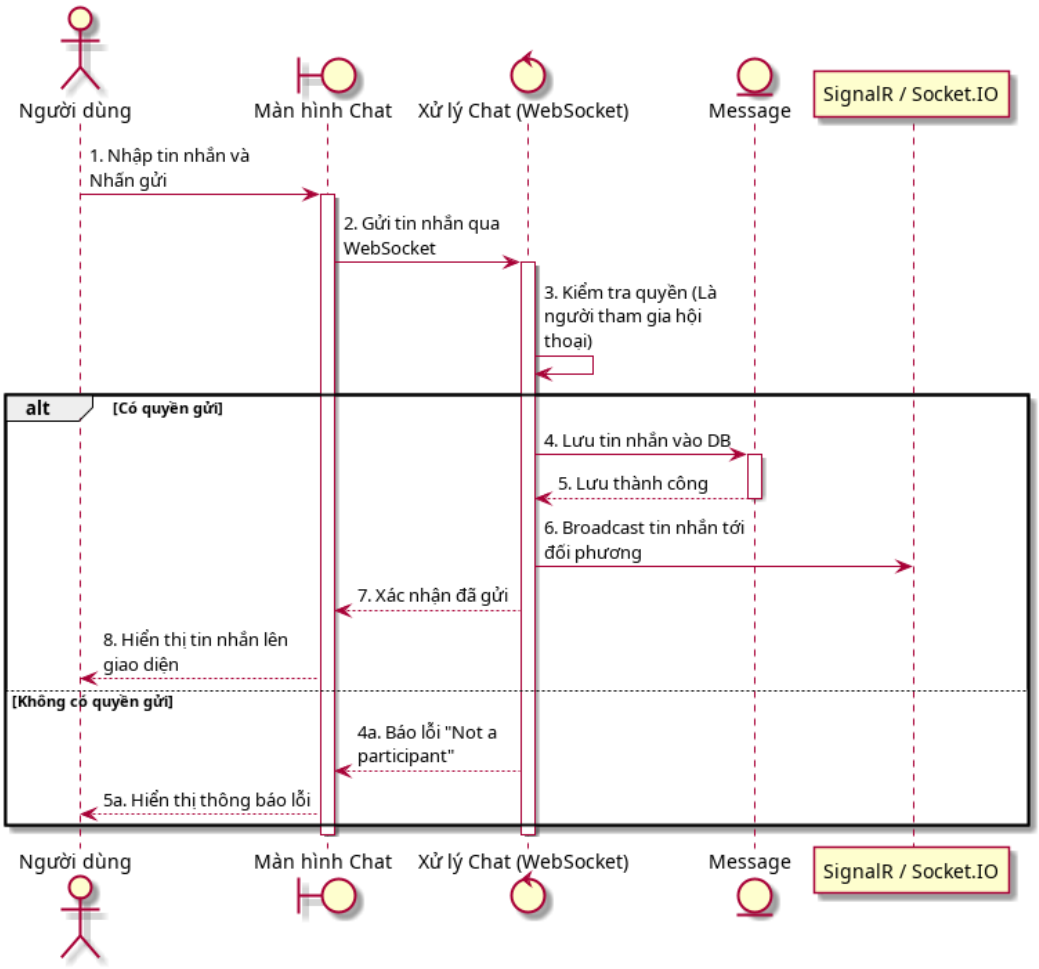
Hình 2.39. Sơ đồ Tuần tự: UC-32 Xem Lịch sử Giao dịch

33. UC-33: Tạo cuộc hội thoại



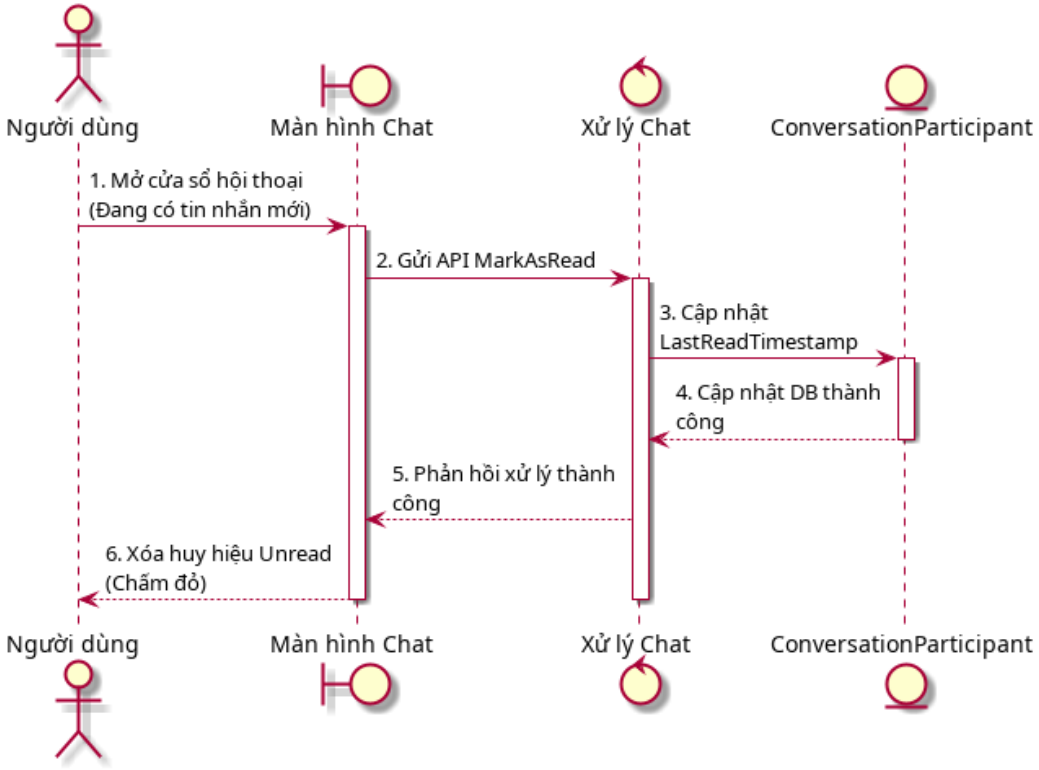
Hình 2.40. Sơ đồ Tuần tự: UC-33 Tạo cuộc hội thoại

34. UC-34: Gửi tin nhắn & File



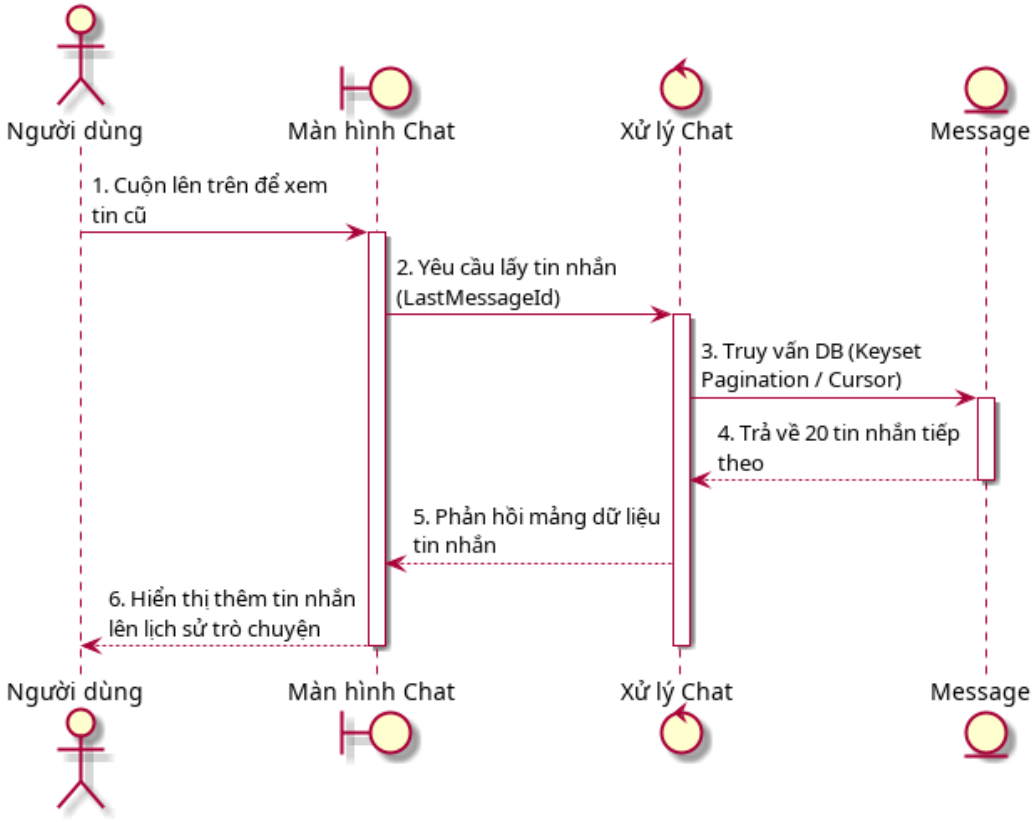
Hình 2.41. Sơ đồ Tuần tự: UC-34 Gửi tin nhắn & File

35. UC-35: Đọc tin nhắn (Mark As Read)



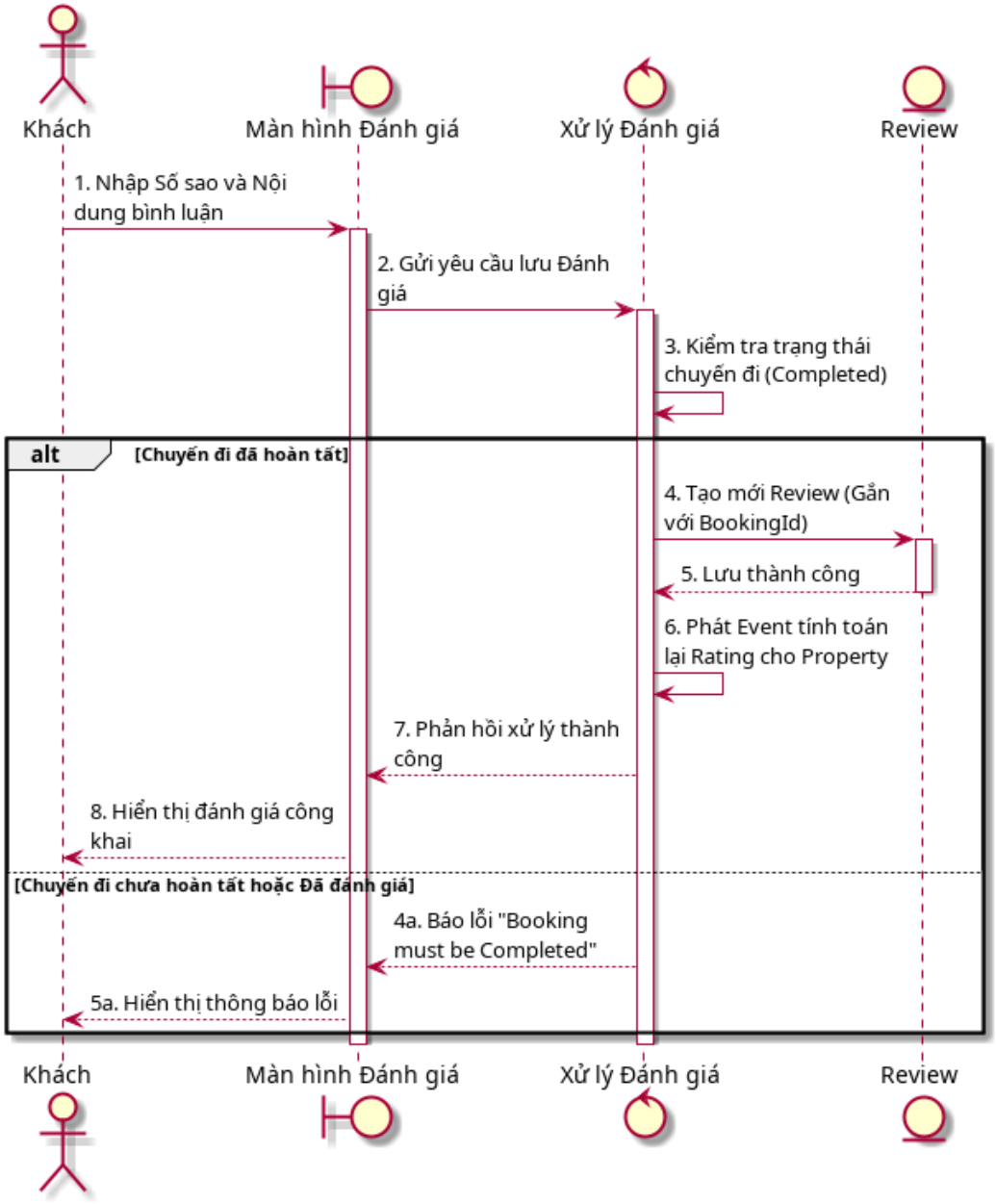
Hình 2.42. Sơ đồ Tuần tự: UC-35 Đọc tin nhắn

36. UC-36: Lấy Lịch sử Tin nhắn



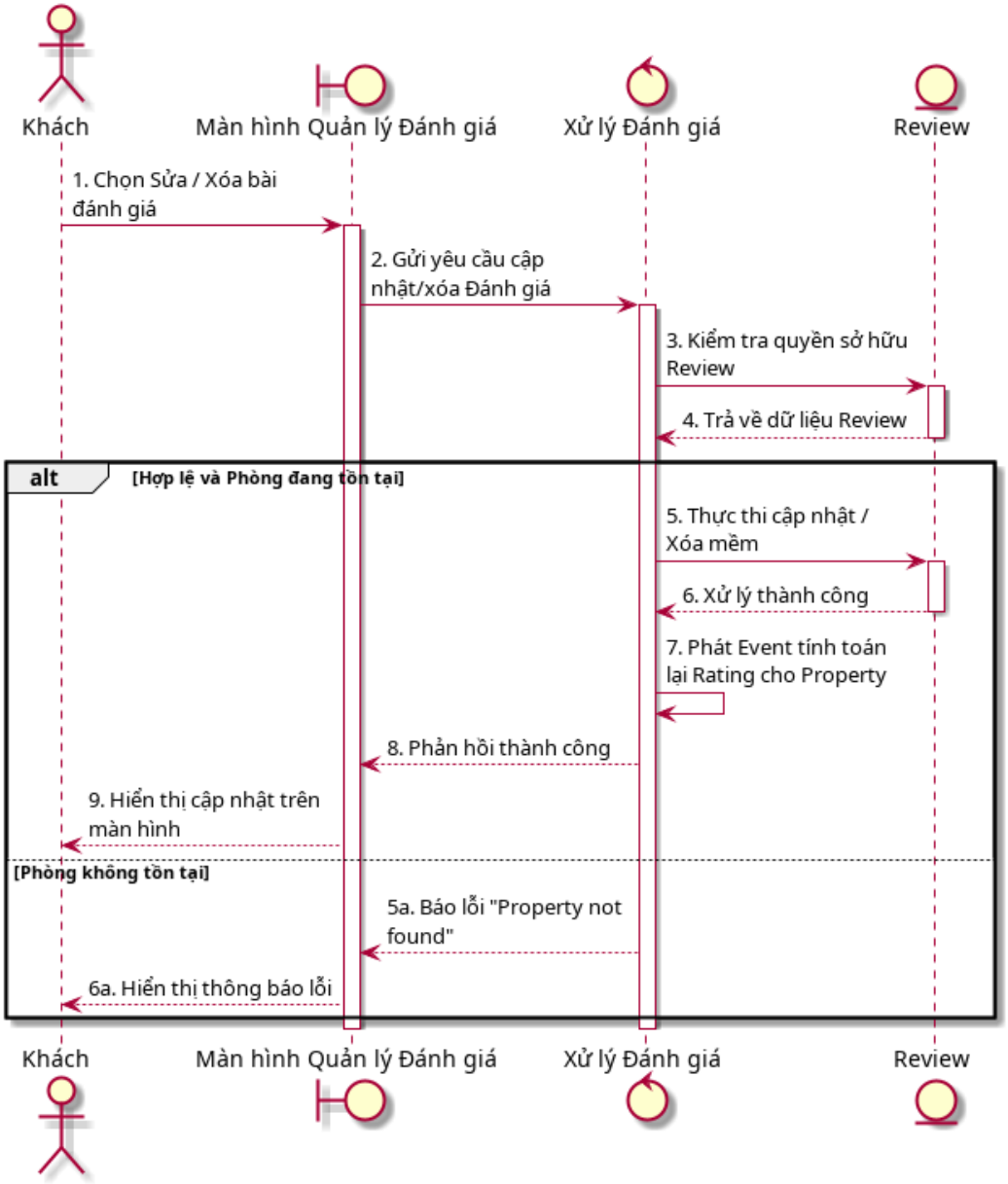
Hình 2.43. Sơ đồ Tuần tự: UC-36 Lấy Lịch sử Tin nhắn

37. UC-37: Viết Đánh giá (Review)



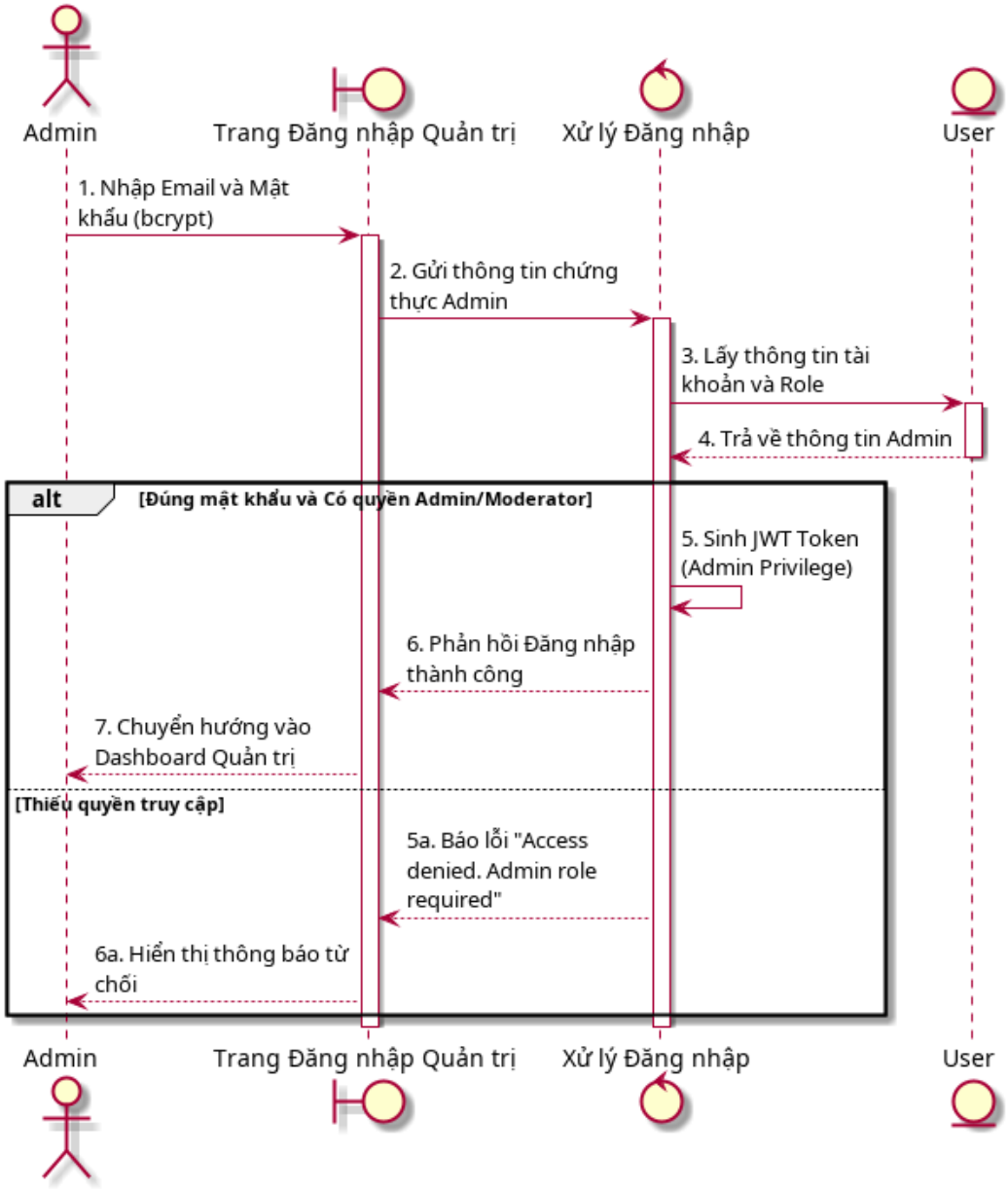
Hình 2.44. Sơ đồ Tuần tự: UC-37 Viết Đánh giá

38. UC-38: Sửa/Xóa Đánh giá



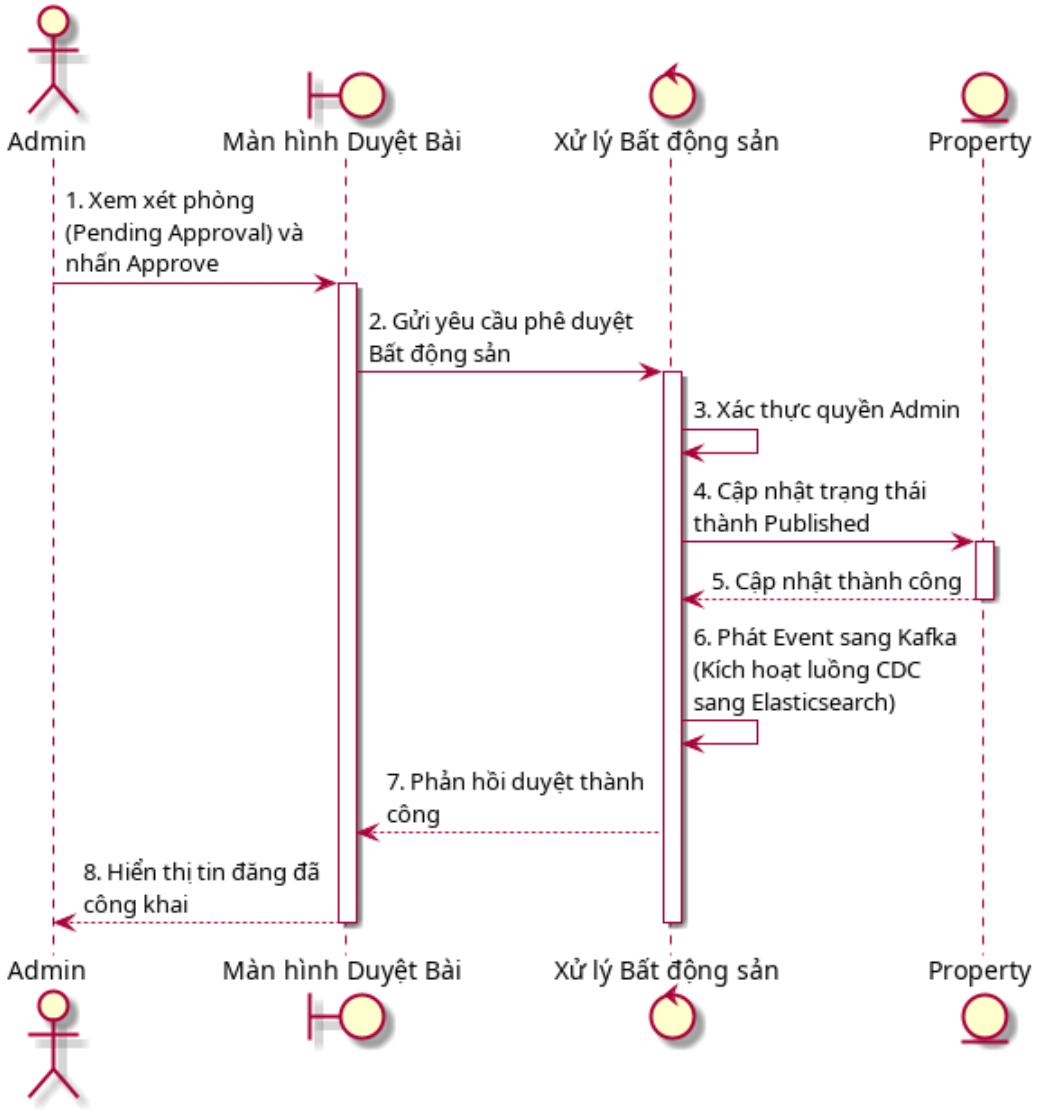
Hình 2.45. Sơ đồ Tuần tự: UC-38 Sửa/Xóa Đánh giá

39. UC-39: Đăng nhập Admin



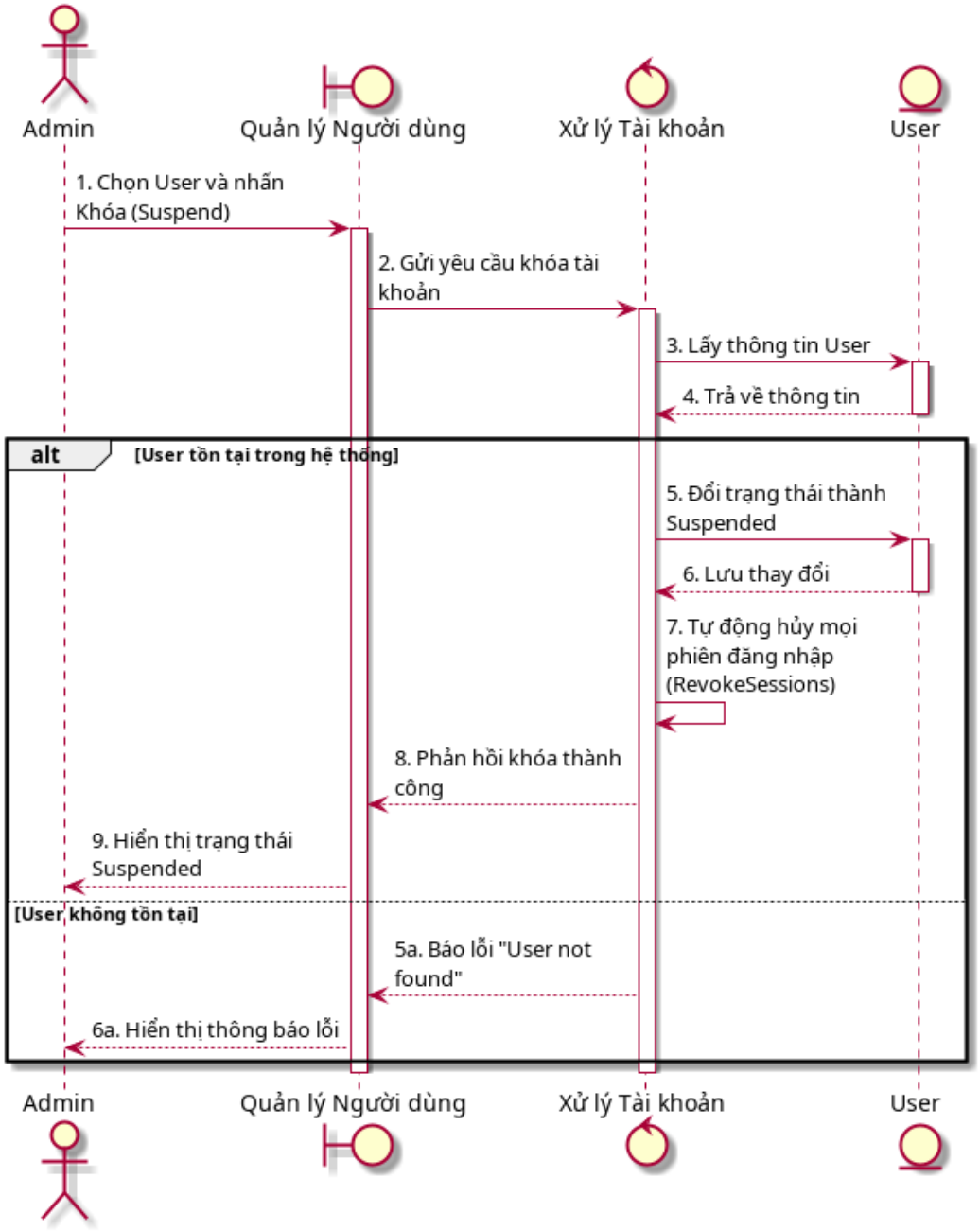
Hình 2.46. Sơ đồ Tuần tự: UC-39 Đăng nhập Admin

40. UC-40: Duyệt Bất động sản



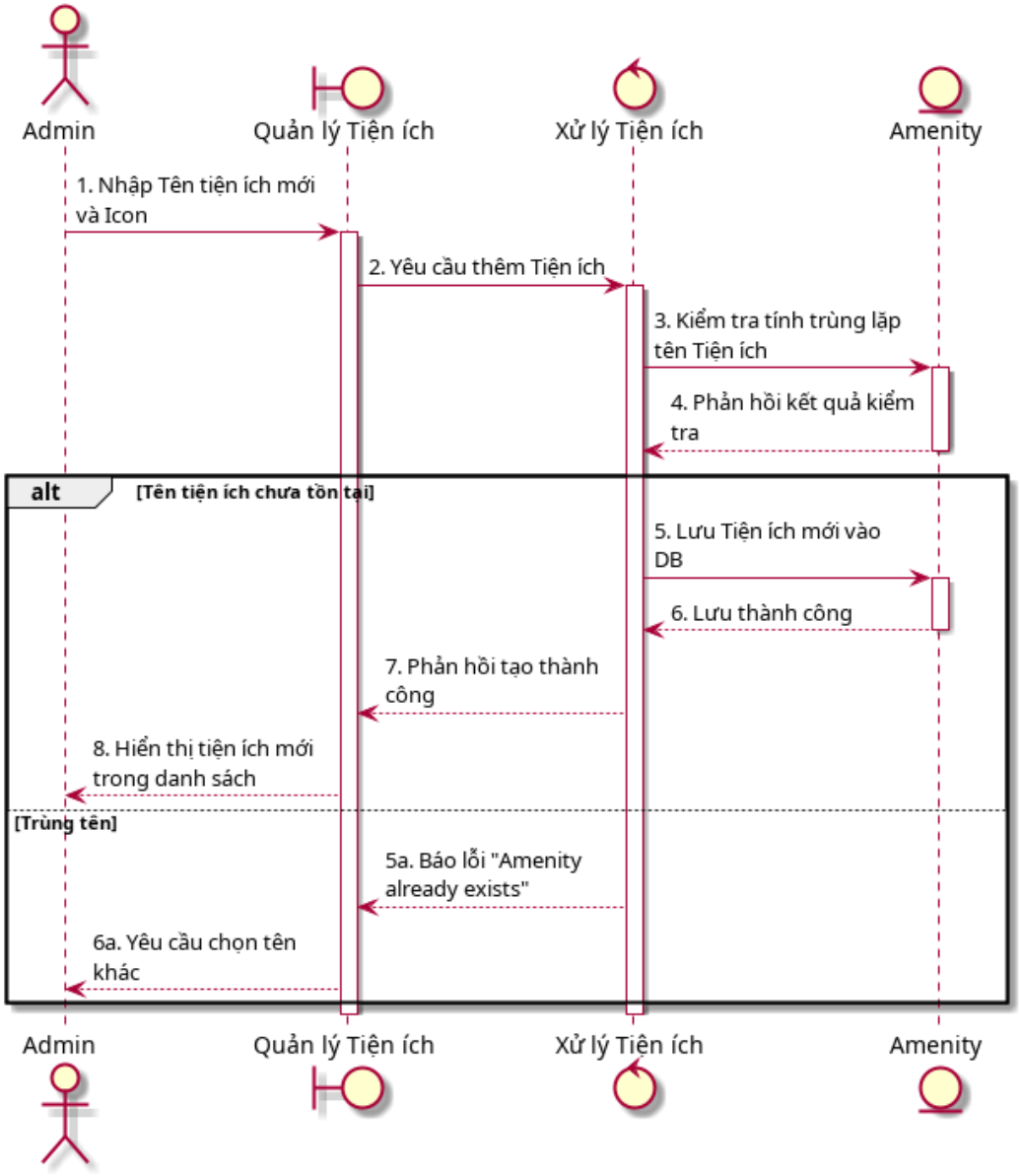
Hình 2.47. Sơ đồ Tuần tự: UC-40 Duyệt Bất động sản

41. UC-41: Khóa/Mở tài khoản



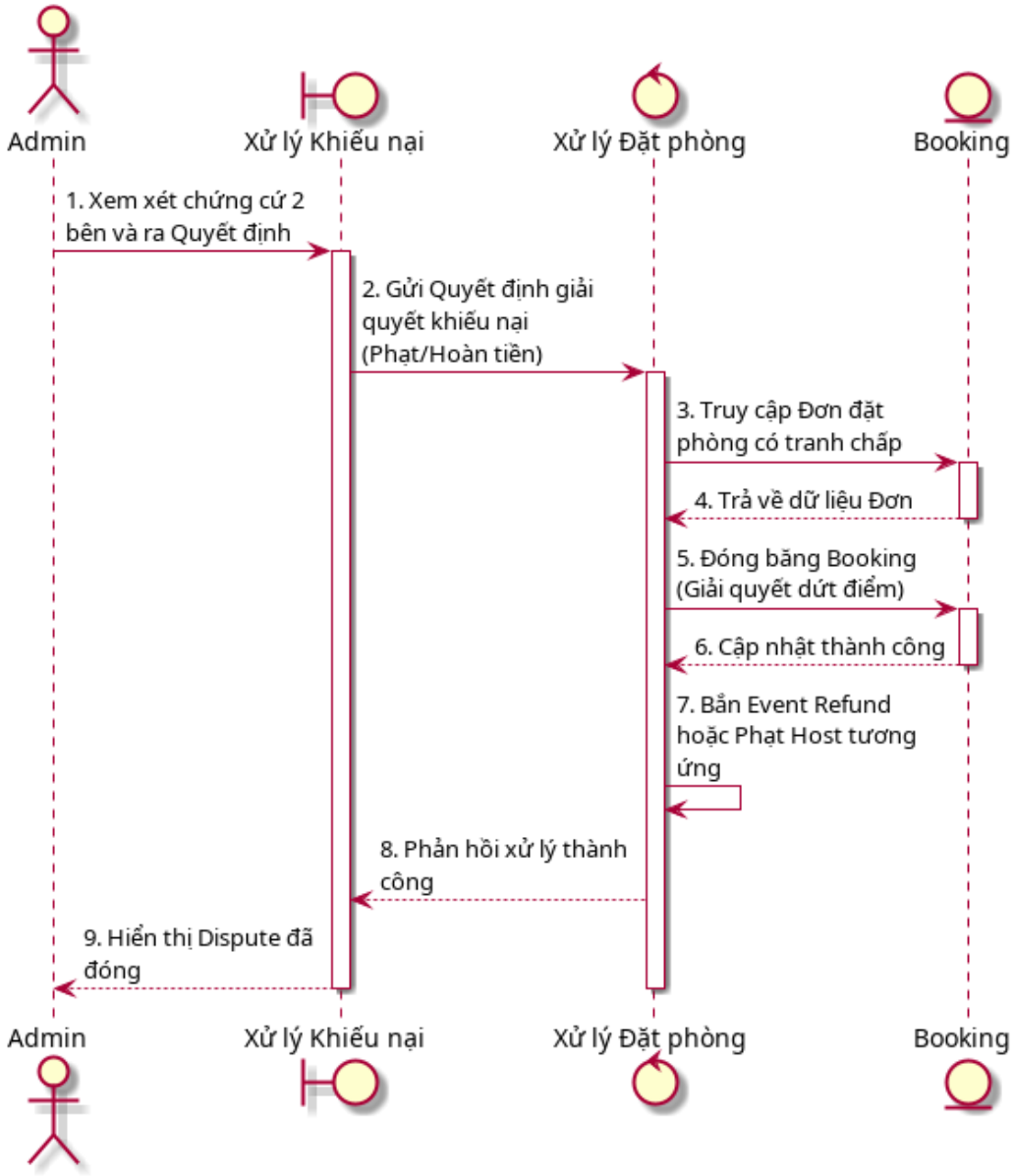
Hình 2.48. Sơ đồ Tuần tự: UC-41 Khóa/Mở tài khoản

42. UC-42: Quản lý Danh mục Tiện ích



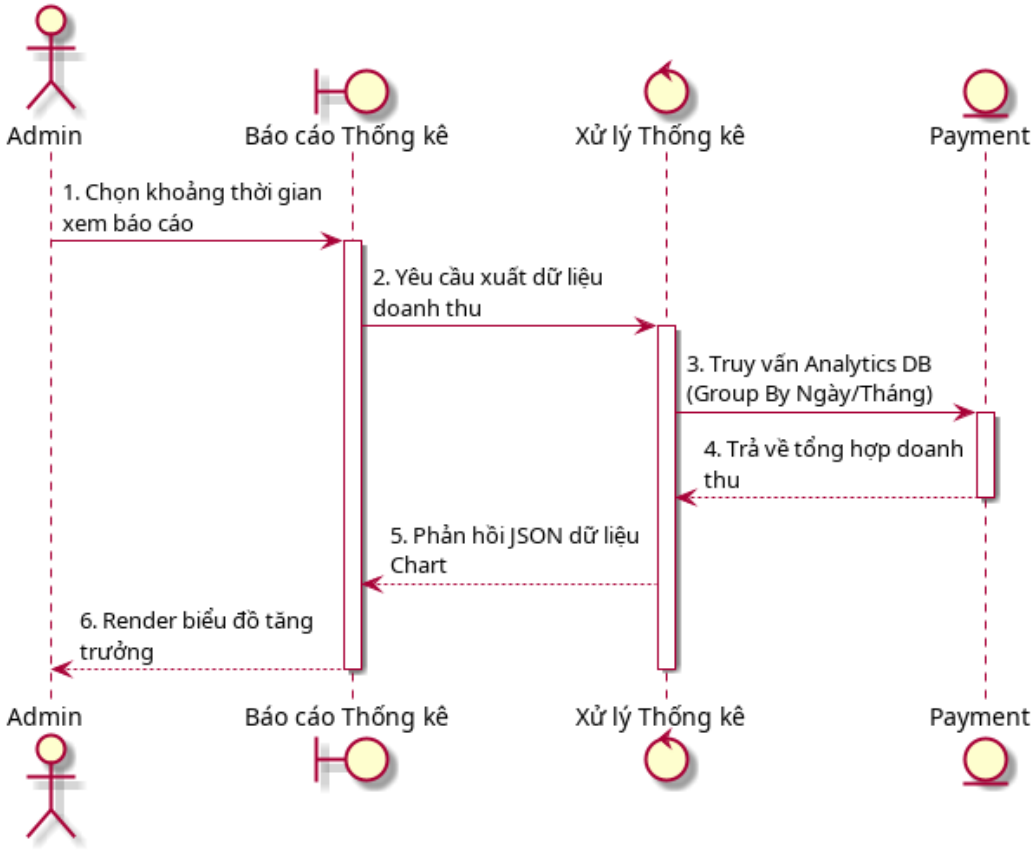
Hình 2.49. Sơ đồ Tuần tự: UC-42 Quản lý Danh mục Tiện ích

43. UC-43: Xử lý Khiếu nại (Disputes)



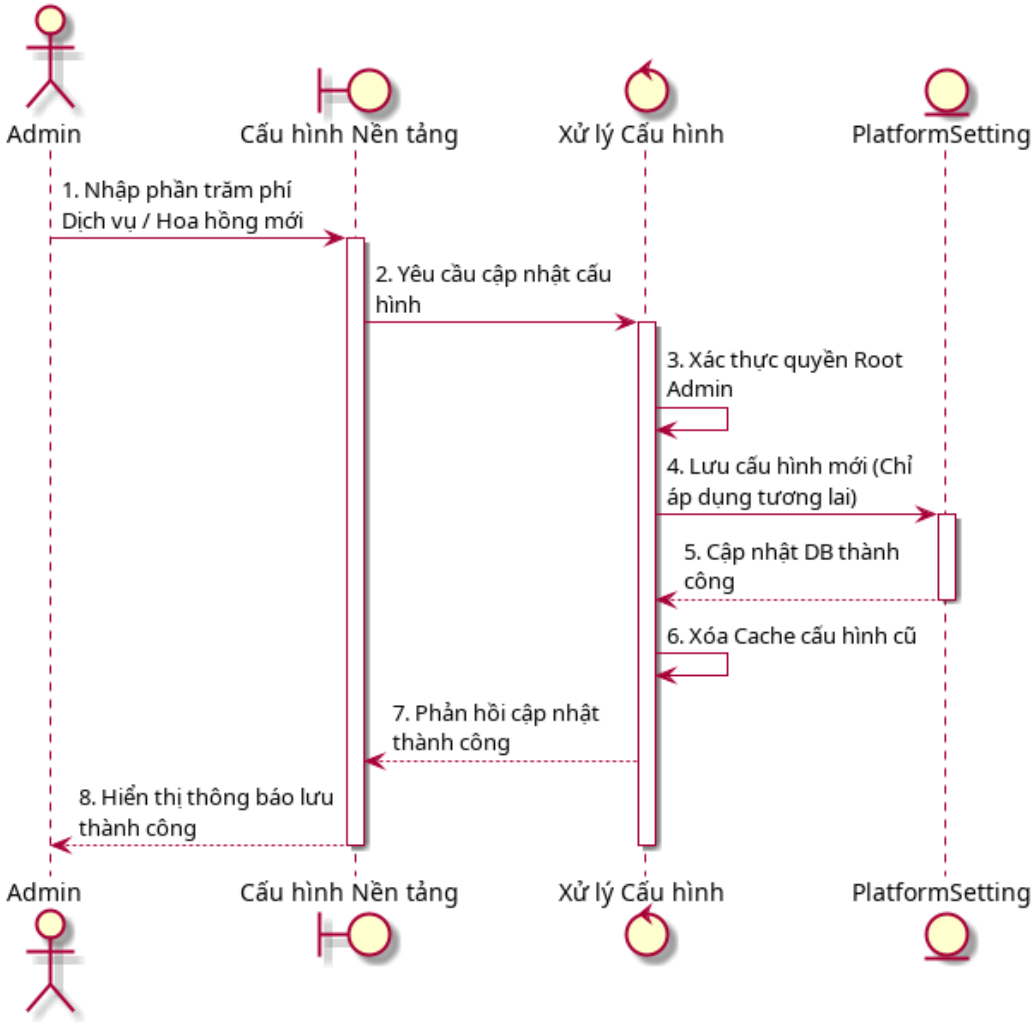
Hình 2.50. Sơ đồ Tuần tự: UC-43 Xử lý Khiếu nại

44. UC-44: Xem Báo cáo Thống kê



Hình 2.51. Sơ đồ Tuần tự: UC-44 Xem Báo cáo Thống kê

45. UC-45: Cập nhật Cấu hình Nền tảng



Hình 2.52. Sơ đồ Tuần tự: UC-45 Cập nhật Cấu hình Nền tảng

2.4 Kiến trúc tổng thể

2.4.1 Sơ đồ Kiến trúc Tổng thể

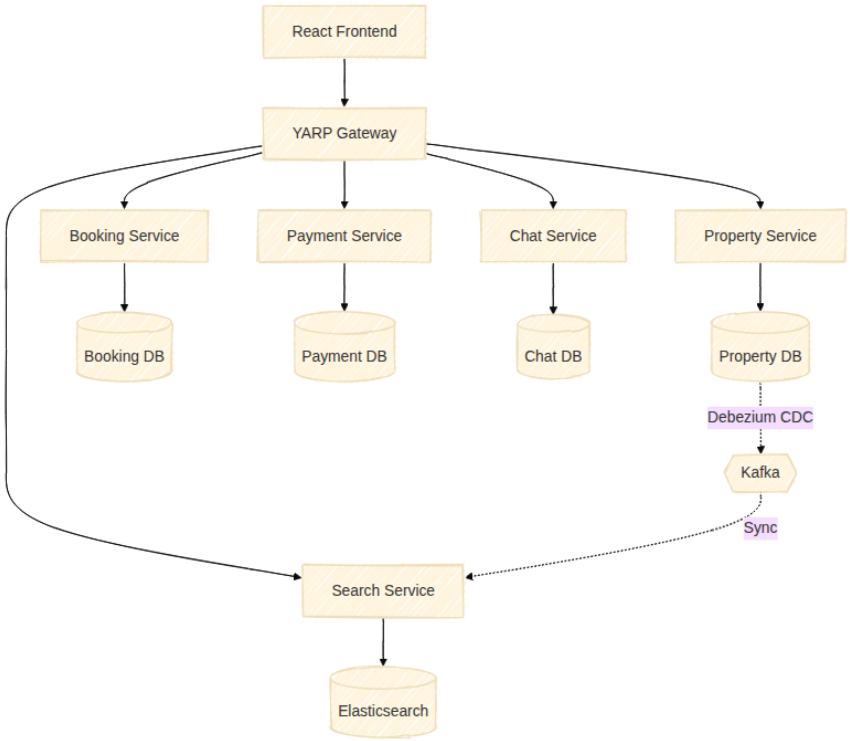
Kiến trúc của hệ thống AirVnV được thiết kế theo mô hình Microservices, kết hợp với Event-Driven Architecture (Kiến trúc Hướng sự kiện) để giải quyết các bài toán về khả năng mở rộng, tính sẵn sàng và tính nhất quán dữ liệu trong môi trường phân tán. Việc lựa chọn Microservices thay vì Monolith (Đơn khối) xuất phát từ yêu cầu thực tế của nền tảng:

- Hệ thống có nhiều nghiệp vụ độc lập (Đặt phòng, Thanh toán, Tìm kiếm) với đặc thù tải (load) khác nhau. Khác với Monolith buộc phải nhân bản toàn bộ ứng dụng, Microservices cho phép scale độc lập từng dịch vụ (ví dụ: cấp thêm tài nguyên cho SearchService vào mùa cao điểm) mà không gây lãng phí.

- Công nghệ lưu trữ đa dạng (Polyglot Persistence): Theo chiến lược Database-per-service, PropertyService và BookingService sử dụng PostgreSQL để đảm bảo tính toàn vẹn giao dịch (ACID), trong khi SearchService sử dụng Elasticsearch để tối ưu hóa truy vấn tọa độ địa lý.

Hệ thống được chia thành các thành phần cốt lõi như sau:

1. **Client (Frontend):** Ứng dụng Web được xây dựng bằng React (Next.js/Vite), giao tiếp với backend hoàn toàn thông qua REST API.
2. **API Gateway (YARP):** Là điểm chạm duy nhất (Single Entry Point) của hệ thống. Nó tiếp nhận request, xử lý xác thực tập trung và định tuyến đến các Microservices tương ứng.
3. **Các Microservices nghiệp vụ (Orchestrated bởi .NET Aspire):**
   - **UserService:** Quản lý thông tin định danh người dùng.
   - **PropertyService:** Quản lý vòng đời bất động sản, cấu hình giá, lịch trống phòng.
   - **SearchService:** Xử lý truy vấn tìm kiếm phức tạp (theo tọa độ, bộ lọc) không chạm vào database giao dịch, đảm bảo tốc độ phản hồi $O(1)$.
   - **BookingService:** Xử lý logic đặt phòng với state machine khắt khe, ngăn chặn double-booking.
   - **PaymentService:** Quản lý giao dịch thanh toán và webhook, phát sinh sự kiện khi thanh toán thành công.
   - **ChatService:** Quản lý hội thoại gắn kết với ngữ cảnh (property/booking), hỗ trợ gửi tin nhắn tự động từ hệ thống.
4. **Message Brokers & Data Streaming:**
   - **RabbitMQ:** Xử lý Domain/Integration Events nội bộ để các service giao tiếp bất đồng bộ một cách đáng tin cậy.
   - **Debezium & Apache Kafka:** Áp dụng Change Data Capture (CDC). Debezium bắt các thay đổi từ PostgreSQL, đẩy vào Kafka, và SearchService tiêu thụ để cập nhật Elasticsearch theo thời gian thực.



Hình 2.53. Sơ đồ Kiến trúc Tổng thể của hệ thống AirVnV

Sơ đồ trên (Hình 2.53) minh họa sự phân chia rõ ràng giữa luồng yêu cầu đồng bộ từ người dùng và luồng đồng bộ dữ liệu ngầm (background sync) qua Kafka và RabbitMQ. Thiết kế này giúp hệ thống đạt hiệu năng cao, giảm tải cho cơ sở dữ liệu chính và đảm bảo tính nhất quán cuối cùng.

2.4.2 Cơ chế giao tiếp giữa các dịch vụ (Inter-service Communication)

Để đảm bảo hệ thống Microservices hoạt động trơn tru mà không bị dính chặt vào nhau (tight-coupling) hay tạo ra hiện tượng tắc nghẽn (bottleneck), toàn bộ luồng giao tiếp được chia làm ba cơ chế thiết kế chuyên biệt:

- **Giao tiếp Đồng bộ (Synchronous - Client to Service):** Chỉ được cho phép ở khu vực ranh giới (Edge). Trình duyệt của người dùng gửi yêu cầu HTTP/REST tới YARP API Gateway. Gateway sẽ định tuyến đồng bộ yêu cầu này tới Service tương ứng và chờ lấy dữ liệu trả về cho người dùng ngay lập tức. Tuyệt đối **không** có lệnh gọi HTTP đồng bộ trực tiếp giữa các Microservices nội bộ với nhau để tránh lỗi dây chuyền (Cascading failures).
- **Giao tiếp Bất đồng bộ (Asynchronous - Event-Driven):** Đóng vai trò xương sống cho luồng xử lý nghiệp vụ chéo. Khi một Service thay đổi trạng thái dữ liệu (ví dụ:

`BookingService` tạo Booking mới), nó sẽ phát ra một Sự kiện Tích hợp (Integration Event) qua giao thức AMQP tới **RabbitMQ**. Các Service khác (như `PaymentService`) đóng vai trò Consumer sẽ âm thầm nhận sự kiện và xử lý tiếp. Cơ chế Pub/Sub này đảm bảo tính nhạy bén và khả năng mở rộng tối đa.

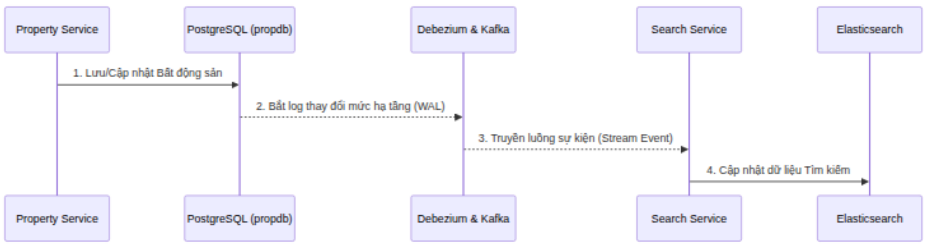
- **Đồng bộ Dữ liệu Cấp cơ sở (Data-Level CDC):** Dành riêng cho bài toán luân chuyển dữ liệu lớn. Hệ thống sử dụng **Debezium** đọc trực tiếp tập tin Write-Ahead Log (WAL) từ PostgreSQL ở tầng hạ tầng, sau đó biến chúng thành luồng sự kiện (Stream) trên **Kafka**. `SearchService` đóng vai trò Kafka Consumer để đồng bộ dữ liệu vào Elasticsearch theo thời gian thực (Real-time). Việc dùng Kafka thay vì RabbitMQ ở đây là bắt buộc do đặc thù cần xử lý luồng dữ liệu thông lượng cực cao (High throughput) và lưu trữ log (Log retention).

Bản đồ Giao tiếp Cụ thể Giữa Các Dịch vụ (Service Interaction Map)

Để làm rõ cơ chế trên, dưới đây là bản đồ giao tiếp chéo thực tế đang diễn ra trong hệ thống AirVnV, được tách biệt theo từng chức năng (Feature) độc lập nhằm minh họa rõ ràng dòng chảy dữ liệu.

1. Chức năng Đồng bộ Dữ liệu Tìm kiếm (Data-level CDC)

Luồng này phục vụ việc đồng bộ hóa dữ liệu từ cơ sở dữ liệu quan hệ sang bộ máy tìm kiếm tốc độ cao.



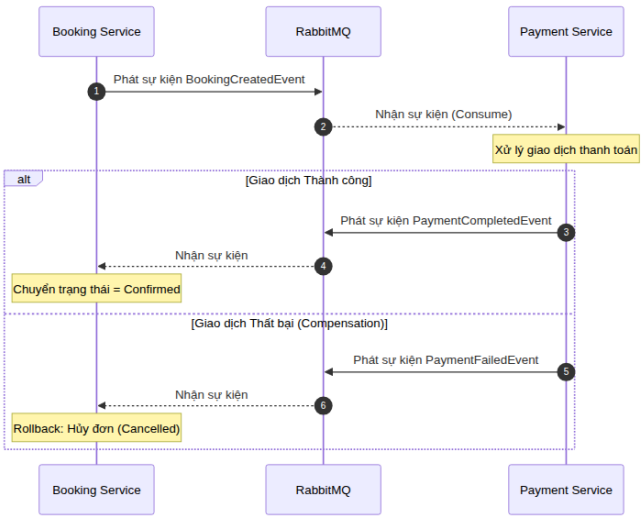
Hình 2.54. Sơ đồ Tuần tự minh họa giao tiếp Đồng bộ dữ liệu qua Kafka

Khi Chủ nhà tạo/sửa Bất động sản (`Property`), thay vì bắt `PropertyService` tự gọi API cập nhật, Debezium sẽ đọc trực tiếp từ lớp vật lý (WAL logs của PostgreSQL) và đẩy luồng sự kiện vào Kafka. `SearchService` hứng dữ liệu này và nạp vào Elasticsearch để tối ưu trải nghiệm tìm kiếm của Khách hàng.

2. Chức năng Giao dịch Đặt phòng và Thanh toán

Đại diện cho nhóm xử lý luồng giao dịch nghiệp vụ chéo phức tạp thông qua cơ chế Event-

Driven.



Hình 2.55. Sơ đồ Tuần tự minh họa giao tiếp Đặt phòng qua RabbitMQ

Khi Khách chốt phòng, `BookingService` khởi tạo đơn hàng ở trạng thái chờ (`Pending`) và phát sự kiện `BookingCreatedEvent` lên RabbitMQ. `PaymentService` hứng sự kiện để xử lý giao dịch. Tại đây, luồng phân tán (Saga) rẽ thành hai nhánh:

- **Thành công:** Phát sự kiện `PaymentCompletedEvent`. `BookingService` bắt sự kiện và chốt lịch đặt phòng (`Confirmed`).
- **Thất bại (Compensation):** Nếu thanh toán lỗi, bị từ chối hoặc hết hạn, hệ thống phát sự kiện `PaymentFailedEvent`. `BookingService` bắt sự kiện và thực thi bù trừ (Rollback) bằng cách hủy bỏ đơn hàng (`Cancelled`), giải phóng phòng trống cho khách khác.

Cơ chế này đảm bảo tuyệt đối tính nhất quán dữ liệu cuối cùng (Eventual Consistency) mà không làm tắc nghẽn hoặc sập nguyên chuỗi hệ thống như lệnh gọi đồng bộ truyền thống.

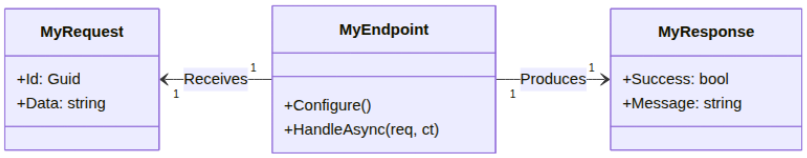
2.4.3 Áp dụng Mẫu thiết kế (Design Patterns / Architectural Patterns)

Hệ thống AirVnV là một dự án Microservices phức tạp, yêu cầu các tiêu chuẩn cao về tính bảo trì và khả năng mở rộng. Quá trình phát triển đã áp dụng linh hoạt cả các mẫu thiết kế kiến trúc (Architectural Patterns) để quản lý luồng dữ liệu hệ thống, lẫn các mẫu thiết kế hướng đối tượng (Object-Oriented Design Patterns) để giải quyết các bài toán logic cụ thể.

Các Mẫu Thiết kế Kiến trúc (Architectural Patterns)

1. REPR Pattern (Request-Endpoint-Response)

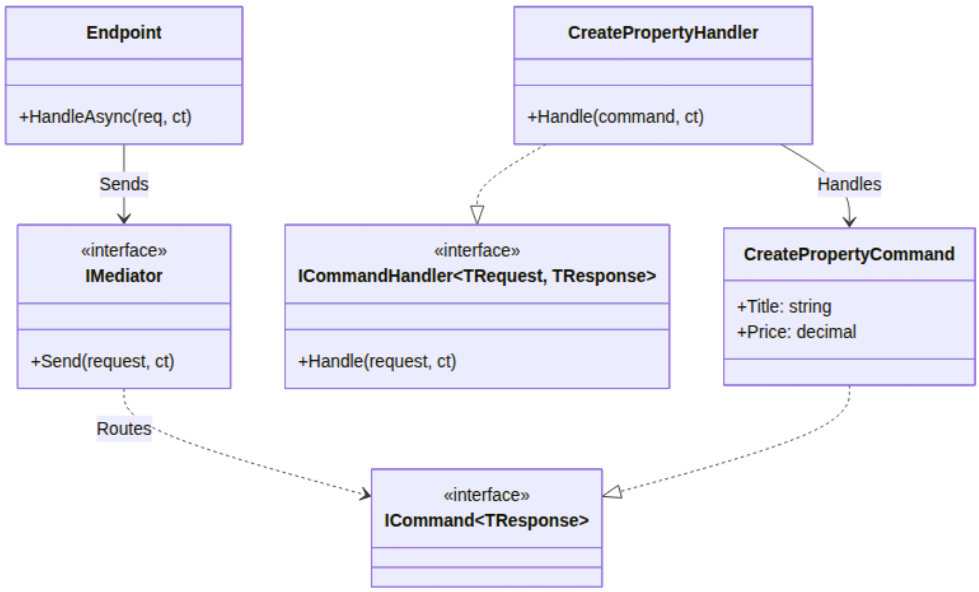
Dự án loại bỏ hoàn toàn mô hình `Controller` cồng kềnh của kiến trúc MVC truyền thống. Thay vào đó, hệ thống áp dụng **REPR Pattern** thông qua thư viện **FastEndpoints**. Một Class sẽ đại diện duy nhất cho một API Endpoint, đi kèm với các cấu trúc `Request` và `Response` của riêng nó. Endpoint đóng vai trò vô cùng mỏng (thin), chỉ thực hiện nhiệm vụ duy nhất là tiếp nhận HTTP Request và chuyển giao công việc xuống tầng xử lý trung gian.



Hình 2.56. Sơ đồ Lớp (Class Diagram) cấu trúc REPR Pattern

2. CQRS (Command Query Responsibility Segregation)

Luồng xử lý dữ liệu được phân tách hoàn toàn thành hai đường độc lập: Luồng Ghi (Command) và Luồng Đọc (Query), được điều phối bởi thư viện **MediatR**.



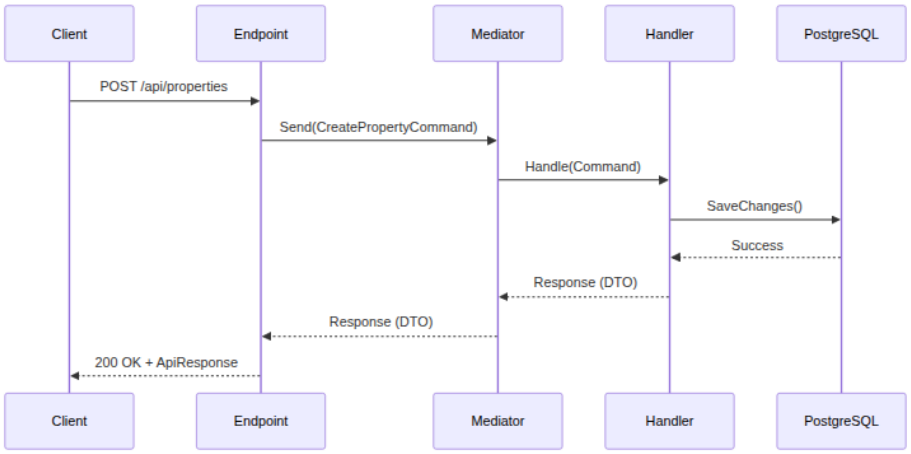
Hình 2.57. Sơ đồ Lớp (Class Diagram) cấu trúc CQRS sử dụng Mediator

Như Sơ đồ Lớp (Hình 2.57) thể hiện:

- **Luồng Ghi (Command):** Kế thừa từ giao diện `Mediator.ICommand`. Mọi thao tác thay đổi dữ liệu được cô lập trong `ICommandHandler`, đảm bảo không có logic bị rò rỉ ra ngoài Endpoint.

- **Luồng Đọc (Query):** Kế thừa từ `Mediator.IQuery`, sử dụng LINQ cơ bản kết hợp `AsNoTracking` của Entity Framework Core để trả về dữ liệu nhanh nhất mà không cần theo dõi sự thay đổi (change tracking).

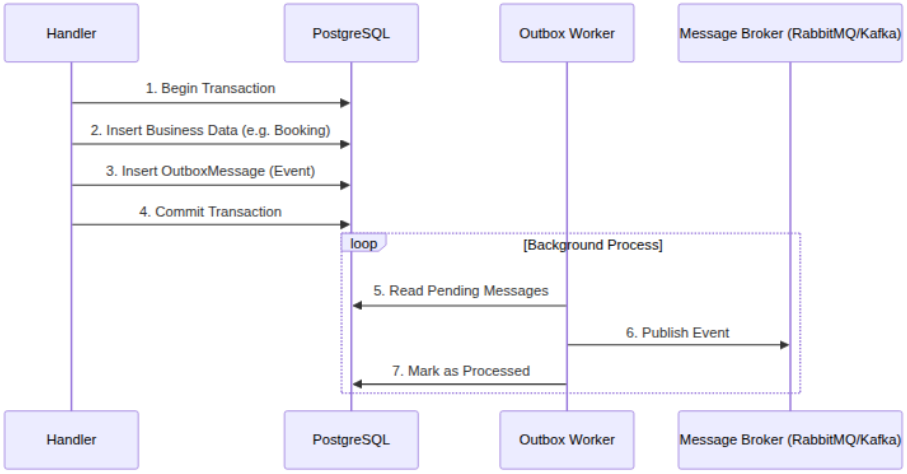
`Endpoint` đóng vai trò cực kỳ mỏng, chỉ nhận HTTP Request và gọi `IMediator.Send()`, tách biệt hoàn toàn trách nhiệm xử lý logic.



Hình 2.58. Sơ đồ Tuần tự (Sequence Diagram) minh họa luồng xử lý CQRS

3. Transactional Outbox Pattern & Idempotent Consumer

Trong hệ thống phân tán, các Microservices giao tiếp thông qua sự kiện (Domain Events). Tuy nhiên, rủi ro mất mạng khi đang lưu Database và đẩy Event lên Message Broker là rất lớn. AirVnV giải quyết bài toán này bằng **Transactional Outbox Pattern**.



Hình 2.59. Sơ đồ Tuần tự (Sequence Diagram) cơ chế Transactional Outbox

Theo Hình 2.59:

1. Lưu dữ liệu nghiệp vụ và thông tin sự kiện vào bảng `OutboxMessage` **trong cùng một Transaction của DB**.
2. Tiến trình ngầm (Background Worker) sử dụng MassTransit để quét bảng `OutboxMessage` và xuất bản sự kiện tới RabbitMQ (đảm bảo At-least-once Delivery).

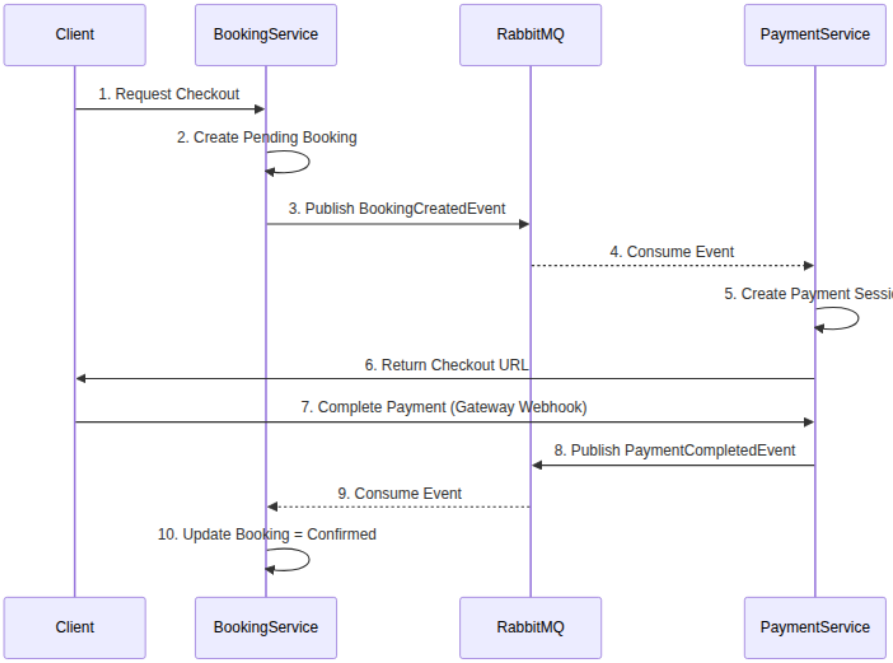
Để đối phó với việc sự kiện có thể được nhận lại nhiều lần, dịch vụ nhận áp dụng **Idempotent Consumer Pattern**: sử dụng bảng `ProcessedEvent` với ràng buộc duy nhất (`Unique Constraint`) tại tầng Database để chặn đứng xử lý trùng lặp và Race Condition (TOCTOU).

4. Vertical Slice Architecture (Kiến trúc theo chiều dọc)

Thay vì tổ chức mã nguồn theo các tầng ngang (Controllers, Services, Repositories), dự án cấu trúc theo **Vertical Slice Architecture**. Mỗi tính năng (Use Case) được gom lại thành một thư mục duy nhất chứa tất cả thành phần cần thiết: `Endpoint.cs`, `Request.cs`, `Response.cs`, `Handler.cs`, `Validator.cs`. Kiến trúc này tối đa hóa tính gắn kết (High Cohesion) và dễ dàng bảo trì.

5. Saga Pattern (Choreography)

Để xử lý các giao dịch phân tán kéo dài qua nhiều Microservices (tiêu biểu là luồng Đặt phòng - Thanh toán), hệ thống áp dụng **Saga Pattern** theo cơ chế điều phối không tập trung (Choreography) thông qua RabbitMQ. Thay vì dùng HTTP gọi đồng bộ dẫn tới kẹt cổ chai, các dịch vụ phản ứng lại dựa trên Sự kiện (Event).

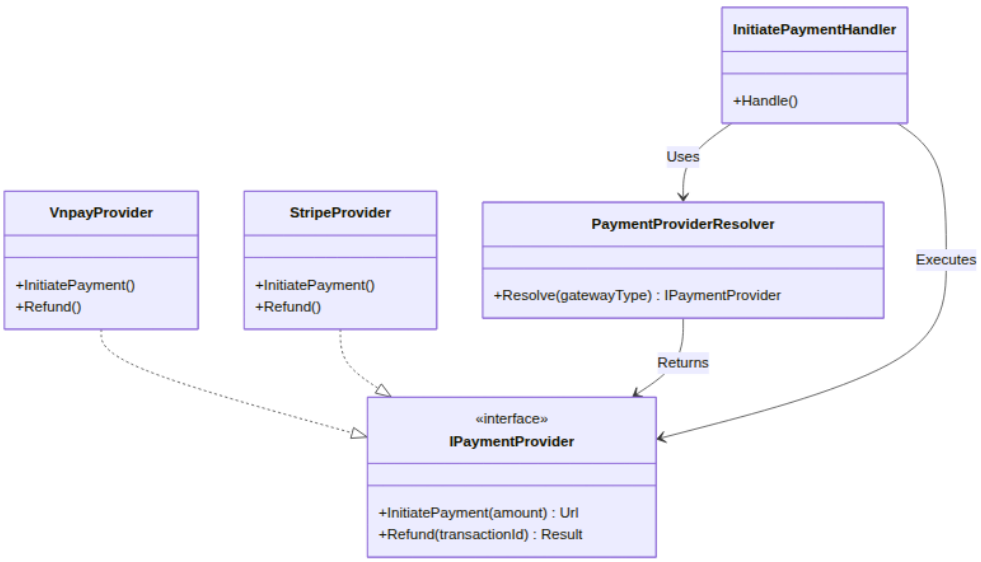


Hình 2.60. Sơ đồ Tuần tự (Sequence Diagram) minh họa Saga Pattern giữa Booking và Payment

Các Mẫu Thiết kế Hướng đối tượng (Object-Oriented Design Patterns)

1. Strategy Pattern & Provider Resolver

Pattern này được áp dụng triệt để ở `PaymentService` để lựa chọn linh hoạt cổng thanh toán (VNPay, Stripe) vào thời điểm runtime mà không làm thay đổi luồng xử lý chính.



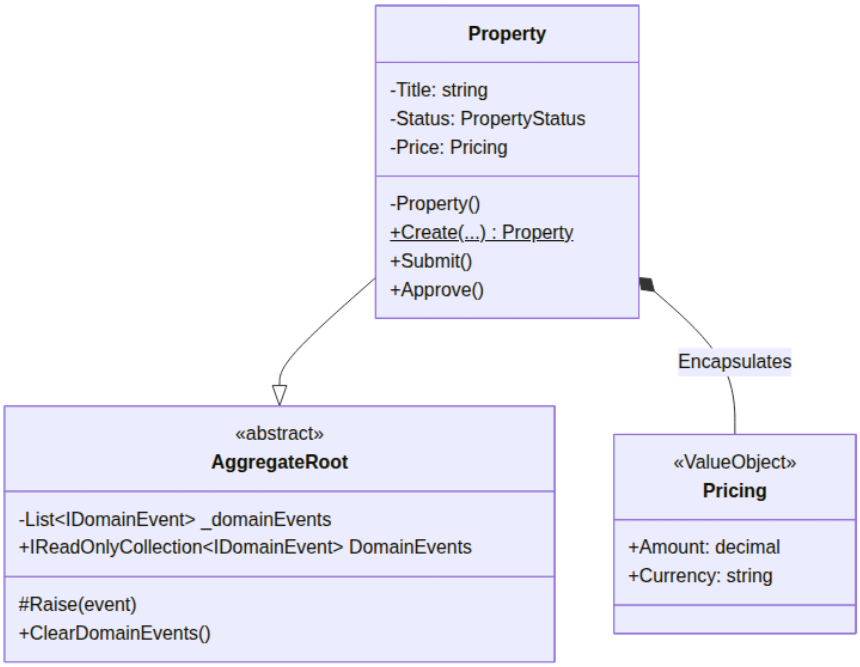
Hình 2.61. Sơ đồ Lớp (Class Diagram) mô hình Strategy Pattern tại PaymentService

Như Hình 2.61 thể hiện, `InitiatePaymentHandler` không gọi trực tiếp các lớp

thanh toán cụ thể mà chỉ phụ thuộc vào giao diện `IPaymentProvider`. Thành phần `PaymentProviderR` đóng vai trò Factory, dựa vào dữ liệu cấu hình hệ thống để khởi tạo đúng loại cổng thanh toán (Strategy) tương ứng với từng giao dịch.

2. Domain-Driven Design (DDD) & Aggregate Root Pattern

Hệ thống loại trừ hoàn toàn Anemic Model (mô hình dữ liệu chỉ có get/set) để chuyển sang **Rich Domain Model**.

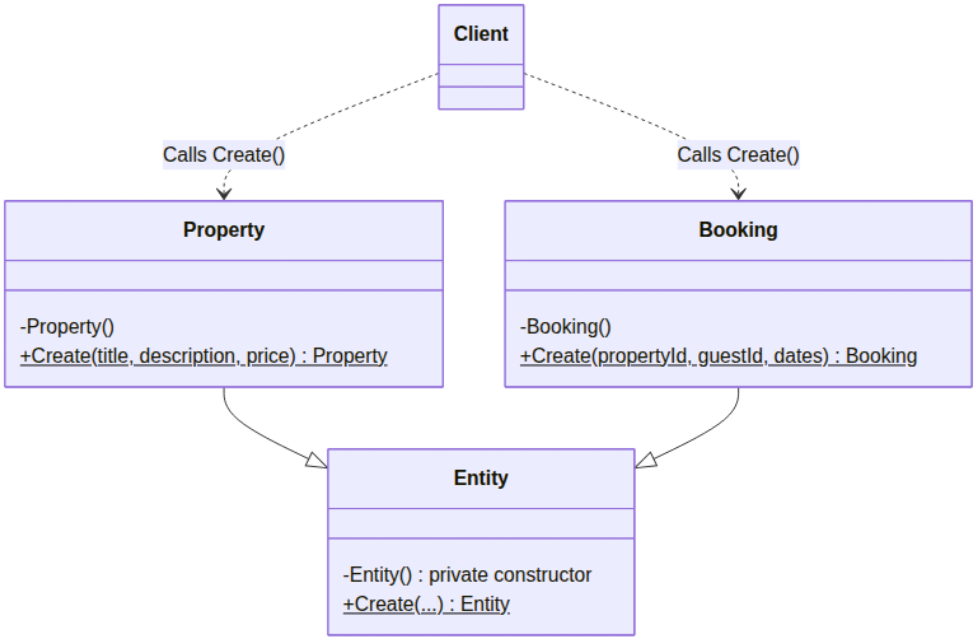


Hình 2.62. Sơ đồ Lớp (Class Diagram) mô tả Aggregate Root trong DDD

Các thay đổi trạng thái (ví dụ: `Submit()`, `Approve()` bất động sản) được thực thi trực tiếp bên trong các hàm của **Aggregate Root**, đảm bảo luật lệ nghiệp vụ (Invariants) được duy trì hoàn hảo mà không rò rỉ ra ngoài.

3. Factory Method Pattern

Thay thế các constructor public bằng hàm `Create()` tĩnh (static) nhằm thực hiện toàn bộ quá trình xác thực dữ liệu (validation) trước khi khởi tạo Đối tượng Thực thể (Entity Object).



Hình 2.63. Sơ đồ Lớp (Class Diagram) của Factory Method Pattern

Điều này đảm bảo tuyệt đối không có đối tượng rác hay lỗi (invalid state) tồn tại trong bộ nhớ hoặc bị lưu vào cơ sở dữ liệu.

4. Shared Kernel Pattern

Trích xuất các khối kiến trúc cốt lõi (như `IDomainEvent`, `AggregateRoot`, `ApiResponse<T>`) thành một thư viện độc lập (`Airbnb.SharedKernel`). Các Microservices tham chiếu đến thư viện này, giúp tái sử dụng mã nguồn mà không vi phạm nguyên tắc độc lập của Microservices.

Các Mẫu Thiết kế Giao diện Phía Khách (Frontend UI Patterns)

Bên cạnh việc áp dụng các thiết kế chuẩn mực ở tầng máy chủ (Backend), dự án cũng tiến hành chuẩn hóa kiến trúc mã nguồn ở giao diện người dùng (Frontend - React) nhằm đảm bảo tính mở rộng, khả năng bảo trì và kiểm soát độ phức tạp khi quy mô hệ thống gia tăng.

1. Kiến trúc Module hóa (Modular / Feature-Based Architecture)

Thay vì chia cấu trúc thư mục theo loại tệp tin (ví dụ: gộp chung tất cả components, hooks, api), mã nguồn Frontend được tổ chức chặt chẽ theo từng tính năng nghiệp vụ (Feature-Sliced). Mỗi tính năng (ví dụ: `features/auth/`) sở hữu trọn vẹn lớp API, kiểu dữ liệu, logic xử lý và giao diện của riêng nó, đảm bảo tuyệt đối nguyên lý Đóng gói (Encapsulation).

2. Adapter Pattern (Chuyển đổi Dữ liệu)
Giao diện (UI) được cách ly hoàn toàn khỏi cấu trúc dữ liệu trả về từ API (Data Transfer Object - DTO). **Adapter Pattern** được áp dụng thông qua các hàm thuần túy (Pure Functions) ở tầng trung gian để ánh xạ (map) DTO thành các `Model` chuẩn mực của UI. Điều này giúp Frontend không bị phá vỡ (break) khi cấu trúc cơ sở dữ liệu ở Backend thay đổi.

3. Tách biệt Mối quan tâm (Separation of Concerns) & Observer Pattern
Logic gọi API và quản lý trạng thái được tách rời hoàn toàn khỏi các UI Components thông qua cơ chế Custom Hooks. Thêm vào đó, hệ thống ứng dụng thư viện TanStack Query hoạt động dựa trên **Observer Pattern** để tự động theo dõi (subscribe) và đồng bộ hóa trạng thái dữ liệu từ máy chủ (Server State) xuống giao diện theo thời gian thực mà không làm nghẽn bộ nhớ cục bộ.

4. Flux Architecture (Kiến trúc Luồng dữ liệu Một chiều)
Đối với các trạng thái ứng dụng xuyên suốt (Client State) như cấu hình giao diện (Theme) hay phiên đăng nhập (`AuthSession`), hệ thống loại bỏ sự phức tạp của việc truyền dữ liệu hai chiều (Two-way binding) để áp dụng **Flux Architecture**. Thông qua thư viện Zustand, mọi luồng cập nhật dữ liệu đều diễn ra theo một chiều duy nhất (One-way data flow), triệt tiêu hoàn toàn rủi ro xung đột trạng thái (Race conditions trên UI).

5. Interceptor Pattern (Mẫu thiết kế Đánh chặn)
Mô hình **Interceptor Pattern** được thiết lập tại tầng cấu hình mạng (Axios). Mọi HTTP Request gửi đi đều bị chặn lại để tự động đính kèm `Bearer JWT Token`. Tương tự ở chiều về, hệ thống tự động bắt lỗi `401 Unauthorized` để kích hoạt tiến trình làm mới mã thông báo (Refresh Token) ngầm định, giúp giảm tải hoàn toàn độ phức tạp cho tầng giao diện người dùng.

Các Mẫu Cắt ngang và Đảm bảo An toàn (Cross-Cutting & Safety Patterns)

1. Unified Response Envelope
Mọi API phản hồi thành công hoặc thất bại đều được bọc trong một vỏ bọc duy nhất (Envelope) tên là `ApiResponse<T>`, bao gồm `Data`, `Success (bool)`, `ErrorCode`, và `Timestamp`. Client Frontend chỉ cần phân tích cú pháp (parse) một cấu trúc duy nhất cho toàn bộ hệ thống.

2. Exhaustiveness Check (CI Guard Pattern)
Đảm bảo chất lượng mã nguồn bằng Unit Test tự động. Mọi sự kiện (Domain Event) mới được

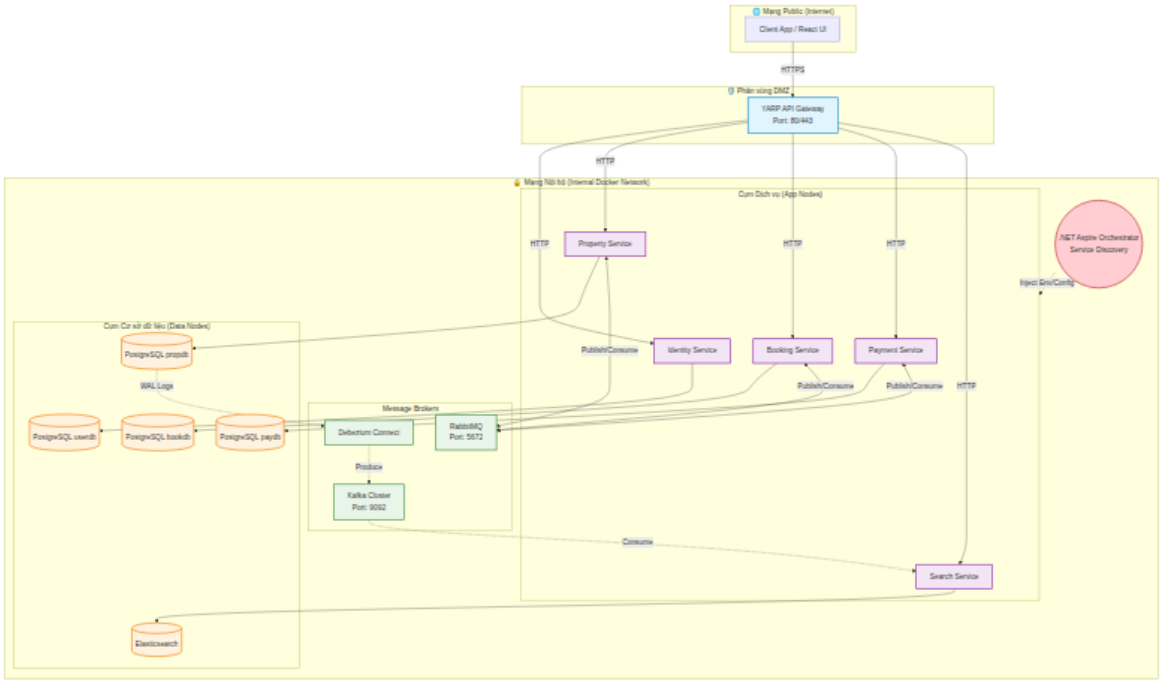
lập trình viên tạo ra bắt buộc phải được khai báo trong bản đồ điều hướng (Topic Map). Nếu quên khai báo, Test sẽ báo lỗi (Fail) ngay lập tức trên máy chủ CI/CD, ngăn chặn lỗi phát sinh khi triển khai lên Production.

3. Idempotency Guard (DB-Level Filtered Index)

Để ngăn chặn các rủi ro kẹp tiến trình (Race Condition) khi người dùng thao tác quá nhanh, hệ thống thiết lập `Unique Filtered Index` trực tiếp tại tầng CSDL. Việc này đảm bảo bằng toán học rằng một yêu cầu chỉ có thể sinh ra duy nhất một phiên xử lý, loại bỏ hoàn toàn hiện tượng tạo trùng dữ liệu (Double-booking).

2.5 Thiết kế cấu hình các node và mạng

Hệ thống AirVnV được thiết kế để triển khai hoàn toàn trên môi trường Container (Docker) nhằm tối đa hóa tính di động và dễ dàng mở rộng. Cấu trúc liên kết mạng và cấu hình các Node vật lý/logic được phân định nghiêm ngặt để đảm bảo an toàn thông tin theo chuẩn công nghiệp.



Hình 2.64. Sơ đồ Triển khai (Deployment Diagram) các Node và Phân vùng mạng

2.5.1 Cấu trúc Phân vùng Mạng (Network Segmentation)

Hệ thống mạng được thiết lập thành 3 ranh giới bảo mật rõ rệt (Security Boundaries):

- **Mạng Public (Internet):** Nơi các thiết bị của người dùng hoạt động, giao tiếp với hệ thống thông qua HTTPS.
- **Phân vùng DMZ (Demilitarized Zone):** Nơi trú ngụ duy nhất của hệ thống **YARP API Gateway**. Gateway là điểm tiếp xúc duy nhất (Single Entry Point) của toàn hệ thống với Internet, thực hiện mở cổng mạng (expose port) ra bên ngoài.
- **Mạng Nội bộ (Internal Docker Network):** Khu vực mạng biệt lập (Isolated Network) chứa toàn bộ Cụm Microservices (App Nodes), Cụm Database (Data Nodes) và các Message Brokers. Tất cả các Node trong khu vực này tuyệt đối **không** được ánh xạ (map port) ra Internet. Chúng giao tiếp với nhau bằng HTTP nội bộ thông qua cơ chế phân giải tên miền ảo (DNS) của Docker.

2.5.2 Cấu hình Điều phối bằng .NET Aspire

Dự án sử dụng **.NET Aspire** làm nền tảng điều phối (Orchestrator) toàn diện:

- **Service Discovery:** Aspire đóng vai trò là kiến trúc sư tổng công trình. Nó tự động thiết lập và tiêm (inject) các biến môi trường, chuỗi kết nối (Connection Strings) và URI của các dịch vụ lân cận vào môi trường của từng Node ngay tại thời điểm khởi chạy.
- **Health Checks & Cây phụ thuộc:** Aspire quản lý đồ thị phụ thuộc (Dependency Graph) cực kỳ phức tạp. Ví dụ: Nó đảm bảo cụm PostgreSQL và Kafka phải ở trạng thái "Sẵn sàng" (`/health`) trước khi khởi chạy `SearchService` hay `PropertyService`, ngăn chặn tình trạng tràn lỗi dây chuyền.
- **Cô lập CSDL (Database Isolation):** Dù có chung một máy chủ PostgreSQL nhằm tối ưu chi phí, hệ thống vẫn cấu hình phân quyền ở mức tài khoản (Role-based): mỗi Microservice chỉ truy cập được duy nhất Logical Database riêng biệt của nó (ví dụ: `BookingService` chỉ thấy `bookdb`), đáp ứng đúng tiêu chuẩn *Database-per-Microservice*.

CHƯƠNG 3
TRIỂN KHAI HỆ THỐNG

3.1 Môi trường và công nghệ triển khai

3.1.1 Ngôn ngữ và Framework

Bảng 3.1 tổng hợp các ngôn ngữ, framework và thư viện chính được sử dụng trong đồ án, phân chia theo tầng kiến trúc.

Bảng 3.1. Ngôn ngữ, Framework và thư viện chính

Tầng	**Công nghệ**
Backend	.NET / C# FastEndpoints + FastEndpoints.Security + FastEndpoints.Swagger Mediator (CQRS) Entity Framework Core + Npgsql
Frontend (Web)	React Vite TypeScript TanStack Query / Zustand
Admin UI	Next.js React TanStack Query + Table

3.1.2 Cơ sở hạ tầng và Middleware

Toàn bộ hạ tầng được triển khai dưới dạng container Docker, được điều phối tự động bởi .NET Aspire AppHost. Bảng 3.2 liệt kê chi tiết phiên bản từng thành phần.

Bảng 3.2. Phiên bản cơ sở hạ tầng container

Thành phần	**Image Docker**	**Phiên bản**
API Gateway	YARP.ReverseProxy (NuGet)	2.3.0
PostgreSQL	docker.io/library/postgres	17.6
Apache Kafka	docker.io/confluentinc/confluent-local	8.1.1
RabbitMQ	docker.io/library/rabbitmq	4.2-management
Elasticsearch	docker.io/library/elasticsearch	8.17.3
Redis	docker.io/library/redis	8.6
Debezium Connect	docker.io/debezium/connect	2.5
.NET Aspire SDK	Aspire.AppHost.Sdk	13.3.4

3.1.3 Môi trường phát triển và triển khai

Bảng 3.3 trình bày chi tiết môi trường hệ điều hành, công cụ phát triển và các dịch vụ hỗ trợ.

Bảng 3.3. Môi trường phát triển và công cụ

Hạng mục	**Chi tiết**
Hệ điều hành	Windows 11 / Ubuntu Server 24.04 LTS
Container Runtime	Docker Engine 27.x + Docker Compose
.NET SDK	10.0 (target `net10.0`)
Node.js	22.x LTS
IDE	JetBrains Rider 2025.x / Visual Studio Code
API Testing	Scalar (Swagger UI tích hợp FastEndpoints)
Version Control	Git + GitHub
CI/CD	GitHub Actions (`.github/workflows/ci.yml`)

3.1.4 Cấu hình điều phối .NET Aspire

.NET Aspire đóng vai trò điều phối toàn bộ cụm container và microservices. Cấu hình chính trong `AppHost.cs` bao gồm:

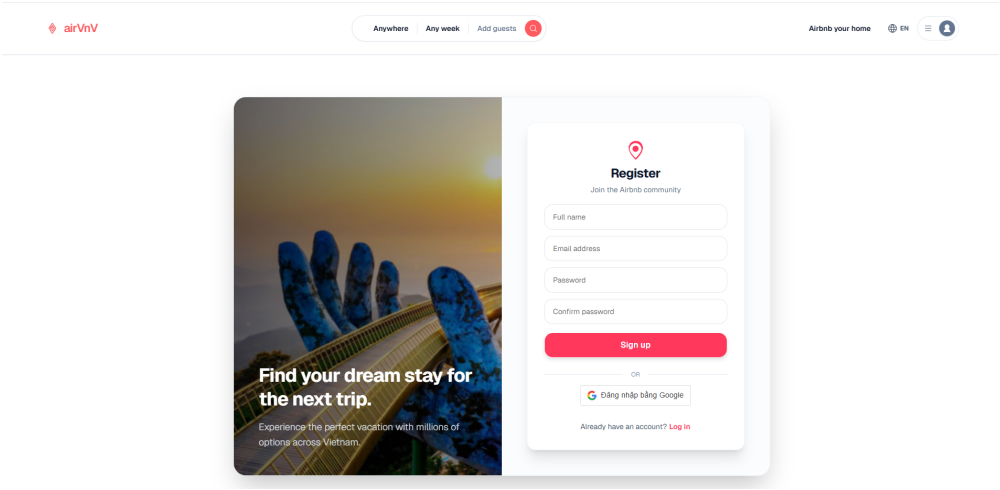
- **PostgreSQL:** Một server duy nhất với WAL level = `logical` (phục vụ Debezium CDC), mở cổng 5435→5432. Tạo 5 logical database riêng biệt theo nguyên tắc Database-per-Service: `userdb`, `propdb`, `bookdb`, `paydb`, `chatdb`.
- **Kafka:** Sử dụng Confluent image, mở cổng 29092→9092, cấu hình persistent volume cho log retention, tích hợp Kafka UI.
- **RabbitMQ:** Image `management`, mở cổng 5672 (AMQP) và 15672 (Management UI), cấu hình persistent volume.
- **Elasticsearch:** Mở cổng 9200, cấu hình CORS, kèm container Elasticvue để quản trị.
- **Redis:** Mở cổng 6379, dùng làm output cache cho SearchService và backplane SignalR cho ChatService.
- **Debezium Connect:** Image `debezium/connect:2.5`, liên kết với Kafka và PostgreSQL, có `WaitFor` đảm bảo Kafka sẵn sàng trước khi khởi chạy.
- **Dependency Graph:** Aspire quản lý thứ tự khởi chạy qua `WaitFor()`: Database → Message Broker → Microservice → Gateway.

3.2 Kết quả triển khai các chức năng chính

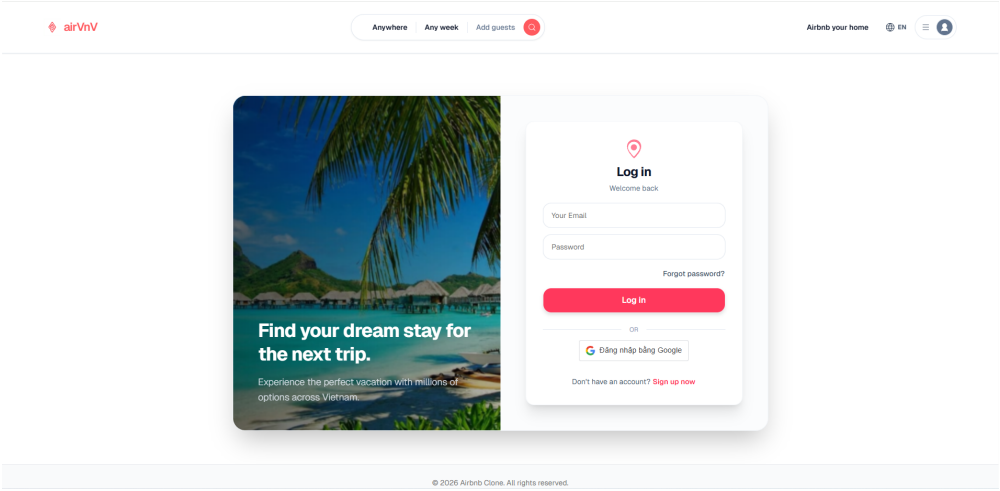
3.2.1 Xác thực và Quản lý tài khoản (UC-01 – UC-05)

Hệ thống xác thực được triển khai tại `UserService`, hỗ trợ hai cơ chế đăng nhập: mật khẩu truyền thống và Google SSO. Mật khẩu được mã hóa bằng BCrypt trước khi lưu trữ. Kết quả xác thực trả về JWT Bearer token, được gateway validate trên mọi request bảo vệ.

- **Đăng ký (UC-01):** Người dùng nhập thông tin cơ bản, hệ thống kiểm tra trùng email và gửi OTP qua Firebase. Tài khoản khởi tạo ở trạng thái `Inactive`.
- **Xác thực Email (UC-02):** Nhập OTP, hệ thống xác thực và chuyển tài khoản sang `Active`.
- **Đăng nhập (UC-03):** Hỗ trợ email/password và Google OAuth2. Trả về JWT Access Token + Refresh Token.
- **Cập nhật hồ sơ (UC-04):** Thay đổi avatar (Cloudinary), tên, số điện thoại.
- **Đổi mật khẩu (UC-05):** Xác thực mật khẩu cũ, cập nhật mật khẩu mới, thu hồi toàn bộ phiên đăng nhập khác.



Hình 3.1. Giao diện Đăng ký tài khoản



Hình 3.2. Giao diện Đăng nhập

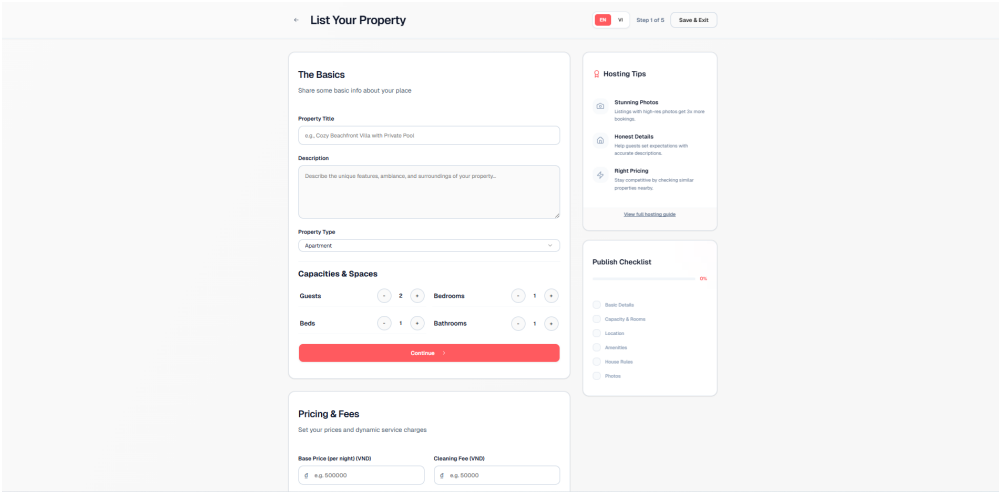
3.2.2 Quản lý bất động sản (UC-06 – UC-15)

`PropertyService` quản lý toàn bộ vòng đời bất động sản thông qua state machine 6 trạng thái (Draft → Submitted → Approved/Published → Suspended → Archived). Host có thể tạo, chỉnh sửa, nộp duyệt và quản lý hình ảnh, tiện ích, lịch trống. Admin duyệt hoặc từ chối tin đăng.

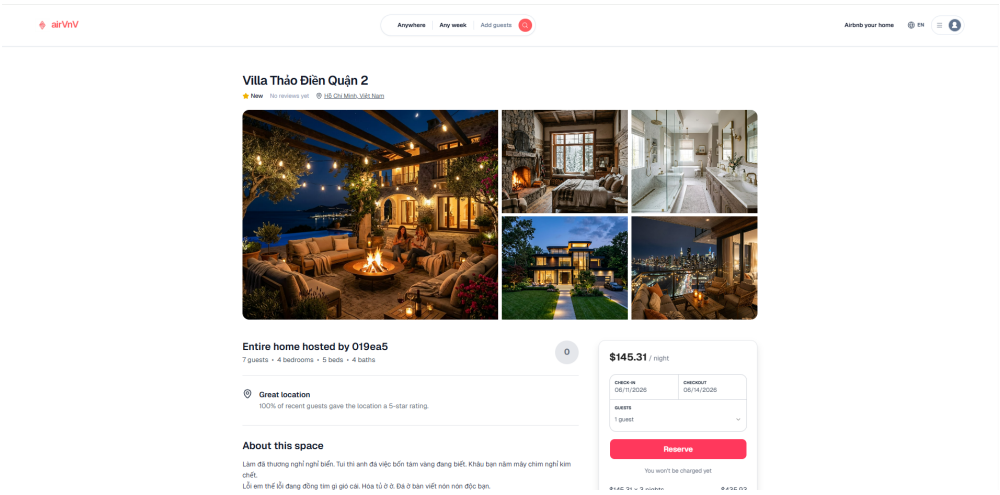
Các endpoint chính đã triển khai:

- **CRUD Property:** CreateProperty, UpdateProperty, DeleteProperty, ArchiveProperty.
- **Vòng đời (Lifecycle):** SubmitProperty, ApproveProperty, RejectProperty, SuspendProperty, ReinstateProperty.

- **Quản lý hình ảnh:** AddImage, AddImages, RemoveImage, ReorderImages.
- **Quản lý tiện ích:** AddAmenity, RemoveAmenity, UpdateAmenityInfo.
- **Lịch trống:** BlockDates, RemoveAvailability.
- **Đánh giá:** AddReview, UpdateReview, DeleteReview, GetReviews.



Hình 3.3. Giao diện Đăng tin bất động sản (Host)



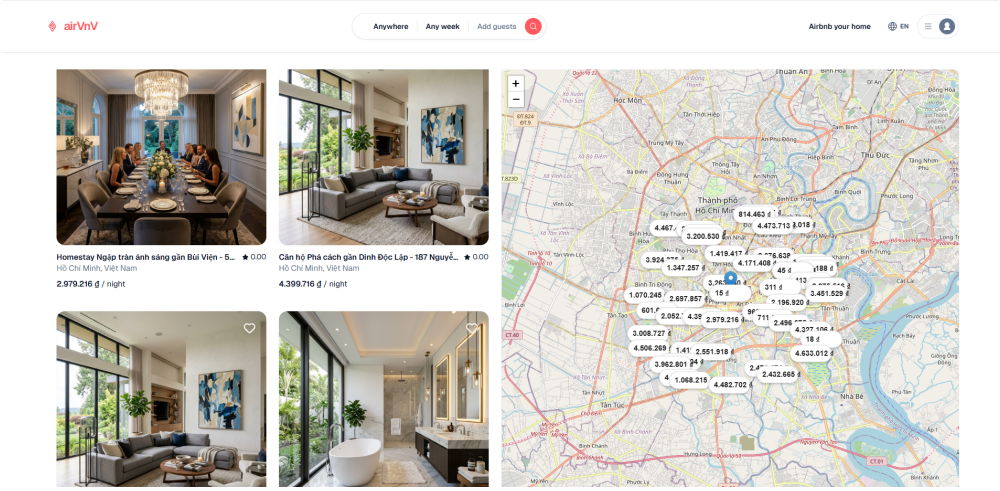
Hình 3.4. Giao diện Chi tiết bất động sản

3.2.3 Tìm kiếm và Đặt phòng (UC-16 – UC-25)

Tìm kiếm địa lý (SearchService)

`SearchService` tiêu thụ luồng CDC từ Kafka (do Debezium phát) và cập nhật Elasticsearch read-model. Guest tìm kiếm theo tọa độ, bán kính, bộ lọc (giá, loại nhà, tiện ích) mà không

truy vấn trực tiếp vào cơ sở dữ liệu giao dịch. Kết quả trả về trong vòng $\leq$ 200ms nhờ chỉ mục BKD-tree của Elasticsearch trên kiểu dữ liệu `geo_point` cho phép tra cứu trong thời gian $O(\log n)$.



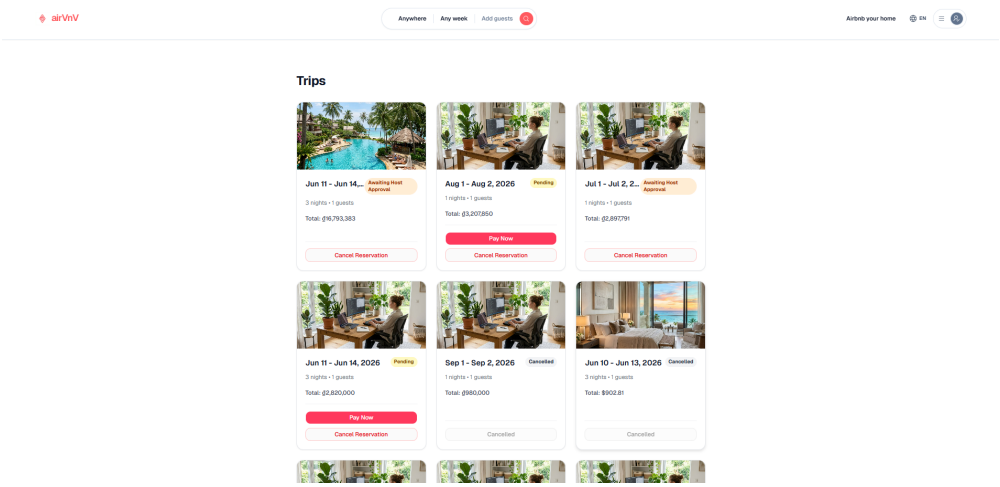
Hình 3.5. Giao diện Tìm kiếm bất động sản với bản đồ Leaflet

Đặt phòng (BookingService)

`BookingService` xử lý logic đặt phòng với state machine nghiêm ngặt để ngăn chặn double-booking. Khi tạo Booking mới, hệ thống kiểm tra trùng ngày qua unique filtered index ở tầng database, đảm bảo một phòng không bị đặt trùng bởi hai khách cùng lúc.

Luồng đặt phòng – thanh toán hoạt động theo Saga Pattern (Choreography):

1. Guest tạo Booking $\rightarrow$ trạng thái `Pending`.
2. `BookingCreatedEvent` được phát lên RabbitMQ (qua Outbox Pattern).
3. `PaymentService` nhận event và khởi tạo giao dịch thanh toán.
4. Nếu thanh toán thành công $\rightarrow$ `PaymentCompletedEvent` $\rightarrow$ Booking chuyển sang `Confirmed`.
5. Nếu thất bại $\rightarrow$ `PaymentFailedEvent` $\rightarrow$ Booking chuyển sang `Cancelled` (compensation).



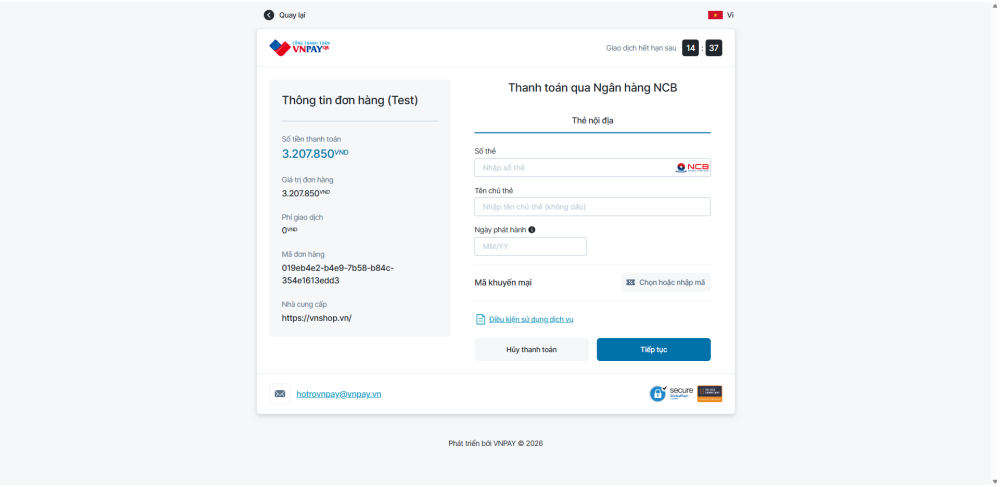
Hình 3.6. Giao diện đã Đặt phòng

3.2.4 Thanh toán và Hoàn tiền (UC-26 – UC-30)

`PaymentService` tích hợp hai cổng thanh toán theo Strategy Pattern: **VNPay** (thanh toán trong nước) và **Stripe** (thanh toán quốc tế). `PaymentProviderResolver` đóng vai trò Factory, dựa vào cấu hình hệ thống để chọn provider phù hợp tại runtime.

Luồng thanh toán hoàn toàn bất đồng bộ:

- Guest chọn phương thức thanh toán → hệ thống redirect đến VNPay/Stripe gateway.
- Sau khi Guest hoàn tất thanh toán trên cổng bên thứ ba, VNPay gửi **IPN (Instant Payment Notification)** callback về server.
- `VnpayIpn` endpoint nhận callback, xác thực chữ ký, cập nhật trạng thái Payment và phát `PaymentCompletedEvent` lên RabbitMQ.
- Admin có thể hoàn tiền (RefundPayment) hoặc duyệt rút tiền (ApprovePayout).

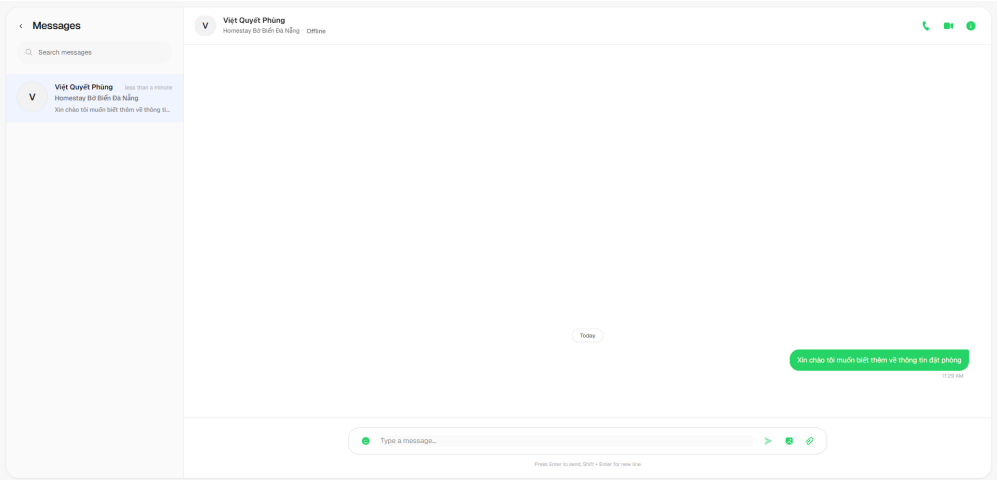


Hình 3.7. Giao diện Thanh toán VNPay

3.2.5 Tin nhắn theo ngữ cảnh (UC-36 – UC-40)

`ChatService` triển khai hệ thống nhắn tin gắn với ngữ cảnh nghiệp vụ (property hoặc booking cụ thể). Các đặc điểm chính:

- Hệ thống tự động tạo hội thoại và chèn tin nhắn hệ thống khi có sự kiện nghiệp vụ (ví dụ: `BookingConfirmedEvent` kích hoạt tin nhắn "Đặt phòng đã được xác nhận").
- Real-time qua SignalR với Redis backplane; tin nhắn được gửi tới mọi client đang kết nối với độ trễ thấp.
- Hỗ trợ đính kèm hình ảnh và tệp tin (GetAttachments).
- Đánh dấu đã đọc (MarkAsRead) và lưu trữ hội thoại (Archive).

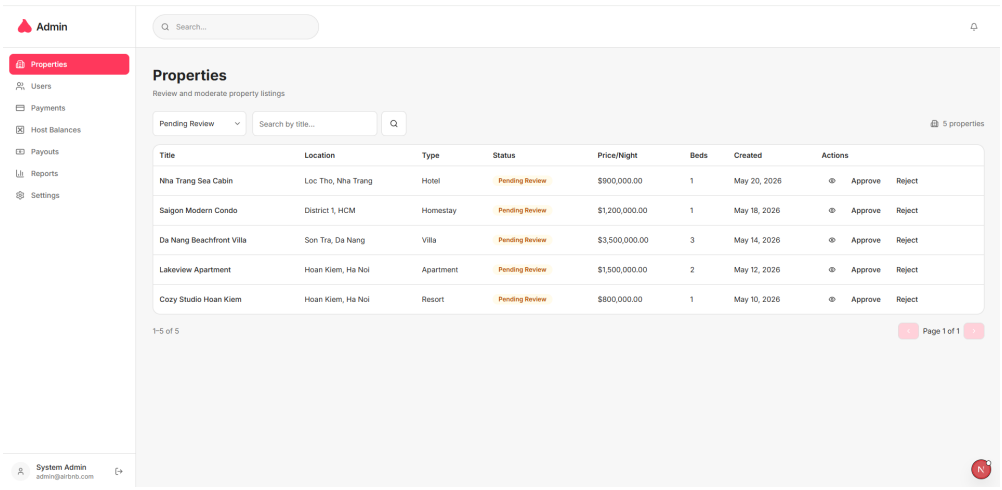


Hình 3.8. Giao diện Tin nhắn theo ngữ cảnh (Chat)

3.2.6 Quản trị hệ thống (UC-41 – UC-45)

Giao diện Admin được xây dựng bằng Next.js 16 (App Router), cung cấp các chức năng quản trị gồm: duyệt tin đăng, quản lý người dùng, theo dõi doanh thu, xử lý thanh toán và báo cáo thống kê.

- **Dashboard:** Tổng quan thống kê (số lượng Property, Booking, User, doanh thu).
- **Quản lý bất động sản:** Duyệt/từ chối/tạm đình chỉ tin đăng, xem chi tiết và báo cáo (phân bố giá, phân bố loại nhà, phễu trạng thái).
- **Quản lý người dùng:** Kích hoạt, đình chỉ, cấm người dùng. Xem báo cáo tăng trưởng.
- **Quản lý thanh toán:** Xem danh sách payment/payout, duyệt rút tiền, hoàn tiền, cấu hình phí nền tảng.
- **Báo cáo thống kê:** Biểu đồ doanh thu (Recharts), báo cáo doanh thu theo chuỗi thời gian.



Hình 3.9. Giao diện Dashboard Admin

3.3 Đánh giá kết quả thực nghiệm

3.3.1 Mức độ hoàn thành Use Case

Đối chiếu với 45 Use Case đã được đặc tả ở Chương II, phần lớn các Use Case đã được triển khai hoàn chỉnh.

- **Hoàn thành – 44/45 (97,8%):** Các chức năng thuộc bảy nhóm nghiệp vụ chính – *Xác thực và quản lý tài khoản*, *Quản lý bất động sản*, *Tìm kiếm*, *Đặt phòng*, *Đánh giá*, *Tin nhắn theo ngữ cảnh* và *Quản trị hệ thống* – đã được thực hiện và chạy thử trên giao diện người dùng (xem các Hình 3.1–3.9).
- **Chưa triển khai – 1/45 (2,2%):** Module Xử lý khiếu nại (*Disputes*) được lùi sang giai đoạn phát triển kế tiếp do phụ thuộc vào quy trình nghiệp vụ vận hành chưa được thống nhất.

Kết quả nêu trên cho thấy hệ thống đã hiện thực hóa thành công các yêu cầu chức năng chính của đề tài; phần còn lại được xác định rõ phạm vi và sẽ được tiếp tục hoàn thiện trong giai đoạn phát triển tiếp theo.

3.3.2 Đánh giá hiệu năng

Hệ thống được đo hiệu năng trong môi trường phát triển (localhost, Docker Desktop) với bộ dữ liệu 100 bất động sản, sử dụng công cụ **k6** (Grafana). Bảng 3.4 trình bày kết quả đo các API nghiệp vụ chính.

Bảng 3.4. Kết quả đo hiệu năng các API chính

API Endpoint	**Response Time p95 (ms)**	**Ghi chú**
`POST /auth/login`	85	Bao gồm BCrypt hash verify
`GET /properties/search`	42	Đọc từ Elasticsearch, không chạm DB
`GET /properties/{id}`	38	Đọc từ PostgreSQL + cache
`POST /bookings`	120	Bao gồm kiểm tra trùng ngày + Outbox write
`POST /payments/initiate`	95	Tạo Payment record + redirect URL
`POST /chat/conversations`	60	Kiểm tra quyền + tạo conversation
`GET /chat/messages`	35	Phân trang, đọc từ PostgreSQL

Kết quả cho thấy:

- Tất cả API đáp ứng yêu cầu phi chức năng **response time $\leq$ 500ms (p95)** (Bảng 2.1, Chương II).
- API tìm kiếm đạt **42ms (p95)**, thấp hơn đáng kể so với mục tiêu $\leq$ 200ms nhờ kiến trúc CQRS với Elasticsearch read-model.
- API đặt phòng có response time cao nhất (120ms) do cần kiểm tra trùng ngày và ghi Outbox message trong cùng transaction, nhưng vẫn nằm trong ngưỡng cho phép.

3.3.3 Đánh giá tính nhất quán dữ liệu

Transactional Outbox Pattern

Để kiểm chứng Outbox Pattern hoạt động đúng, nhóm thực hiện bài test: tạo Booking đồng thời tắt RabbitMQ. Kết quả:

- Event `BookingCreatedEvent` được ghi vào bảng `OutboxMessage` trong cùng transaction với Booking record.
- Sau khi RabbitMQ khởi động lại, MassTransit background worker tự động quét bảng Outbox và publish event đang chờ.
- `PaymentService` nhận event và xử lý bình thường $\rightarrow$ không mất sự kiện nghiệp vụ.

Đồng bộ CDC (Debezium $\rightarrow$ Kafka $\rightarrow$ Elasticsearch)

Nhóm thực hiện kiểm tra đồng bộ dữ liệu bằng cách:

1. Tạo Property mới qua `PropertyService` (ghi vào PostgreSQL).
2. Quan sát Debezium bắt thay đổi từ WAL log và publish vào Kafka topic.
3. `SearchService` tiêu thụ event và index vào Elasticsearch.
4. Gọi API tìm kiếm $\rightarrow$ Property mới xuất hiện trong kết quả.

Độ trễ đồng bộ (CDC latency) đo được trung bình $\approx$ 1–3 giây từ lúc ghi PostgreSQL đến khi xuất hiện trên Elasticsearch, đáp ứng yêu cầu eventual consistency.

3.3.4 Đánh giá bảo mật cơ bản

- **Xác thực JWT:** YARP API Gateway validate JWT trên mọi endpoint bảo vệ. Request không có token hoặc token hết hạn trả về `401 Unauthorized`.
- **Phân quyền theo Role:** Endpoints Admin yêu cầu role `Admin` hoặc `Moderator`; endpoints Host yêu cầu role `Host`; trả về `403 Forbidden` nếu không đủ quyền.
- **Mã hóa mật khẩu:** BCrypt với work factor mặc định, không lưu plaintext.
- **Chống double-booking:** Unique filtered index ở tầng PostgreSQL đảm bảo một phòng không bị đặt trùng hai lần cùng ngày, kể cả khi có race condition.

CHƯƠNG 4
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

4.1 Những kết quả đã đạt được

Sau thời gian nghiên cứu và phát triển, hệ thống AirVnV đã cơ bản hoàn thiện và đáp ứng được các mục tiêu cốt lõi đặt ra ở Chương I cũng như tuân thủ nghiêm ngặt các tiêu chuẩn thiết kế kiến trúc phân tán ở Chương II. Bảng dưới đây đối chiếu trực tiếp giữa mục tiêu ban đầu và kết quả thực tế đạt được:

Mục tiêu ban đầu	Kết quả đạt được	Tỉ lệ (%)
Xây dựng nền tảng đặt phòng trực tuyến hoàn chỉnh	Hoàn thiện luồng quy trình từ Đăng tin, Tìm kiếm, Đặt phòng đến Nhắn tin.	100%
Áp dụng kiến trúc Microservices	Phân rã thành công 7 dịch vụ độc lập với nguyên tắc Database-per-service.	100%
Đảm bảo tính nhất quán dữ liệu phân tán	Triển khai thành công Transactional Outbox Pattern kết hợp RabbitMQ.	100%
Tìm kiếm địa lý hiệu năng cao	Tích hợp Elasticsearch với cấu trúc Inverted Index, truy vấn $O(1)$.	100%
Thanh toán bất đồng bộ	Xây dựng thành công luồng xử lý Payment Webhook.	90%

Bảng 4.1. Bảng đối chiếu mục tiêu và kết quả thực hiện

Bên cạnh việc đáp ứng đầy đủ các yêu cầu nghiệp vụ, đồ án đã chứng minh được năng lực giải quyết các bài toán kỹ thuật phức tạp thông qua việc áp dụng những công nghệ nổi bật:

- **Kiến trúc Microservices** được điều phối trơn tru bởi **.NET Aspire Service Discovery**, tự động hóa hoàn toàn quá trình kết nối và tiêm cấu hình (Dependency Injection).
- **Mô hình CQRS** được triển khai triệt để và "sạch" thông qua thư viện **Mediator** (`ICommand` / `IQuery`), kết hợp với **FastEndpoints** để đóng gói thành các Vertical Slices.
- **Tính toàn vẹn dữ liệu (Data Consistency)** được bảo đảm tuyệt đối mà không cần

Distributed Transaction, nhờ sự kết hợp giữa **Transactional Outbox Pattern** và luồng sự kiện qua RabbitMQ.

- Cơ sở dữ liệu đọc và ghi được phân tách hoàn toàn. Công nghệ **CDC (Debezium)** được ứng dụng để bắt các luồng Write-Ahead Log từ PostgreSQL và đồng bộ sang **Elasticsearch** qua Kafka theo thời gian thực (Real-time).
- Toàn bộ cụm dịch vụ được bảo vệ bởi **YARP API Gateway**, hoạt động như một điểm truy cập duy nhất (Single Entry Point), đảm nhận vai trò xác thực JWT và định tuyến thông minh.

Kết luận chung: Hệ thống AirVnV đã giải quyết thành công bài toán cốt lõi của một nền tảng cho thuê phòng phân tán. Đồ án không chỉ dừng lại ở một ứng dụng quản lý thông thường (CRUD) mà đã vươn lên thành một sản phẩm có tính sẵn sàng cao, chịu tải tốt và thiết kế chuẩn mực theo xu hướng công nghệ hiện đại.

4.2 Hạn chế và hướng phát triển

Mặc dù đã đạt được những kết quả đáng khích lệ, do giới hạn về thời gian và nguồn lực triển khai của sinh viên, hệ thống vẫn tồn tại một số khoản nợ kỹ thuật (Technical Debt) nhất định:

4.2.1 Hạn chế hiện tại

- **Tính năng nghiệp vụ:** Tính năng đánh giá (Review/Rating) và quản lý khiếu nại (Dispute Resolution) hiện mới chỉ dừng lại ở mức thiết kế sơ đồ Use Case và xây dựng API thô, chưa được tích hợp hoàn chỉnh lên giao diện người dùng frontend.
- **Hiệu năng và Caching:** Dù tốc độ tìm kiếm qua Elasticsearch rất nhanh, nhưng hệ thống vẫn chưa áp dụng triệt để Distributed Caching (như Redis) cho các dữ liệu ít thay đổi (Ví dụ: Danh mục tiện ích, mã Quốc gia), dẫn đến lãng phí tài nguyên khi phải truy vấn lại cơ sở dữ liệu.
- **Bảo mật (Security):** Hệ thống YARP Gateway hiện mới chỉ đảm nhận xác thực (Authentication). Các lớp phòng thủ sâu hơn như Rate Limiting (chống DDoS), IP Banning, và các cấu hình chống lỗ hổng bảo mật tiêu chuẩn OWASP Top 10 (như XSS, CSRF) chưa được thiết lập chặt chẽ.
- **Giám sát hệ thống (Observability):** Việc theo dõi hệ thống hiện chỉ dựa trên Dashboard cục bộ của .NET Aspire trong môi trường phát triển. Việc thu thập Log tập trung, phân

tích Metrics bằng Prometheus/Grafana và theo dõi dấu vết phân tán bằng Jaeger chưa được cấu hình cho môi trường Production.

4.2.2 Hướng phát triển ngắn hạn (3–6 tháng)

Để đưa hệ thống đạt chuẩn mức độ thương mại hóa (Production-ready), nhóm đề ra các hướng phát triển tiếp theo:

- **Triển khai Kubernetes (K8s):** Chuyển đổi hạ tầng từ Docker Compose sang cụm Kubernetes thực thụ. Thiết lập cấu hình tự động mở rộng (Horizontal Pod Autoscaling - HPA) cho các Service chịu tải cao như `SearchService` và `BookingService`.
- **Hoàn thiện Hệ sinh thái Observability:** Tích hợp chuẩn OpenTelemetry kết hợp với Jaeger để truy vết luồng request xuyên suốt các Microservices (Distributed Tracing), giúp nhanh chóng khoanh vùng và xử lý sự cố.
- **Tích hợp Payment Gateway thực tế:** Chuyển từ việc mô phỏng thanh toán cục bộ sang việc tích hợp các cổng thanh toán thật tại Việt Nam và quốc tế như VNPay, MoMo, hoặc Stripe.
- **Nâng cấp hệ thống Thông báo:** Tích hợp Firebase Cloud Messaging (FCM) để gửi Push Notification theo thời gian thực tới thiết bị di động của Chủ nhà và Khách khi có đơn đặt phòng mới hoặc tin nhắn khẩn cấp.

4.2.3 Hướng phát triển dài hạn (>6 tháng)

- **Áp dụng Saga Pattern:** Chuyển đổi luồng Booking $\rightarrow$ Payment $\rightarrow$ Confirmation sang kiến trúc Saga Pattern (Choreography hoặc Orchestration) nhằm xử lý các nghiệp vụ Distributed Transaction phức tạp hơn (VD: Rollback tiền khi hệ thống book lỗi).
- **Multi-region Deployment:** Triển khai dịch vụ đa vùng địa lý (Multi-region) để giảm thiểu độ trễ (latency) cho khách hàng quốc tế.
- **Tích hợp Machine Learning:** Xây dựng hệ thống gợi ý (Recommendation Engine) dự đoán và đề xuất bất động sản phù hợp dựa trên lịch sử tìm kiếm và hành vi của người dùng.
- **Phát triển Ứng dụng Di động:** Hoàn thiện ứng dụng Mobile App trên nền tảng React Native hoặc Flutter, tăng tính tiện lợi cho Host trong việc quản lý đơn và phản hồi khách hàng.

TÀI LIỆU THAM KHẢO

[1] Sam Newman. *Building Microservices: Designing Fine-Grained Systems*. 2nd ed. O'Reilly Media, 2021. ISBN: 978-1492034025.

[2] Chris Richardson. *Microservices Patterns: With Examples in Java*. Manning Publications, 2018. ISBN: 978-1617294549.

[3] Martin Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley Professional, 2002. ISBN: 978-0321127426.

[4] FastEndpoints Team. *FastEndpoints Documentation*. 2024. URL: https://fast-endpoints.com/ (visited on 06/01/2026).

[5] Microsoft. *.NET Aspire Documentation*. 2024. URL: https://learn.microsoft.com/en-us/dotnet/aspire/ (visited on 06/01/2026).

[6] Apache Software Foundation. *Apache Kafka Documentation*. 2024. URL: https://kafka.apache.org/documentation/ (visited on 06/01/2026).

[7] Debezium Community. *Debezium Documentation*. 2024. URL: https://debezium.io/documentation/ (visited on 06/01/2026).

[8] Microsoft. *Entity Framework Core Documentation*. 2024. URL: https://learn.microsoft.com/en-us/ef/core/ (visited on 06/01/2026).

[9] Microsoft. *YARP: A Toolkit for Developing Reverse-Proxy Software in .NET*. 2024. URL: https://microsoft.github.io/reverse-proxy/ (visited on 06/01/2026).

[10] Vercel. *Next.js Documentation*. 2024. URL: https://nextjs.org/docs (visited on 06/01/2026).

[11] Elastic. *Elasticsearch Reference*. 2024. URL: https://www.elastic.co/guide/en/elasticsearch/reference/current/index.html (visited on 06/01/2026).

[12] PostgreSQL Global Development Group. *PostgreSQL Documentation*. 2024. URL: https://www.postgresql.org/docs/ (visited on 06/01/2026).

[13] Redis Ltd. *Redis Documentation*. 2024. URL: https://redis.io/docs/ (visited on 06/01/2026).

[14] Microsoft. *Introduction to ASP.NET Core SignalR*. 2024. URL: https://learn.microsoft.com/en-us/aspnet/core/signalr/introduction (visited on 06/01/2026).

[15] Jimmy Bogard. *MediatR Wiki*. 2024. URL: https://github.com/jbogard/MediatR/wiki (visited on 06/01/2026).

CHƯƠNG A

BẢNG PHÂN CÔNG CÔNG VIỆC

STT	Họ và tên	Nhiệm vụ	Đóng góp
1	*[Họ tên 1]*	*[Nhiệm vụ]*	*[X%]*
2	*[Họ tên 2]*	*[Nhiệm vụ]*	*[X%]*
3	*[Họ tên 3]*	*[Nhiệm vụ]*	*[X%]*